



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Single-core and Multi-core Efficient Execution of QNN on Edge AI Processors

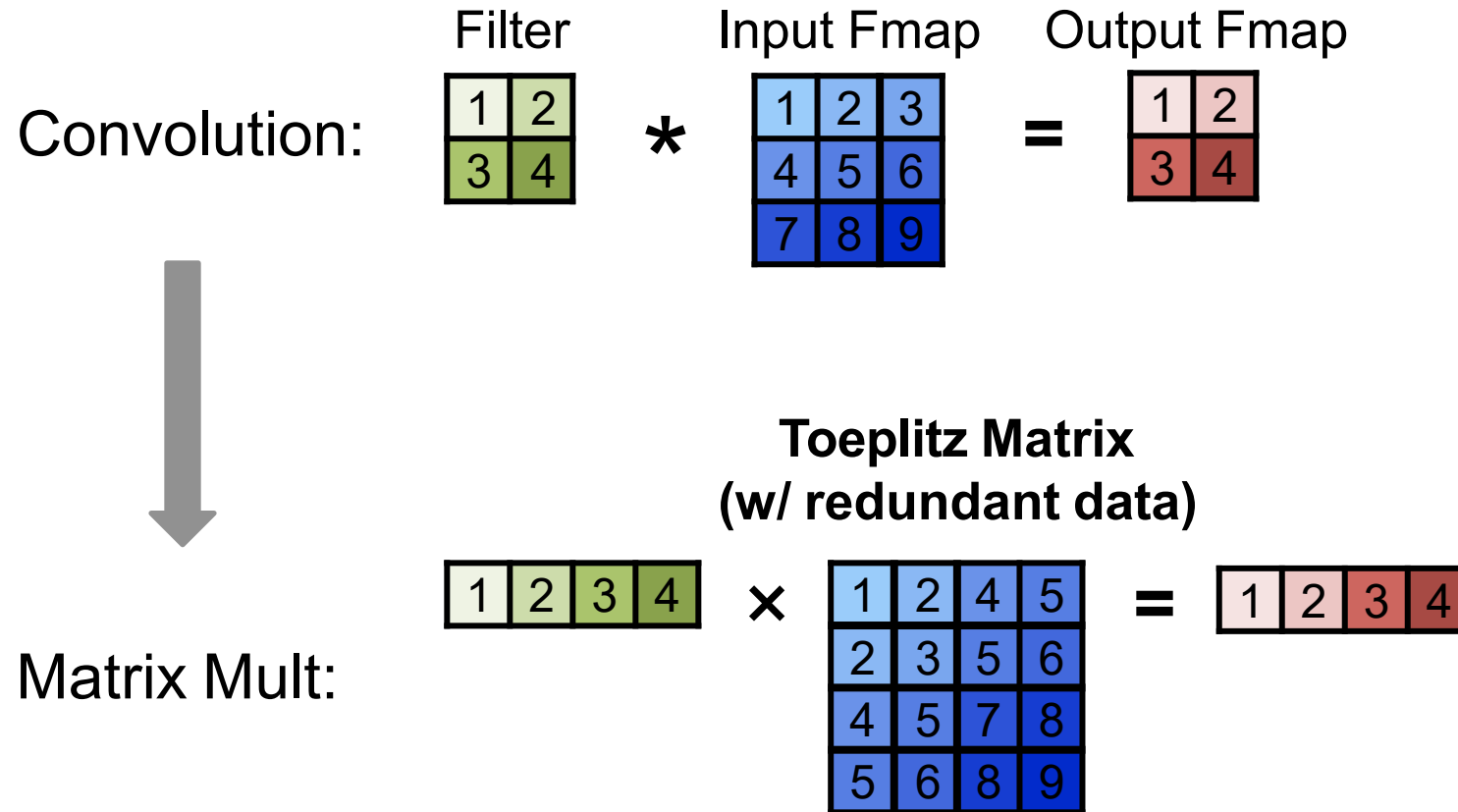
Prof. Luca Benini and prof. **Angelo Garofalo**

Outline

- Convolution as Matrix Multiplication kernel
- Characteristics and Metrics of Edge AI Processors
- Recap of ISA (RISC-V) and processors with focus on Data processing
- How to specialize RISC-V processors (ISAs and uArchitecture) for AI
- Fundamentals of parallel processor architecture
- Parallel Execution of AI Workloads and speedup analysis
- Introduction to Tensor Tiling

Convolution (CONV) Layer as a Matrix Mult

- Convert to matrix mult. using the **Toeplitz Matrix**



Convolution (CONV) Layer as Matrix Mult

- Convert to matrix mult. using the **Toeplitz Matrix**

Convolution:

$$\begin{array}{|c|c|} \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array}$$



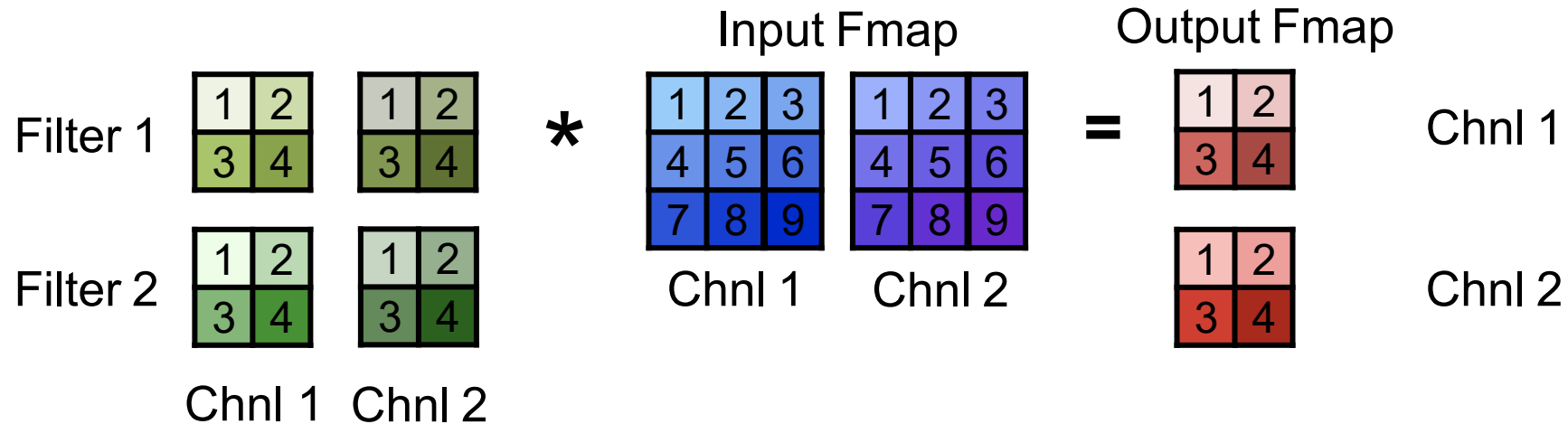
Matrix Mult:

$$\begin{array}{|c|c|c|c|} \hline 1 & 2 & 3 & 4 \\ \hline \end{array} \times \begin{array}{|c|c|c|c|} \hline 1 & 2 & 4 & 5 \\ \hline 2 & 3 & 5 & 6 \\ \hline 4 & 5 & 7 & 8 \\ \hline 5 & 6 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 1 & 2 & 3 & 4 \\ \hline \end{array}$$

Data is repeated

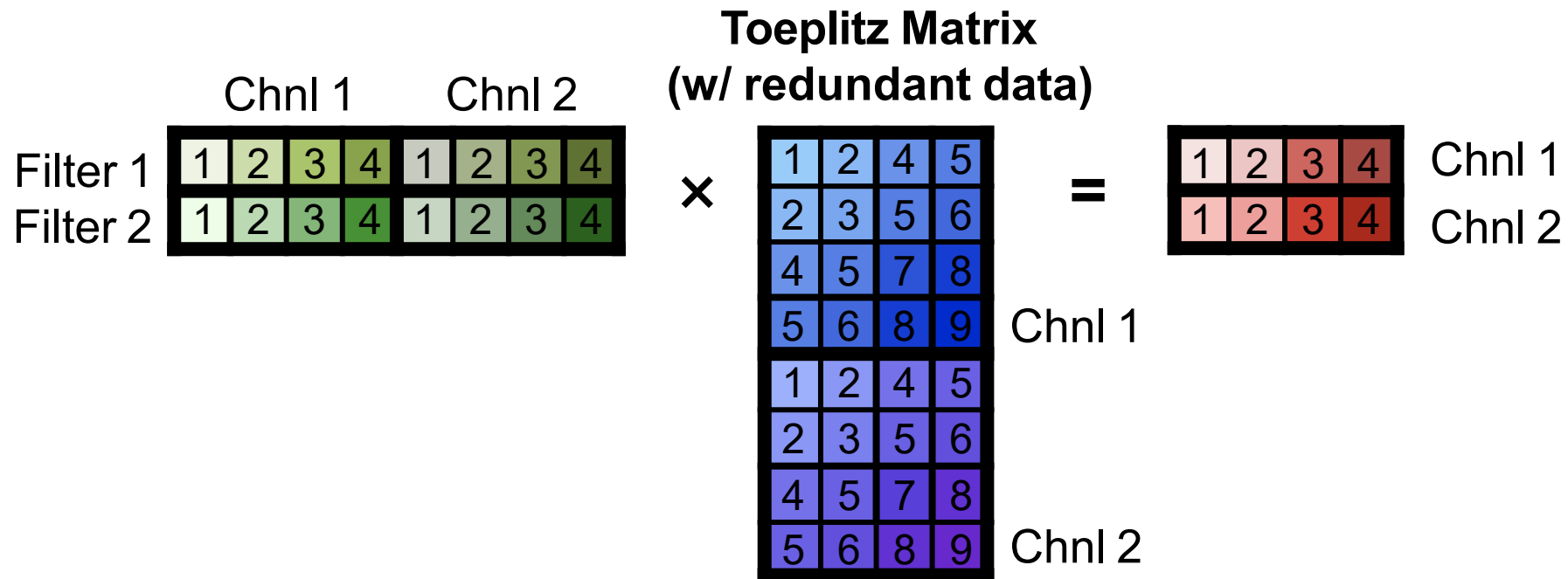
Convolution (CONV) Layer as Matrix Mult

- Multiple Channels and Filters



Convolution (CONV) Layer as Matrix Mult

- Multiple Channels and Filters



Data duplication + Memory Bandwidth are major concerns
But fundamentally, the operation needed is DOT PRODUCT

$$O = r \cdot c = \sum_{i=1}^N c_i w_i$$

Programmable Edge AI Processors

Metrics	GPUs	Edge AI Processors (MCUs for TinyML)
Power Budget	~[10 - 300 W]	~[10 mW - 5 W]
Op. Frequency	~[1 - 4 GHz]	~[1 - 500 MHz]
Throughput (Max)	Up to 100+ TFLOPs	~[1 - 5 GFLOPs]
Energy Efficiency (Max)	~[1 - 10 GFLOPs/W]	~[10 - 100+ GFLOPs/W]
Silicon Area	~[100 - 800 mm ²]	~[1 - 50 mm ²]
Performance	High (for AI, parallel tasks)	Moderate (for TinyML tasks)
Flexibility	Very flexible (broad support)	Optimized for specific AI tasks
Cost	High (\$100 - \$500+)	Low (\$1 - \$50)
Use Case	High-performance AI, Deep Learning, Data Centers, Gaming	Low-power AI inference, IoT, Real-time Edge Computing

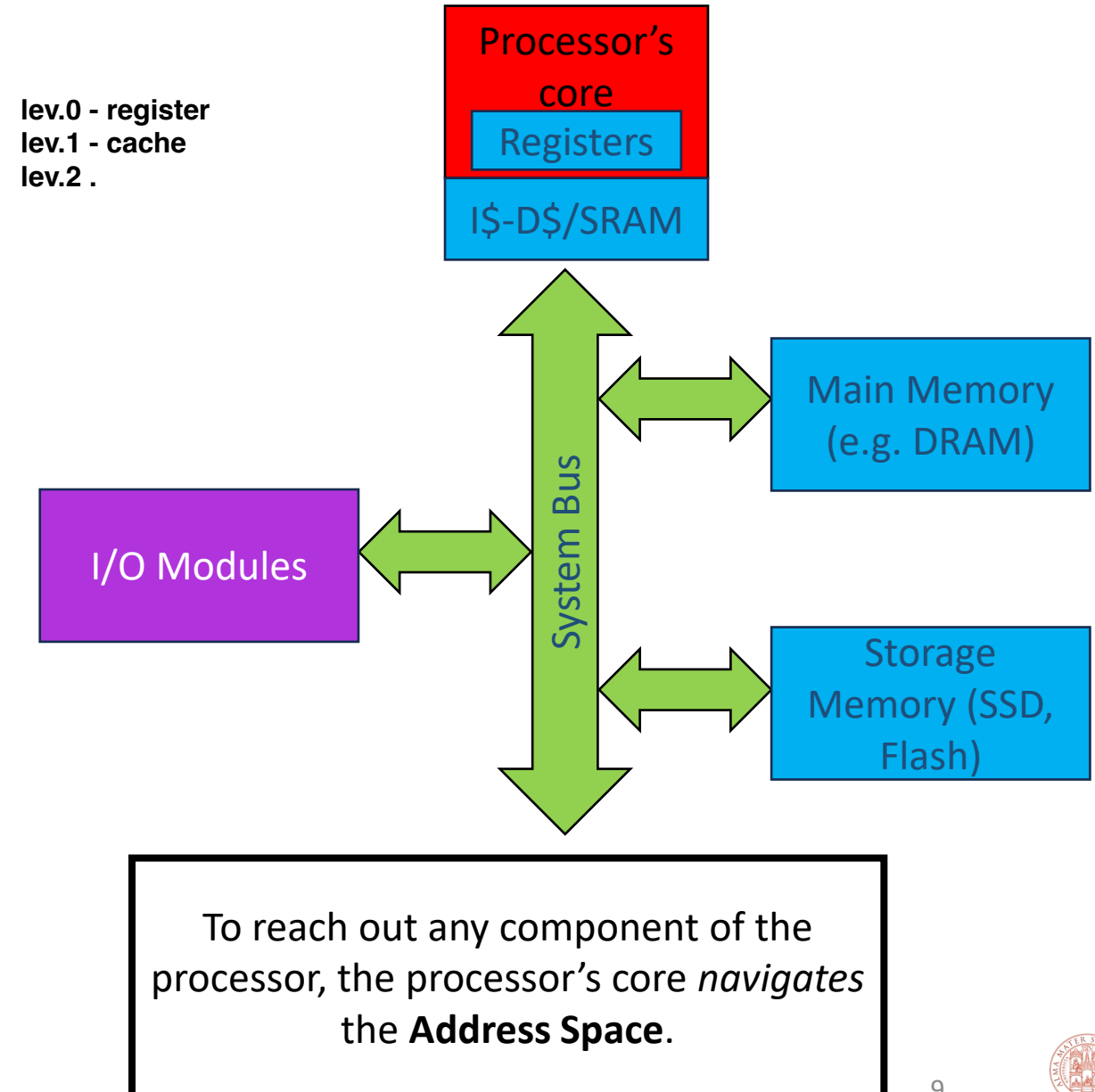
Edge AI processors are mainly used for battery-powered embedded systems. Therefore they must be:

- **Cheap and Small** → limited on-chip memory and compute resources
- **Low-Power and Energy Efficient** → simple and efficient architectural solutions; hw/sw co-design for max efficiency
- **Perform control tasks other than AI** → software-programmable, with hw/sw specialization for AI

(Very) Quick Recap on Basics of Processors and ISA

The Basic Processors' Components

- **Processor's core**
 - It executes code and processes data
 - Often associated with temporary memory like registers and caches (both instruction cache (I\$) and data cache D\$)
- **Main memory**
 - Support processor's internal memory: code and data are temporary and permanently stored (e.g. DRAM)
- **Storage Memory**
 - Usually non-volatile memory where to store program code and/or permanent data (e.g. Flash)
- **System bus**
 - The interconnection interface among all the different components, where the on-chip communication happens
 - The bus carries Addresses, Data and Control Signals
- **Input/Output**
 - Provide interfaces between the processor and the user



How Do We Execute Code/Algorithms on a Processor?

High-level (math)

Let's assume we want to execute a simple math operations ('+') among two integers: A and B



Lower-level of abstraction (C code)

From the high-level problem we describe it in a more machine-friendly language. C code is widely used for embedded systems.



Lower-level of abstraction (assembly)

We then translate the C code into into a machine-friendly language, the assembly code, i.e. into a list of basic operations (instructions) that the processors can execute



Executable

We represent the list of instructions and data into binary digits (0s and 1s) and we deploy them to the processor's memory.

We want to sum the integer number 3 with the integer number 5 and get the result



```
We declare two variables:  
int A = 3; int B = 5;  
int C;  
We specify the operation:  
C = A + B;
```

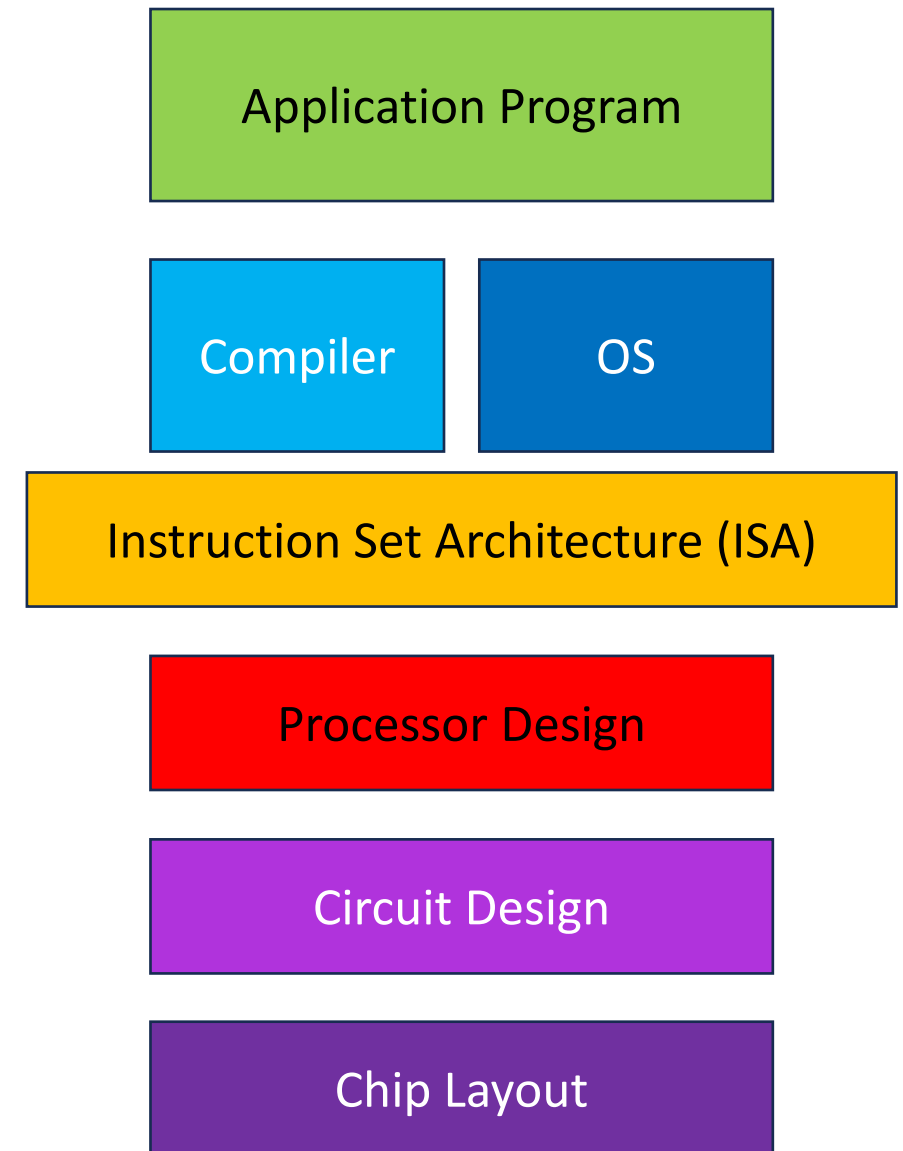


This code is translated into simple instructions that the processor can directly execute:

- load data from MEM at address X0 (A) and store it in R0
- load data from MEM at address X1 (B) and store it in R1
- execute add insn on R0 and R1 and put result in R2
- store R2 into the MEM at address X2 (C)

How do we link the Hardware to the Software?

- The interface between hardware and software is defined by the **Instruction Set Architecture (ISA)** of a processor.
- Understanding the **ISA** is of extreme importance for both software developers and hardware designers → it is crucial for Embedded System Designers
 - It is the bridge between hardware and software
 - It helps develop optimized and efficient software (really a deep breath for embedded resource constrained systems)



The RISC-V ISA

- **Open-source** RISC Instruction Set ARchitecture
- Developed at UC Berkeley as open ISA starting in 2010
 - Now managed by the RISC-V Foundation (riscv.org)
- Typical of many modern ISAs (especially for edge processors)
- Similar ISAs have a large share of embedded core market
 - Applications in consumer electronics, network/storage equipment, cameras, printers, ...
- Which are the main characteristics?
 - It's publicly available
 - It's **extensible** with custom instructions → we can increase the skills of GP processors on some specific workloads (like AI)

RISC-V All in One Page

Free & Open RISC-V Reference Card

Base Integer Instructions (RV32I, RV64I, and RV128I)										RV Privileged Instructions									
Category	Name	Format	RV32I Base	RV64I Base	RV128I Base	Category	Name	Format	RV Privileged										
Loads	Load Byte	I	LB	rd, rs1, imm		CSR Access	Atomic R/W	CSRW	rd, csr, rs1										
	Load Halfword	I	LH	rd, rs1, imm			Atomic Read & Set Bit	CSRRS	rd, csr, rs1										
	Load Word	I	LW	rd, rs1, imm	L(D)Q		Atomic Read & Clear Bit	CSRRC	rd, csr, rs1										
	Load Byte Unsigned	I	LBUH	rd, rs1, imm			Atomic R/W	CSRRI	rd, csr, imm										
	Load Half Unsigned	I	LHUH	rd, rs1, imm	L(W)DU		Atomic Read & Clear Bit	CSRRCI	rd, csr, imm										
Stores	Store Byte	S	SB	rs1, rs2, imm		Change Level	Env. Call	ECALL											
	Store Halfword	S	SH	rs1, rs2, imm			Environment B	EBREAK											
	Store Word	S	SW	rs1, rs2, imm	S(D)Q		Environment B	EBREAK											
Shifts	Shift Left	R	SLL	rd, rs1, rs2	SLL(W)Q	Trap Redirect	Trap to Supervisor	TRAP											
	Shift Left Immediate	I	SLLI	rd, rs1, shamt	SLLI(W)Q		Redirect Trap to Hypervisor	TRAPH											
	Shift Right	R	SRL	rd, rs1, rs2	SRL(W)Q		Hypervisor Trap to Supervisor	TRAPH											
	Shift Right Immediate	I	SRLI	rd, rs1, shamt	SRLI(W)Q		Interrupt	Wait for Interrupt	WFI										
	Shift Right Arithmetic	R	SRA	rd, rs1, rs2	SRA(W)Q		MMU	Supervisor FENCE	FENCE_VM	rs1									
Arithmetic	ADD	I	ADD	rd, rs1, rs2	ADD(W)Q	Optional Compressed (16-bit) Instruction Extension: RVC													
	ADD Immediate	I	ADDI	rd, rs1, imm	ADDI(W)Q														
	SUBTRACT	I	SUB	rd, rs1, rs2															
	Load Upper Imm	I	LUI	rd, imm															
	Add Upper Imm to PC	I	AUIPC	rd, imm															
Logical	XOR	I	XOR	rd, rs1, rs2		Loads	Load Word	CL	C.LW rd', rs1', imm*4										
	XOR Immediate	I	XORI	rd, rs1, imm			Load Word SP	CL	C.LWSP rd', sp, imm*4										
	OR	I	OR	rd, rs1, rs2			Load Double	CL	C.LD rd', rs1', imm*8										
	OR Immediate	I	ORI	rd, rs1, imm			Load Double SP	CL	C.LDSP rd', sp, imm*8										
	AND	I	AND	rd, rs1, rs2			Load Quad	CL	C.LQ rd', rs1', imm*16										
Compare	Set <	I	SLT	rd, rs1, rs2		Stores	Store Word	CS	C.SW rs2', rs2', imm*4										
	Set < Immediate	I	SLTI	rd, rs1, imm			Store Word SP	CS	C.SWSP rs2', sp, imm*4										
	Set < Unsigned	I	SLTU	rd, rs1, rs2			Store Double	CS	C.SD rs2', rs2', imm*8										
	Set < Imm Unsigned	I	SLTUI	rd, rs1, imm			Store Double SP	CS	C.SDSP rs2', sp, imm*8										
	Branch	I	BEQ	rs1, rs2, imm			Store Quad	CS	C.SQ rs2', rs2', imm*16										
Branches	Branch ne	I	BNE	rs1, rs2, imm		Arithmetic	Store Quad SP	CS	C.SQSP rs2', sp, imm*16										
	Branch <	I	BLT	rs1, rs2, imm			ADD Word	CR	C.ADDW rd, rs1										
	Branch <=	I	BLTU	rs1, rs2, imm			ADD Immediate	CR	C.ADDIW rd, imm										
	Branch >	I	BGT	rs1, rs2, imm			ADD Word I	CR	C.ADDIW rd, imm										
	Branch >=	I	BGTU	rs1, rs2, imm			ADD SP Imm * 4	CR	C.ADDIW rd, sp, imm*4										
Jump & Link	Jump	I	JAL	rd, rs1, imm		Shifts	Shift Left Imm	CR	C.SLL rd, rs1, imm										
	Jump & Link Register	I	JALR	rd, rs1, imm			Shift Left Imm	CR	C.SLLI rd, rs1, imm										
System	System CALL	I	SCALL			Shifts	Shift Right Imm	CR	C.SRL rd, rs1, imm										
	System BREAK	I	EBREAK				Shift Right Imm	CR	C.SRLI rd, rs1, imm										
Counters	Read CYCLE	I	RDCCYCLE	rd		Jump	Jump	CJ	rd, rs1, imm										
	Read CYCLE upper Half	I	RDCCYCLH	rd			Jump Register	CJ	rd, rs1, imm										
	Read TIME upper Half	I	RDCTIMEH	rd			Jump & Link	CJ	rd, rs1, imm										
	Read TIME	I	RDCTIME	rd			Jump & Link Register	CJ	rd, rs1, imm										
	Read TIME upper Half	I	RDCTIMEH	rd			System Env. BREAK	CJ	rd, rs1, imm										
Counters	Read INSTR RET	I	RDINSTRRET	rd		Shifts	Shift Left Imm	CR	C.SLL rd, rs1, imm										
	Read INSTR upper Half	I	RDINSTRRET	rd			Shift Left Imm	CR	C.SLLI rd, rs1, imm										
	Read INSTR RET	I	RDINSTRRET	rd			Shift Right Imm	CR	C.SRL rd, rs1, imm										
	Read INSTR upper Half	I	RDINSTRRET	rd			Shift Right Imm	CR	C.SRLI rd, rs1, imm										
	Read INSTR RET	I	RDINSTRRET	rd			Shift Right Imm	CR	C.SRLI rd, rs1, imm										

32-bit Instruction Formats

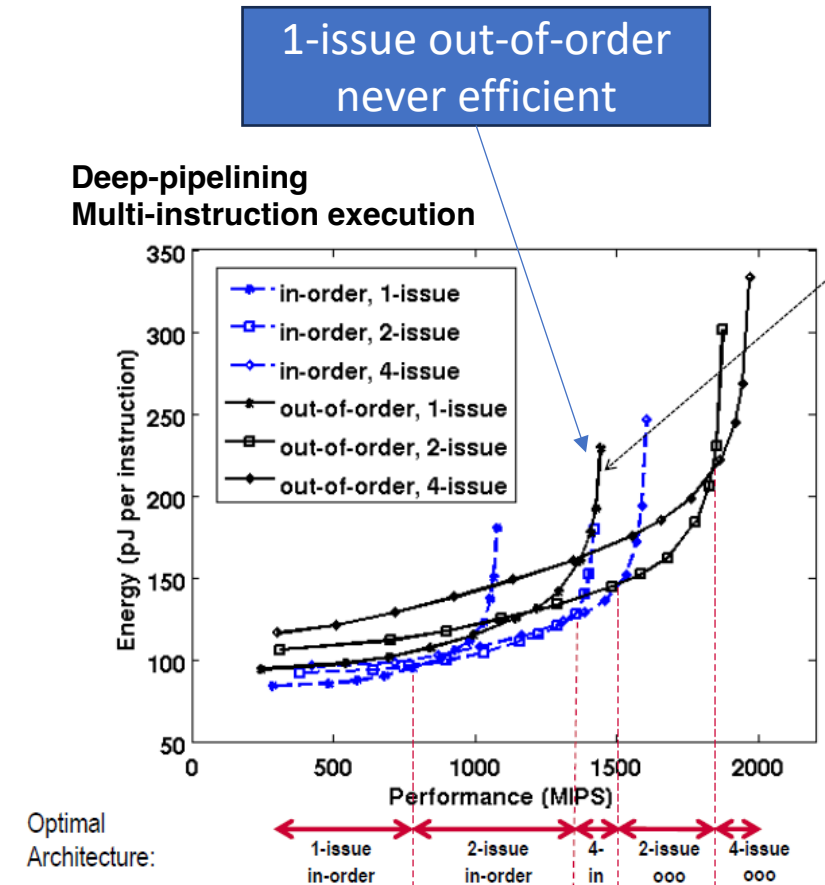
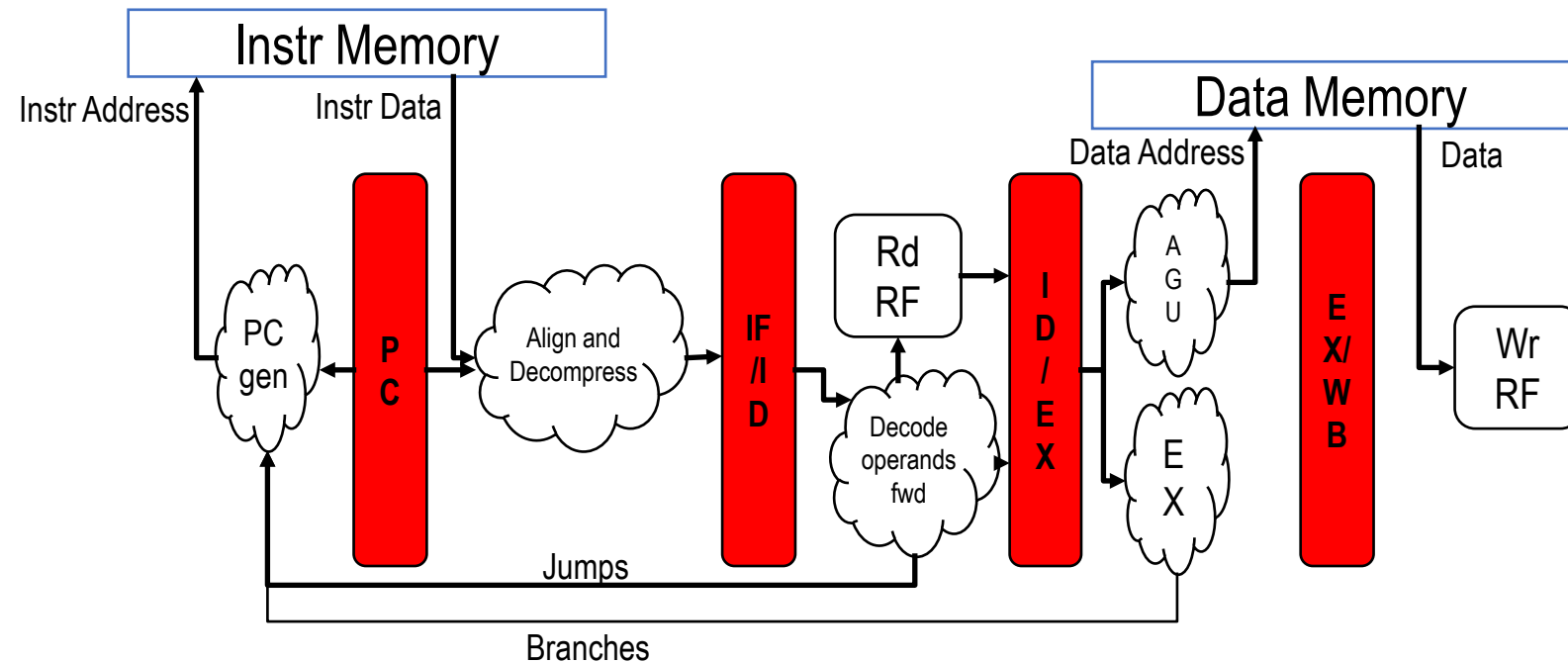
RISC-V Integer Base (RV32I/64I/128I), privileged, and optional compressed extension (RVC). Registers x1-x31 and the pc are 32 bits wide in RV32I, 64 in RV64I, and 128 in RV128I (x0-9). RV64I/128I add 10 instructions for the wider formats. The RVI base of <S0 classic integer RISC instructions is required. Every 16-bit RVC instruction matches an existing 32-bit RVI instruction. See riscv.org.

Optional Atomic Instructions Extension: AQA					Optional Floating-Point Instructions Extension: RFE, RFE2, & RFE3				
Category	Name	Format	RV32M (RV64M, RV128M)	RV32M (RV64M, RV128M)	Category	Name	Format	RV32F (RV64F, RV128F)	RV32F (RV64F, RV128F)
Multiply	Multiply	R	MUL	rd, rs1, rs2	MUL(M)Q	Floating-Point	Load	FLW	rd, rs1, imm
	Multiply upper Half	R	MULH	rd, rs1, rs2	MULH(M)Q		Store	FSW	rs1, rs2, imm
	Multiply Half	R	MULHS	rd, rs1, rs2	MULHS(M)Q		ADD	FADD	rd, rs1, rs2
	Multiply upper	R	MULHU	rd, rs1, rs2	MULHU(M)Q		SUBTRACT	FSUB	rd, rs1, rs2
	Multiply upper	R	MULHU	rd, rs1, rs2	MULHU(M)Q		Multiply	FMUL	rd, rs1, rs2
Divide	DIVIDE	R	DIV	rd, rs1, rs2	DIV(M)Q	Convert	Convert from Int	FCVTW	rd, rs1, rs2
	DIVIDE Unsigned	R	UIV	rd, rs1, rs2	UIV(M)Q		Convert from Int Unsigned	FCVTWU	rd, rs1, rs2
	DIVIDE Unsigned	R	UIV	rd, rs1, rs2	UIV(M)Q		Convert to Int	FCVTW	rd, rs1, rs2
	DIVIDE Unsigned	R	UIV	rd, rs1, rs2	UIV(M)Q		Convert to Int Unsigned	FCVTWU	rd, rs1, rs2
	DIVIDE Unsigned	R	UIV	rd, rs1, rs2	UIV(M)Q		Convert to Int Unsigned	FCVTWU	rd, rs1, rs2
Remainder	REMAINDER	R	REM	rd, rs1, rs2	REM(M)Q	RISC-V Calling Convention	Register	ABI Name	Saver
	REMAINDER	R	REM	rd, rs1, rs2	REM(M)Q		x0	zero	Hard-wired zero
	REMAINDER	R	REM	rd, rs1, rs2	REM(M)Q		x1	ra	Caller
	REMAINDER	R	REM	rd, rs1, rs2	REM(M)Q		x2	sp	Callee
	REMAINDER	R	REM	rd, rs1, rs2	REM(M)Q		x3	gp	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Saved Registers	x4	tp	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x5	sd0	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x6	sd1	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x7	sd2	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x8	sd3	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Arguments/Return Values	x9	sd4	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x10	sd5	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x11	sd6	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x12	sd7	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x13	sd8	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Saved Registers	x14	sd9	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x15	sd10	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x16	sd11	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x17	sd12	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x18	sd13	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Arguments/Return Values	x19	sd14	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x20	sd15	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x21	sd16	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x22	sd17	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x23	sd18	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Saved Registers	x24	sd19	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x25	sd20	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x26	sd21	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x27	sd22	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x28	sd23	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Arguments/Return Values	x29	sd24	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x30	sd25	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x31	sd26	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x32	sd27	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x33	sd28	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Saved Registers	x34	sd29	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x35	sd30	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x36	sd31	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x37	sd32	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x38	sd33	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Arguments/Return Values	x39	sd34	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x40	sd35	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x41	sd36	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x42	sd37	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x43	sd38	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Saved Registers	x44	sd39	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x45	sd40	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x46	sd41	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x47	sd42	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x48	sd43	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Arguments/Return Values	x49	sd44	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x50	sd45	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x51	sd46	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x52	sd47	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x53	sd48	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Saved Registers	x54	sd49	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x55	sd50	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x56	sd51	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x57	sd52	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x58	sd53	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Arguments/Return Values	x59	sd54	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x60	sd55	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x61	sd56	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x62	sd57	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x63	sd58	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Saved Registers	x64	sd59	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x65	sd60	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x66	sd61	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x67	sd62	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x68	sd63	Callee
Optional Atomic Instructions Extension: AQA	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q	FP Arguments/Return Values	x69	sd64	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x70	sd65	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x71	sd66	Callee
	Atomic Read & Set Bit	R	LR.W	rd, rs1, imm	LR.W(M)Q		x72	sd67	Cal

Edge AI Processors And How To Specialize Them for AI

A typical Processing Core for Edge AI Processors

Single issue, in-order pipeline is the most energy efficient solution for power-constraints devices



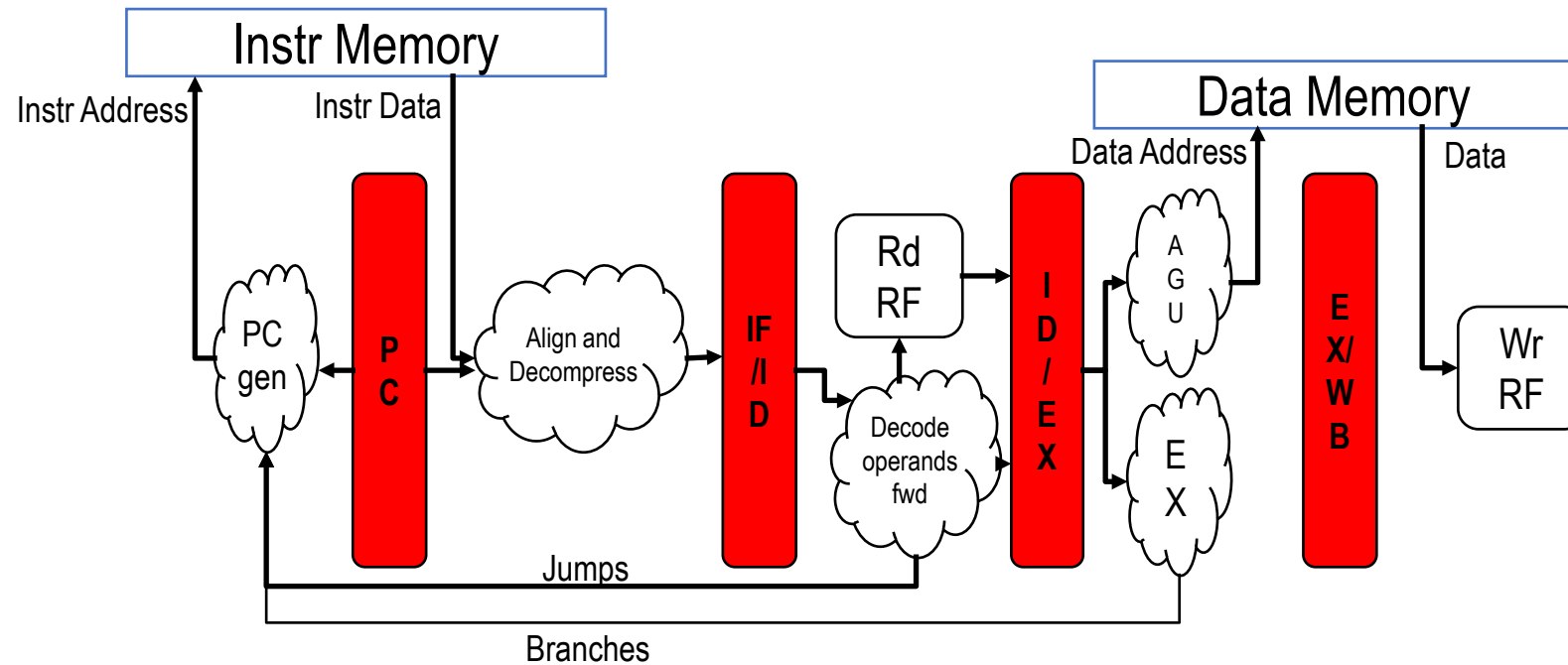
Are classical techniques for improving parallelism thanks to parallelism, but energy goes up because i'm using too much hardware

A typical Processing Core for Edge AI Processors

4-stage RISC-V RV32IMC

32-bit wide datapath

32 32-bit registers in the RF



Single issue, in order is the most energy efficient

- Memory is always critical: 1 full stage for fetch, 2 full stages for load+writeback
- Single stage decode
- Single stage execute+WB

Is this Enough for an AI/DSP Processor?

FIR Inner Loop Exec. in RV32IM core → this is part of a larger piece of code

```
for(int j=0; j<coeff_len; j++) {  
    sum += arr[i+j] * coeff[j];  
}
```

ANSI C

Lower-level view

```
for(int j=0; j<coeff_len; j++) {  
    int8 arr_int = arr[i+j];  
    int8 coeff_int = coeff[j];  
    arr_int = arr_int * coeff_int;  
    sum = sum + arr_int;  
}
```

ANSI C

per iteration:

1 MAC (multiply-accumulate) operation

7 instructions

efficiency = $1/7 = 0.14$ ☹

Assembly code on RV32IM Architecture

```
Lstart:  
lb t2, 0x0(t1)  
lb a4, 0x0(t3)  
mul a4, a4, t2  
add t0, t0, a4  
addi t3, t3, 0x1  
addi t1, t1, 0x1  
bne t3, a7, Lstart
```

RV32IM ASM

We want to minimize instructions that are not DOTP/MAC in the innermost loop of an arithmetic-intensive kernel

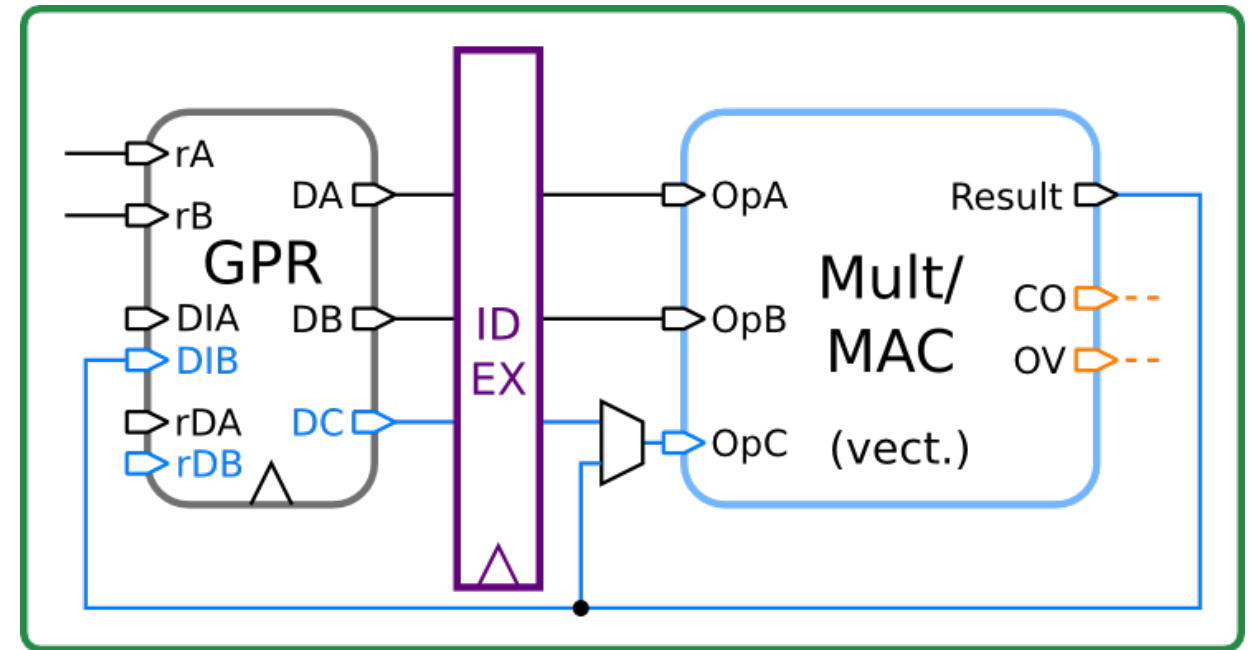
How to Design ISA Extensions for DSP/AI

1. Fuse Multiplication and Addition in a single instruction/operation → MAC op
2. Exploit Regular Memory Access Patterns → LOAD/STORE with automatic increment of the pointer
3. Boost Loop Control in Hardware → HW loops
4. Exploit Low-Bitwidth of DSP/AI operands and data → Packed-SIMD ALU operations & dot products

Multiply Accumulate

- Accumulation on 32 bit data
- Directly on the register file
- Pro:
 - Faster access to mac accumulation
 - Single cycle mult/mac
- Cons:
 - Additional read port on the register file
 - used for pre/post increment with register

p.mac a5,a0,a1



FIR Inner Loop Exec. in RV32IM + MAC

```
for(int j=0; j<coeff_len; j++) {  
    sum += arr[i+j] * coeff[j];  
}
```

ANSI C

Lower-level view

```
for(int j=0; j<coeff_len; j++) {  
    int8 arr_int = arr[i+j];  
    int8 coeff_int = coeff[j];  
    arr_int = arr_int * coeff_int;  
    sum = sum + arr_int;  
}
```

per iteration:

1 MAC (multiply-accumulate) operation

6 instructions

efficiency = $1/6 = 0.17$ ☹

a7 = coeff_len

Lstart:

```
lb t2, 0x0(t1)  
lb a4, 0x0(t3)  
p.mac t0, a4, t2  
addi t3, t3, 0x1  
addi t1, t1, 0x1  
bne t3, a7, Lstart
```

RV32IMXpulp ASM

The memory pointer is incremented with a regular pattern (+0x1) → Can I delegate this operation to hardware?

Post increment

- Automatic address update
 - Update base register with computed address after the memory access
 - ⇒ Save instructions to update address register
- Post-increment:
 - Base address serves as memory address
- Offset can be stored in:
 - Register
 - Immediate

Original RISC-V

```
addi x4, x0, 64
Lstart :
lb x2, 0(x10)
lb x3, 0(x12)
addi x10, x10, 1
addi x12, x12, 1
.....
bne x2,x3, Lstart
```

Auto-incr load/store

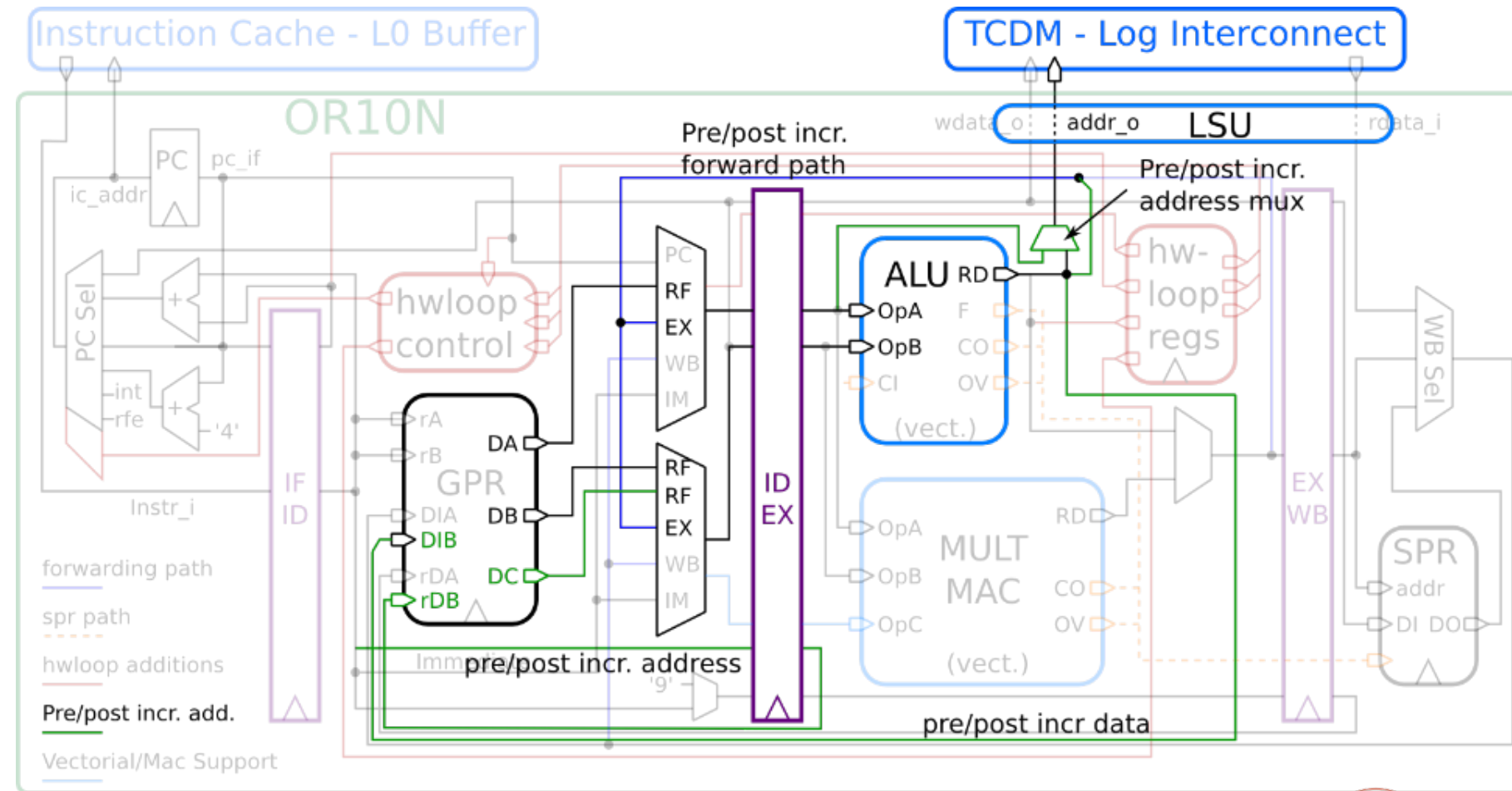
```
addi x4, x0, 64
Lstart :
lb x2, 0(x10!)
lb x3, 0(x12!)
.....
bne x2,x3, Lstart
```

⇒ save 2 additional instructions to update the read addresses of the operands!



Post increment – HW cost

- Register file requires additional read port if offset is stored in register
- Processor core area increases by 8-12 %
 - Ports can be used for other instructions



FIR Inner Loop Exec. in RV32IM + MAC + pointer post-incr.

```
for(int j=0; j<coeff_len; j++) {  
    sum += arr[i+j] * coeff[j];  
}
```

ANSI C

Lower-level view

```
for(int j=0; j<coeff_len; j++) {  
    int8 arr_int = arr[i+j];  
    int8 coeff_int = coeff[j];  
    arr_int = arr_int * coeff_int;  
    sum = sum + arr_int;  
}
```

per iteration:

1 MAC (multiply-accumulate) operation

4 instructions

efficiency = $1/4 = 0.25$ 😊

a7 = coeff_len

Lstart:

p.lb t2, 0x0(t1!)

p.lb a4, 0x0(t3!)

p.mac t0, a4, t2

bne t3, a7, Lstart

RV32IMXpulp ASM

Implicit update after load operation:
t1 = t1 + 0x1 (Byte)

one instruction is spent to control the loop
→ Can I delegate this operation to Hardware?

Hardware loops

- Hardware loop setup with:

- 3 separate instructions

lp.start, lp.end, lp.count, lp.counti

⇒ No restriction on start/end address

- Fast setup instructions

lp.setup, lp.setupi

⇒ Start address= PC + 4

⇒ End address= start address + offset

⇒ Counter from immediate/register

```
c = 0;
for(i=0;i<100;i++)
    c = c + a[i]*b[i];
```

Original RISC-V

```
//initialize counter
mv x4, 100
// init accumulator
mv x5, 0
Lstart:
//decrement counter
addi x4, x4, -1
//load elements from mem
lw x8, 0(x9)
lw x10, 0(x11)
//update memory pointers
add x9, x9, 4
add x11, x11, 4
//mac
mul x8, x8, x10
add x5, x5, x8
bne x4, x0, Lstart
```

HW Loop Ext

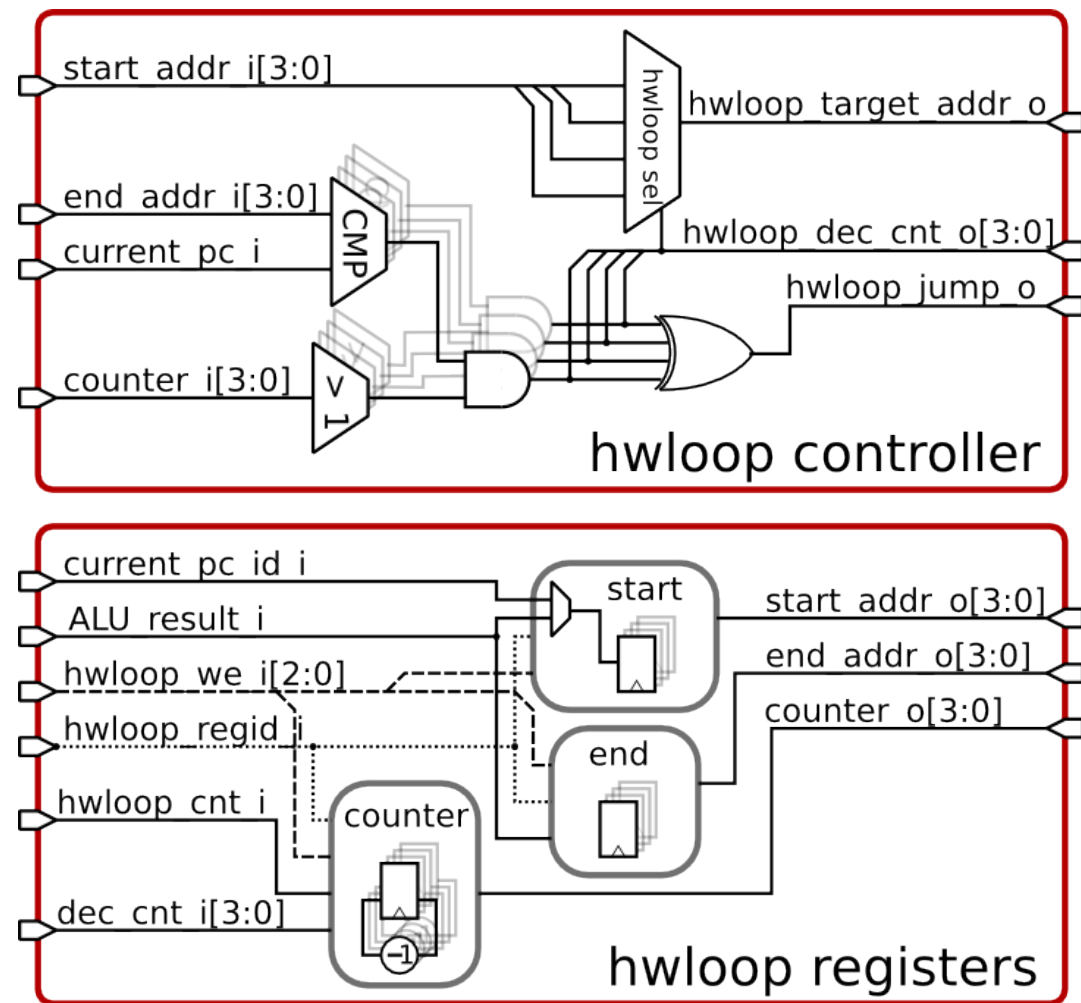
```
// init accumulator
mv x5, 0
//set number iterations, start and end of the loop
lp.setupi 100, Lend
//load elements from mem
lw x8, 0(x9)
lw x10, 0(x11)
//update memory pointers
add x9, x9, 4
add x11, x11, 4
//mac
mul x8, x8, x10
Lend: add x5, x5, x8
```

No counter and branch overhead!



Hardware Loops HW Cost

- Hardware loops to remove the branch overhead for loops.
- After configuration with start, end, count variables no more comparison and branches are required: Smaller loop benefit more!
- Loop needs to be set up beforehand and is fully defined by:
 - Start address
 - End address
 - Counter
- Two sets registers implemented to support nested loops.
- Area costs:
 - Processor core area increases by 5%
- Performance:
 - Speedup can be up to factor 2!



FIR Inner Loop Exec. in RV32IM + MAC + pointer post-incr. + HW Loop

```
for(int j=0; j<coeff_len; j++) {  
    sum += arr[i+j] * coeff[j];  
}
```

ANSI C

Lower-level view

```
for(int j=0; j<coeff_len; j++) {  
    int8 arr_int = arr[i+j];  
    int8 coeff_int = coeff[j];  
    arr_int = arr_int * coeff_int;  
    sum = sum + arr_int;  
}
```

per iteration:

1 MAC (multiply-accumulate) operation

3 instructions

efficiency = $1/3 = 0.33$ ☺

a7 = coeff_len

lp.setup t3, a7, Lend ←

Lstart:

p.lb t2, 0x0(t1!) ←

p.lb a4, 0x0(t3!) ←

Lend:

p.mac t0, a4, t2 ←

RV32IMXpulp ASM

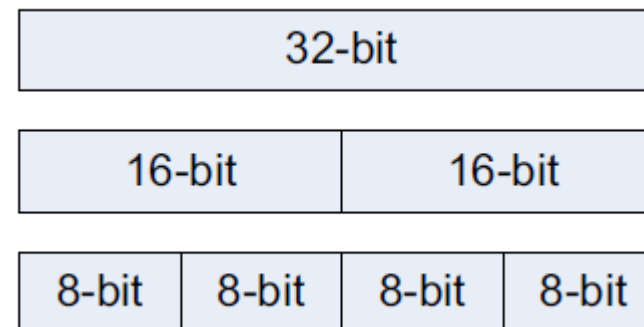
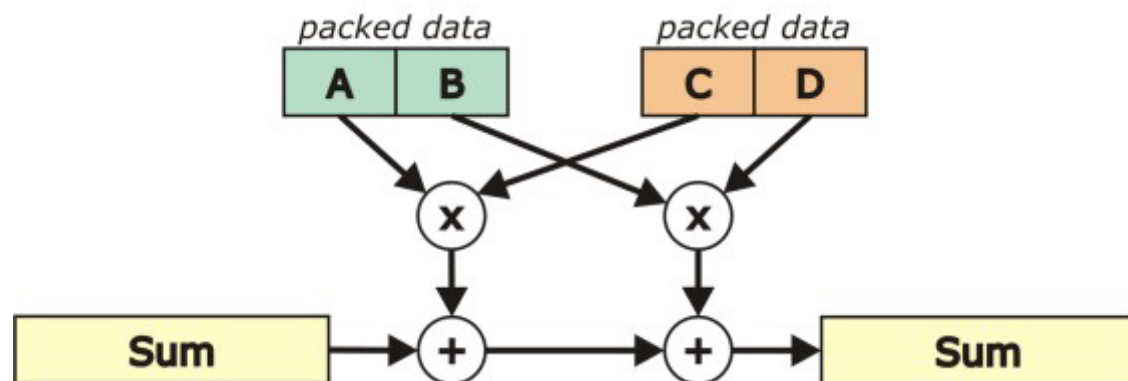
one element at time (1 Byte) is loaded and then processed. However, my processor datapath has a 32-bit wide parallelism → How to exploit it?

Xpulp Extensions: packet-SIMD

- **packet-SIMD extensions**

- Make usage of resources the best in performance with little overhead
- Target for embedded systems, RVV is for high performance
- pSIMD in 32bit machines
 - Vectors are either 4 8bits-elements or 2 16bits-elements
- pSIMD instructions
- Useful for NN as they are tolerant to low-bitwidth operands

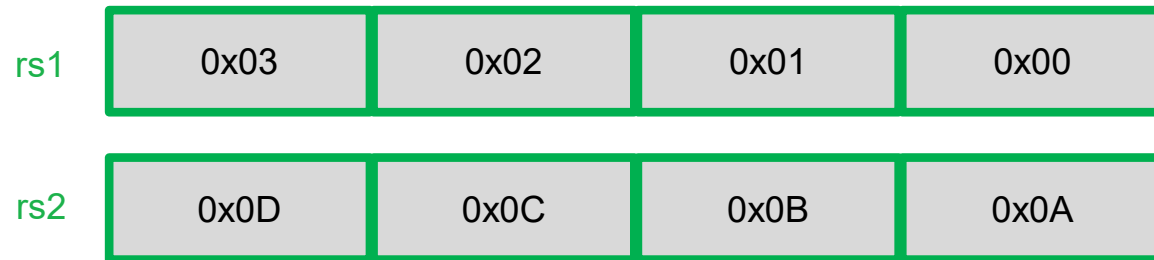
Computation	add, sub, shift, avg, abs, dot product
Compare	min, max, compare
Manipulate	extract, pack, shuffle



Xpulp Extensions: packet-SIMD

- Same Register-file

- The instruction encode how to interpret the content of the register



add rD, rs1, rs2	$rD = 0x03020100 + 0x0D0C0B0A$
add.h rD, rs1, rs2	$rD[0] = 0x0100 + 0x0B0A$ $rD[1] = 0x0302 + 0x0D0C$
add.b rD, rs1, rs2	$rD[0] = 0x00 + 0x0A$ $rD[1] = 0x01 + 0x0B$ $rD[2] = 0x02 + 0x0C$ $rD[3] = 0x03 + 0x0D$

Benefits of SIMD Instructions

- Parallelizes operations (2x to 4x speed gain)
- Minimizes the number of Load/Store instruction for exchanges between memory and register file (up 2 or 4 data transferred at once)
- Improve register file use (1 register holds 2 or 4 values)

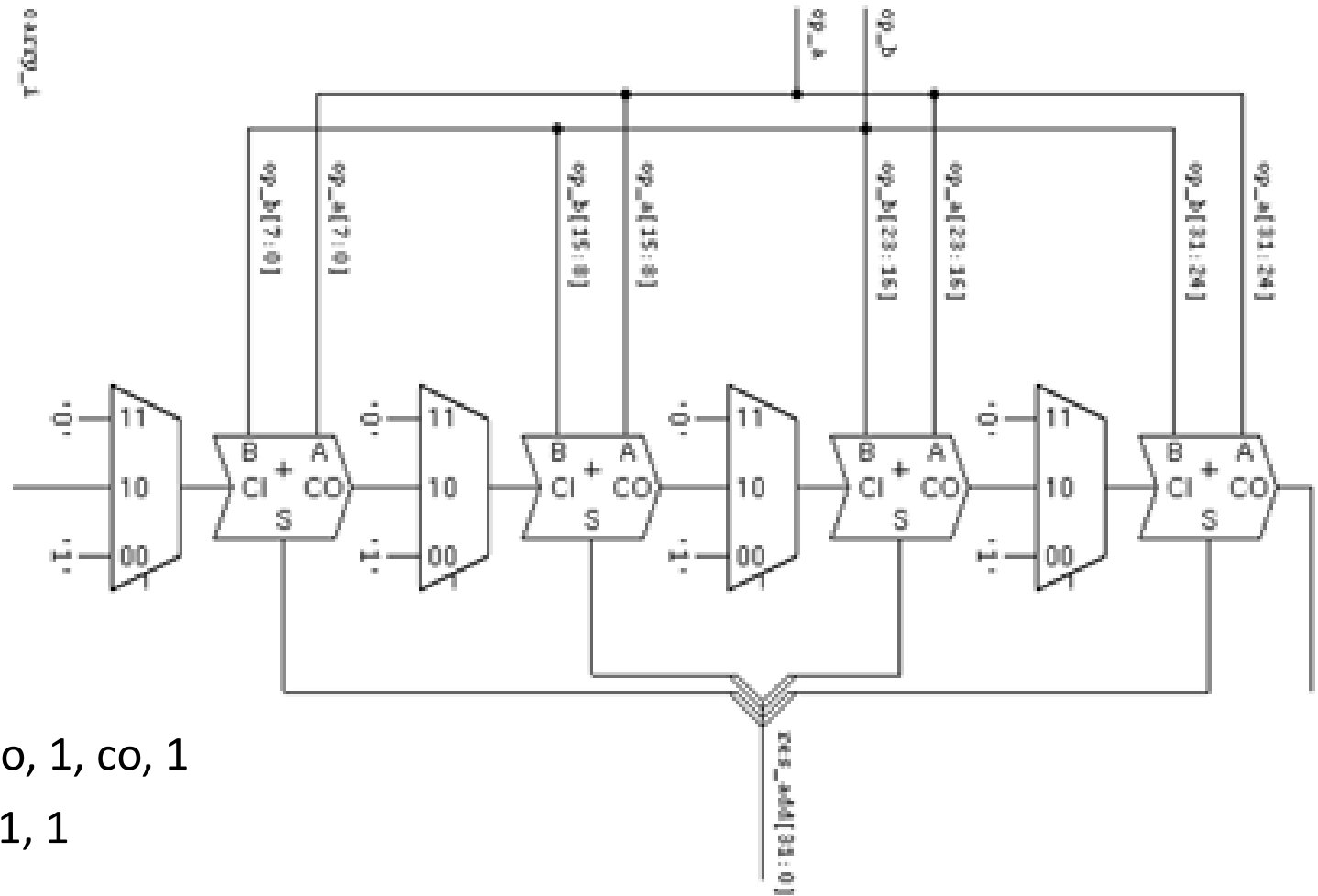
→ Can be used to dramatically increase the performance of most DSP algorithms running on embedded systems (including filtering, image processing, audio processing, **AI**)



Xpulp Extensions: packet-SIMD

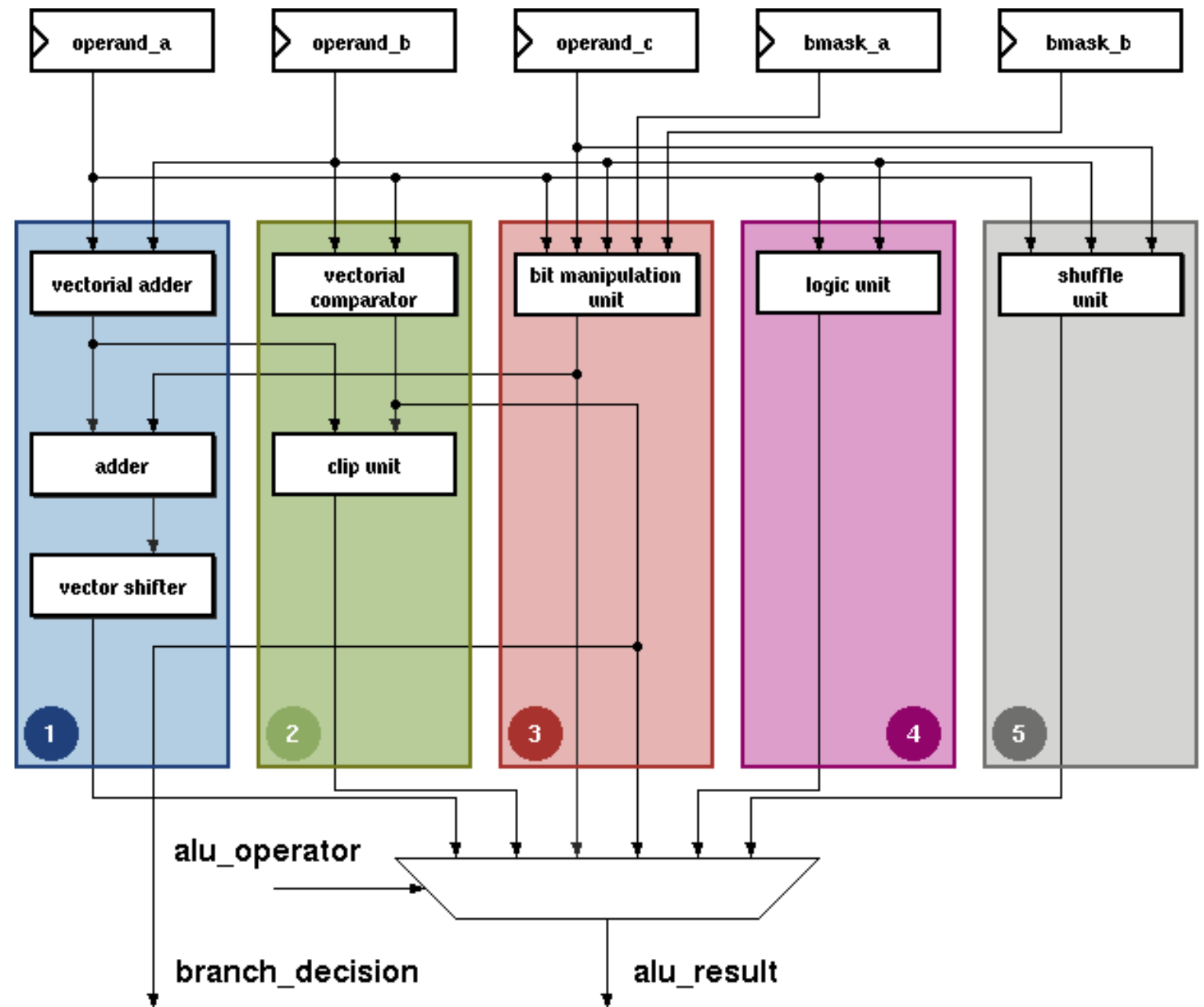
Vectorial Adder

- HW reuse for small overhead
- Vector modes:
 - bytes, halfwords, word
 - 4 byte operations
 - With byte select
 - 2 halfword operations
 - With halfword select
 - 1 word operation
 - Play with carry chain
 - 32bit adder → 35bit adder
 - Vector halfword sub → Carry = co, 1, co, 1
 - Vector byte sub → Carry = 1, 1, 1, 1
 - word sub → Carry = co, co, co, 1



ALU architecture

- Advanced ALU for Xpulp extensions
- Optimized datapath to reduce resources
- Multiple-adders for round
- Adder followed by shifter for fixed point normalization
- Clip unit uses one adder as comparator and the main comparator

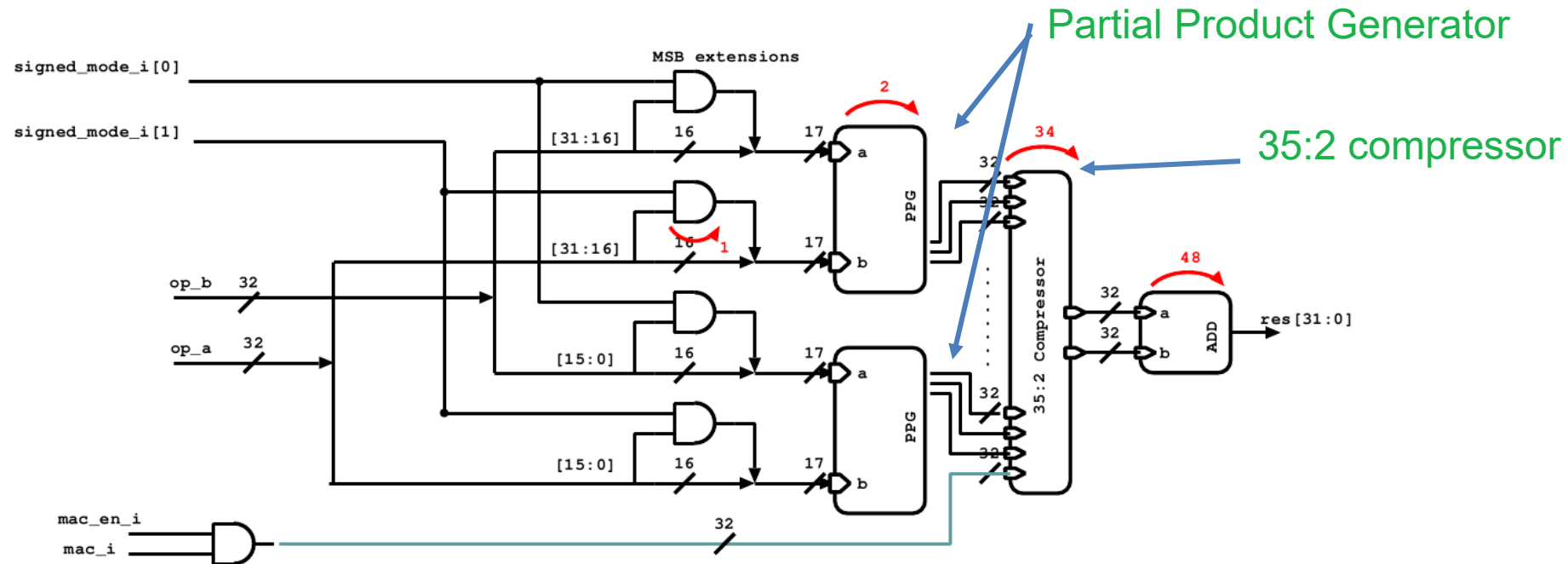


Dot Product Multiplier

- Dot Product: (half word example)

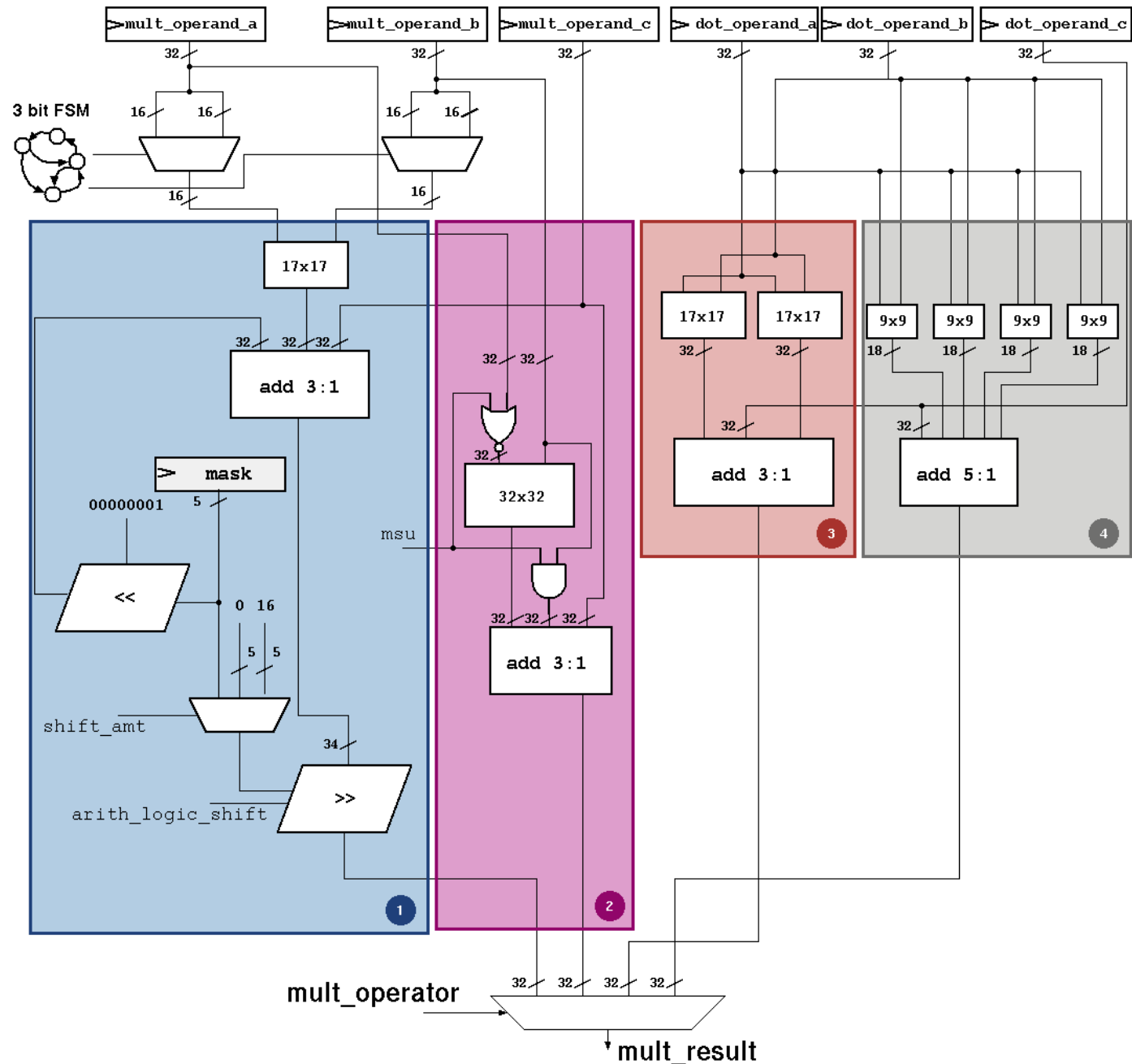
$$C[31:0] = A[31:16] * B[31:16] + A[15:0] * B[15:0] + C[31:0]$$

→ 2 multiplications, 1 addition, 1 accumulation in 1 cycle!



MUL architecture

- (blue) 16x16 with sign selection for short multiplications [with round and normalization]. 5 cycles FSM for higher 64-bits (mulh* instructions)
- (purple) One single cycle mac unit that performs MAC, MSU and MUL
- (red) short parallel dot product
- (grey) byte parallel dot product
- Clock gating to reduce switching activity between the integer and SIMD multiplier



FIR Inner Loop Exec. in RV32IM + MAC + pointer post-incr. + HW Loop + Packed-SIMD (8-bit)

```
for(int j=0; j<coeff_len; j++) {  
    sum += arr[i+j] * coeff[j];  
}
```

ANSI C

Lower-level view

```
for(int j=0; j<coeff_len/4; j++) {  
    int8 arr_int = arr[i+j];  
    int8 coeff_int = coeff[j];  
    arr_int = arr_int * coeff_int;  
    sum = sum + arr_int;  
}
```

per iteration:

1 SIMD sum-of-dot-product operation

3 instructions

efficiency = $1/3 = 0.33$ ☺

$a7 = \text{coeff_len} / 4$

lp.setup **t3**, **a7**, Lend

Lstart:

p.lw **t2**, 0x0(**t1**!)

p.lw **a4**, 0x0(**t3**!)

Lend:

pv.sdotp (**t0**, **a4**, **t2**)

Loop
coeff_len/ 4
times!

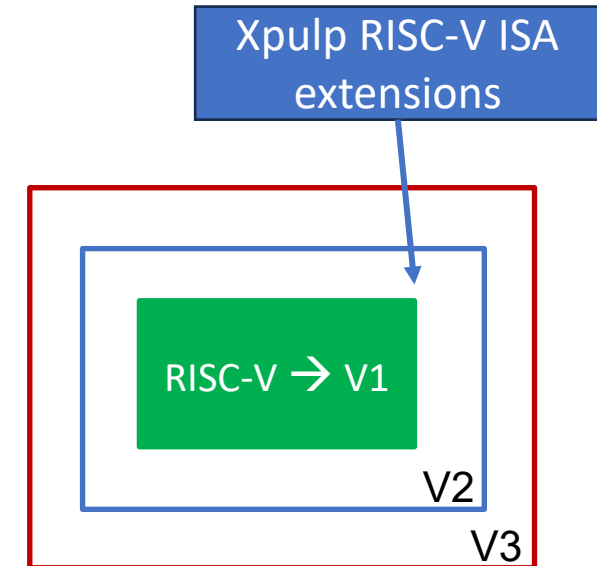
Load 1 SIMD
vector (32-
bit) , of 4 8-
bit elements

Perform 4 MAC ops on SIMD vectors
Each SIMD vector contains 4 8-bit
elements

ISA extensions - Recap

1. Fused MAC operation
2. Post-increment Load/Store
3. HW loops
4. Packed-SIMD ALU operations & dot product
5. Others (Bit manipulation, sub-byte arithmetic SIMD..)

- V1** Baseline RISC-V RV32IMC
HW loops
- V2** Post increment Load/Store
Mac
- V3** SIMD 2/4 + DotProduct + Shuffling
Bit manipulation unit
Lightweight fixed point (**ML centric**)



Small Power and Area overhead but significant performance gain → Improved Energy Efficiency!

Extensions at Work

```
for (i = 0; i < 100; i++)  
    d[i] = a[i] + b[i];
```

Baseline

```
mv    x5, 0  
mv    x4, 100  
Lstart:  
    lb    x2, 0(x10)  
    lb    x3, 0(x11)  
    addi  x10,x10, 1  
    addi  x11,x11, 1  
    add   x2, x3, x2  
    sb    x2, 0(x12)  
    addi  x4, x4, -1  
    addi  x12,x12, 1  
    bne   x4, x5, Lstart
```

11 cycles/output

Auto-incr load/store

```
mv    x5, 0  
mv    x4, 100  
Lstart:  
    lb    x2, 0(x10!)  
    lb    x3, 0(x11!)  
    addi  x4, x4, -1  
    add   x2, x3, x2  
    sb    x2, 0(x12!)  
    bne   x4, x5, Lstart
```

8 cycles/output

HW Loop

```
lp.setupi 100, Lend  
    lb    x2, 0(x10!)  
    lb    x3, 0(x11!)  
    add   x2, x3, x2  
Lend:  sb    x2, 0(x12!)
```

5 cycles/output

Packed-SIMD

```
lp.setupi 25, Lend  
    lw    x2, 0(x10!)  
    lw    x3, 0(x11!)  
    pv.add.b x2, x3, x2  
Lend: sw    x2, 0(x12!)
```

1,25 cycles/output

8.8x performance improvement

Can We Still Improve More?

- Let's look at the larger code..

```
for (int i=0; i<arr_len; i++) {  
    int32_t sum=0;  
    for (int j=0; j<coeff_len; j++) {  
        sum += arr[i+j] * coeff[j];  
    }  
    // shift accumulator and fit it into 16 bits  
    output[i] = (int16_t) ((sum >> right_shift) & 0x0000ffff);  
}
```

ANSI C

- We notice that `coeff[j]` is used for many iterations of the outer product
 - Same coefficient used over different input data (does it remind you of something? 😊)
 - `coeff[j]` can be reused many times in the RF of the processor
- A technique that allows us to improve data reuse (and also to reduce LOAD stalls and amortize LOAD costs) is called “LOOP UNROLLING” [SOFTWARE TECHNIQUE]

Loop Unrolling with Factor=2

```
p.ld      t2,0(t1!)
p.ld      a4,0(t3!)
p.ld      a5,0(t4!)
pv.sdotp  t0,a4,t2
pv.sdotp  t5,a5,t2
// ...
```

```
for (int i=0; i<arr_len; i+=2) {
    int32_t sum0=0, sum1=0;
    for (int j=0; j<coeff_len; j++) {
        sum0 += arr[i+0+j] * coeff[j];
        sum1 += arr[i+1+j] * coeff[j];
    }
    // shift accumulator and fit it into 16 bits
    output0[i] = (int16_t) ((sum0 >> right_shift) & 0xffff);
    output1[i] = (int16_t) ((sum1 >> right_shift) & 0xffff);
}
```

ANSI C

per iteration (unrolled loop):

2 SIMD sum-of-dot-product operation

5 instructions

efficiency = $2/5 = 0.4$ ☺

Loop Unrolling with Factor=4

```
p.ld      t2,0(t1!)
p.ld      a4,0(t3!)
p.ld      a5,0(t4!)
p.ld      a6,0(t6!)
p.ld      a7,0(t7!)
pv.sdotp  t0,a4,t2
pv.sdotp  t5,a5,t2
pv.sdotp  t8,a6,t2
pv.sdotp  t9,a7,t2
// ...
```

```
for (int i=0; i<arr_len; i+=4) {
    int32_t sum0=0, sum1=0, sum2=0, sum3=0;
    for (int j=0; j<coeff_len; j++) {
        sum0 += arr[i+0+j] * coeff[j];
        sum1 += arr[i+1+j] * coeff[j];
        sum2 += arr[i+2+j] * coeff[j];
        sum3 += arr[i+3+j] * coeff[j];
    }
    // shift accumulator and fit it into 16 bits
    output0[i] = (int16_t) ((sum0 >> right_shift) & 0xffff);
    output1[i] = (int16_t) ((sum1 >> right_shift) & 0xffff);
    output2[i] = (int16_t) ((sum2 >> right_shift) & 0xffff);
    output3[i] = (int16_t) ((sum3 >> right_shift) & 0xffff);
}
```

ANSI C

per iteration (unrolled loop):

4 SIMD sum-of-dot-product operation

9 instructions

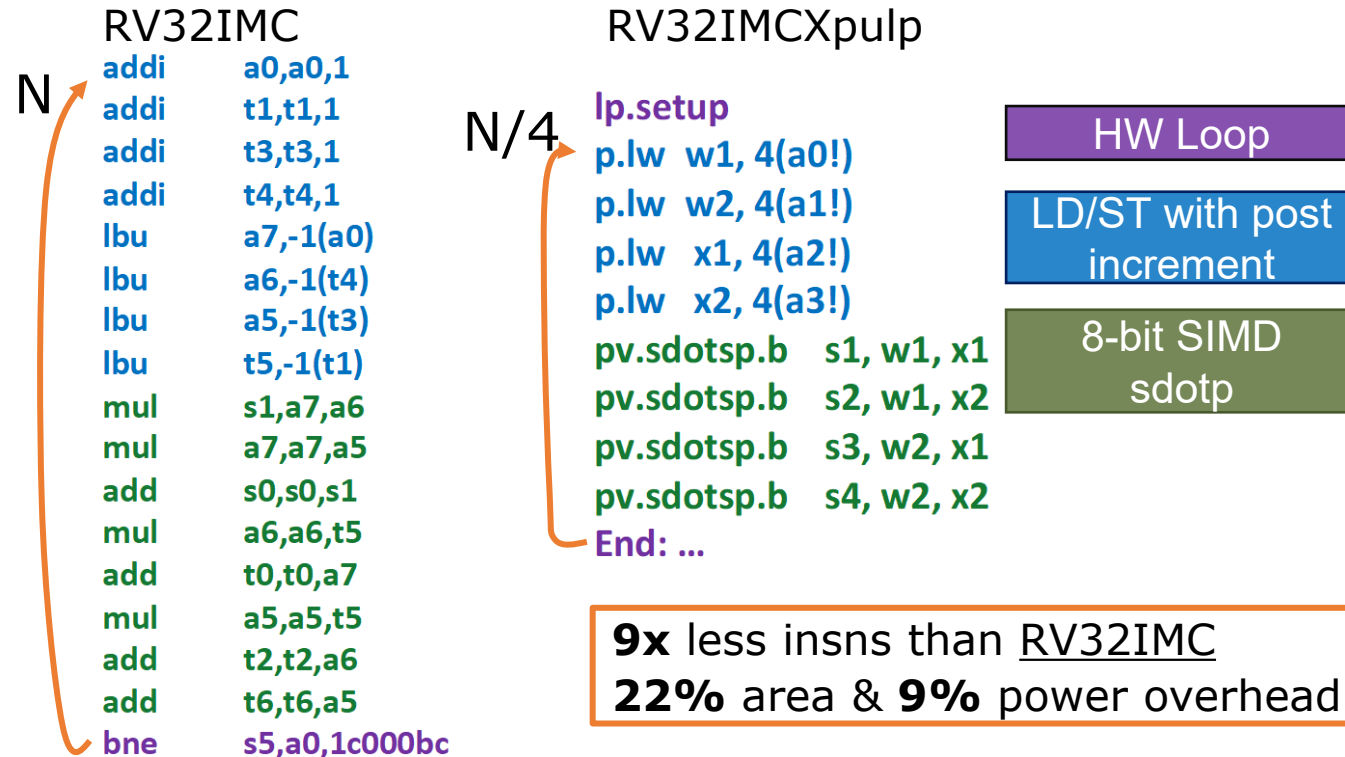
efficiency = $4/9 = 0.44$ ☺

The code becomes larger and larger, so storing it becomes less and less convenient

Can we scale-up indefinitely? No! → We are limited by the number of registers inside our pipeline! If our “innermost loop” does not fit the registers, operands are respilled to the stack (which is in the data memory → higher latency to load it back) → we loose in computing efficiency

QNN Acceleration Through Domain-Specific ISA

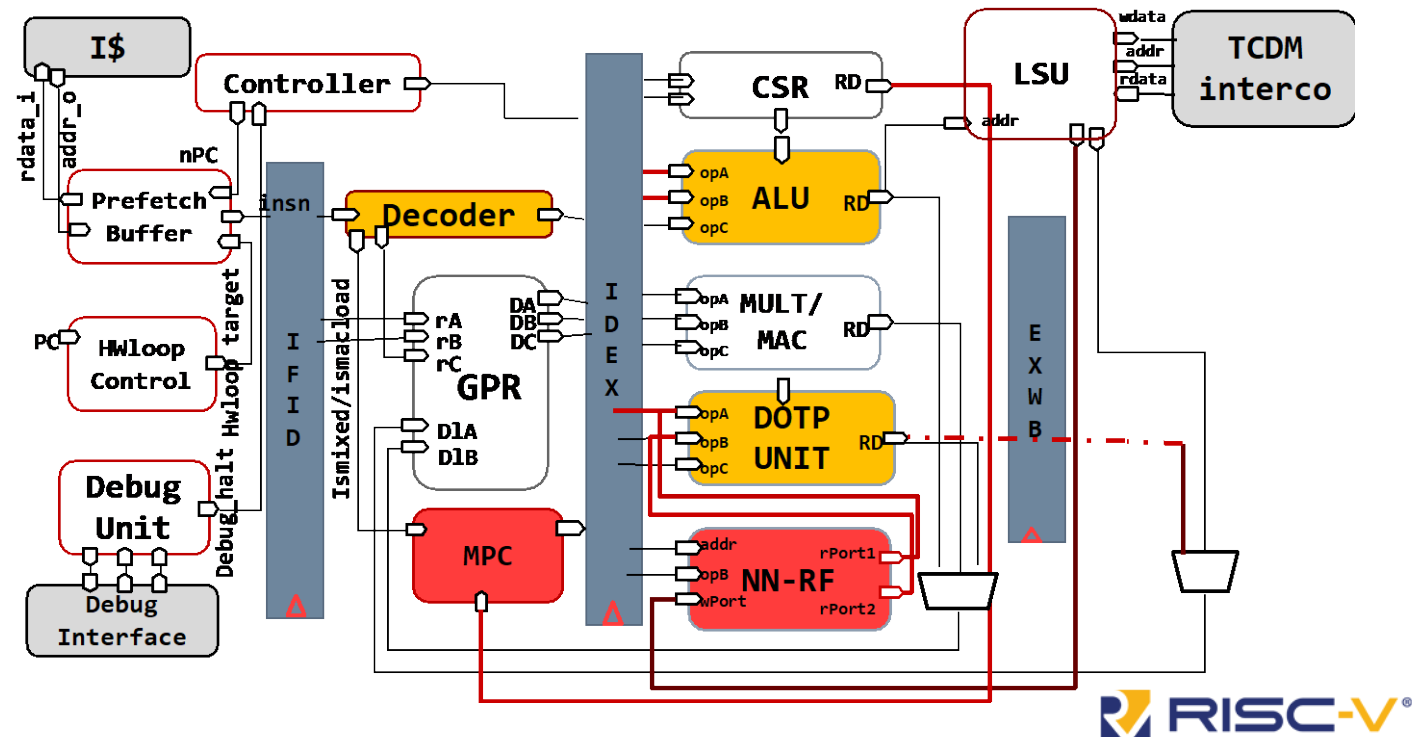
8-bit Convolution Kernel



- Power ↑ (slightly); Latency ↓ ↓ ↓ (>10x) on QNN kernels ⇒ Energy ↓ ↓
- Pushing further: Parallelization of the workload (multi-core) for high throughput

Processor Specialization

4-stage, single in-order issue pipeline



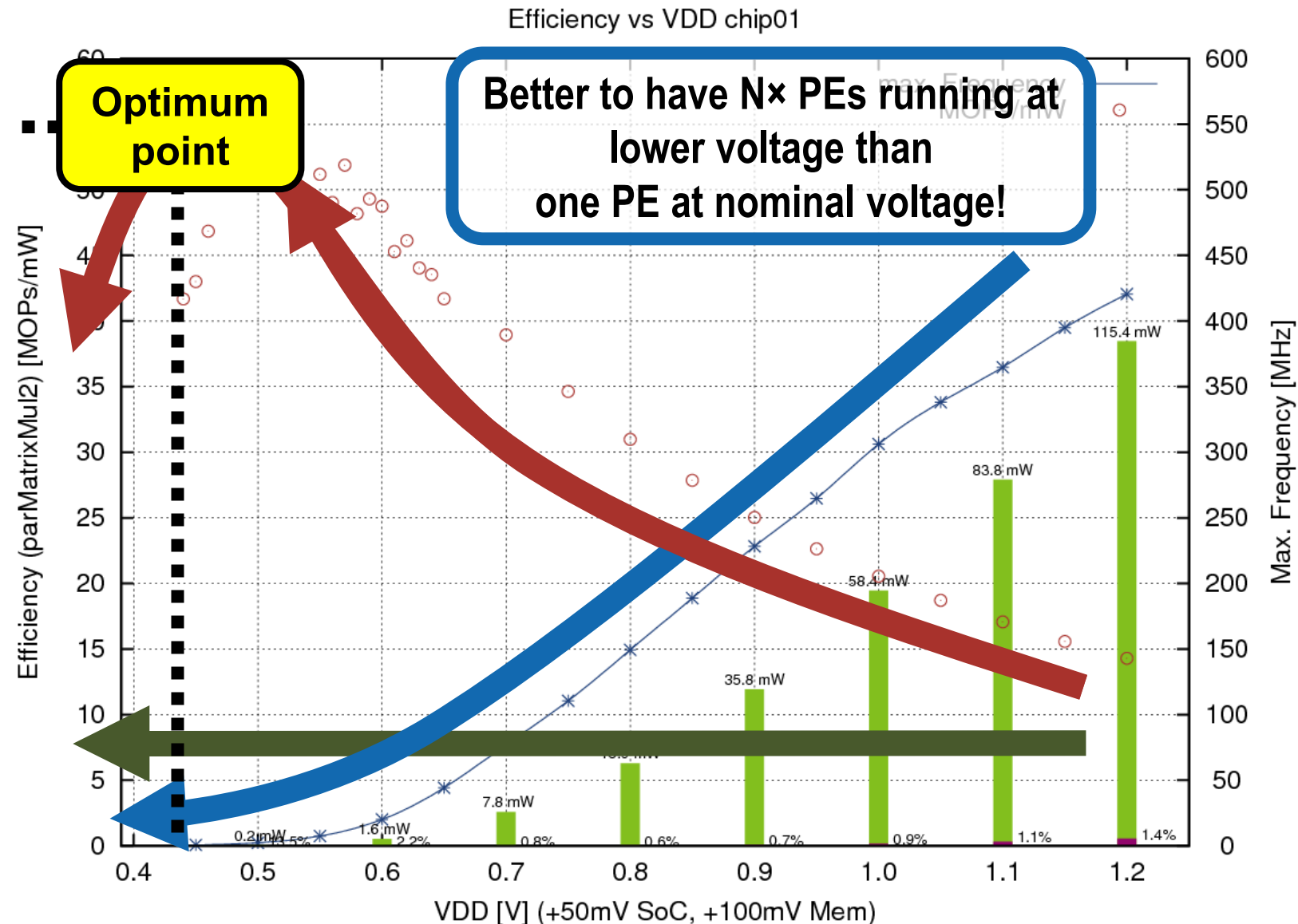
- ❑ **[Baseline] RV32IMC:** not good for machine learning workload (~44kGE)
- ❑ **[DSP] Xpulp:** 16/8-bit SIMD, HW loops, post-modified LD/ST & bit-manip. insns (~55kGE)
- ❑ **[AI] XpulpNN:** 4-bit, 2-bit SIMD (and mixed-precisions) & fused MAC-Load ops (~65kGE)

Garofalo, A., et al. (2021). Xpulpnn: Enabling energy efficient and flexible inference of quantized neural networks on risc-v based iot end nodes. *IEEE Transactions on Emerging Topics in Computing*, 9(3), 1489-1505.

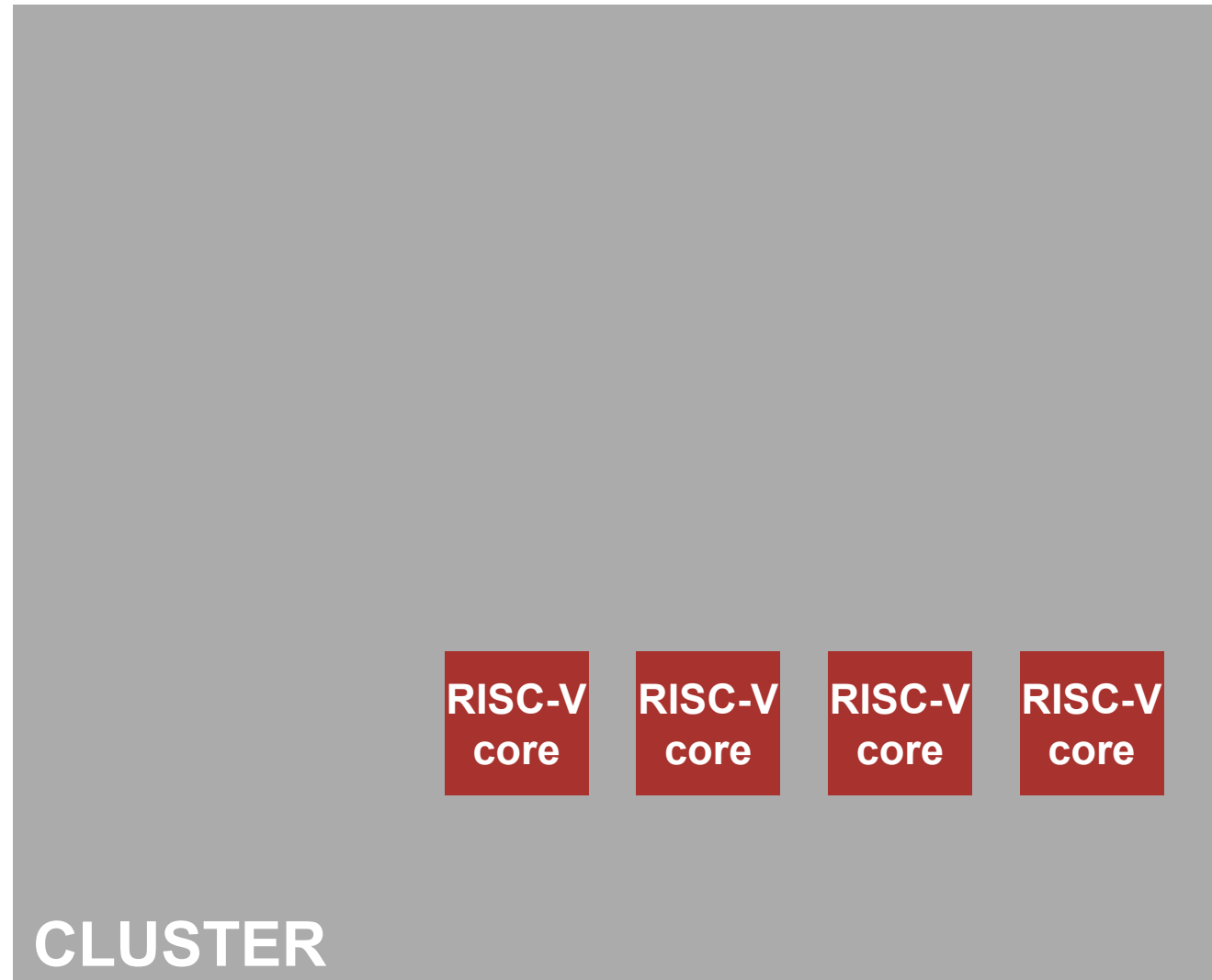
ML & Parallel Near Threshold → PULP

- As **VDD** decreases, **operating speed** decreases frequency
- However **efficiency** increases → more work done per Joule
- Until leakage effects start to dominate
- Put more units in parallel to get performance up and keep them busy with a parallel workload

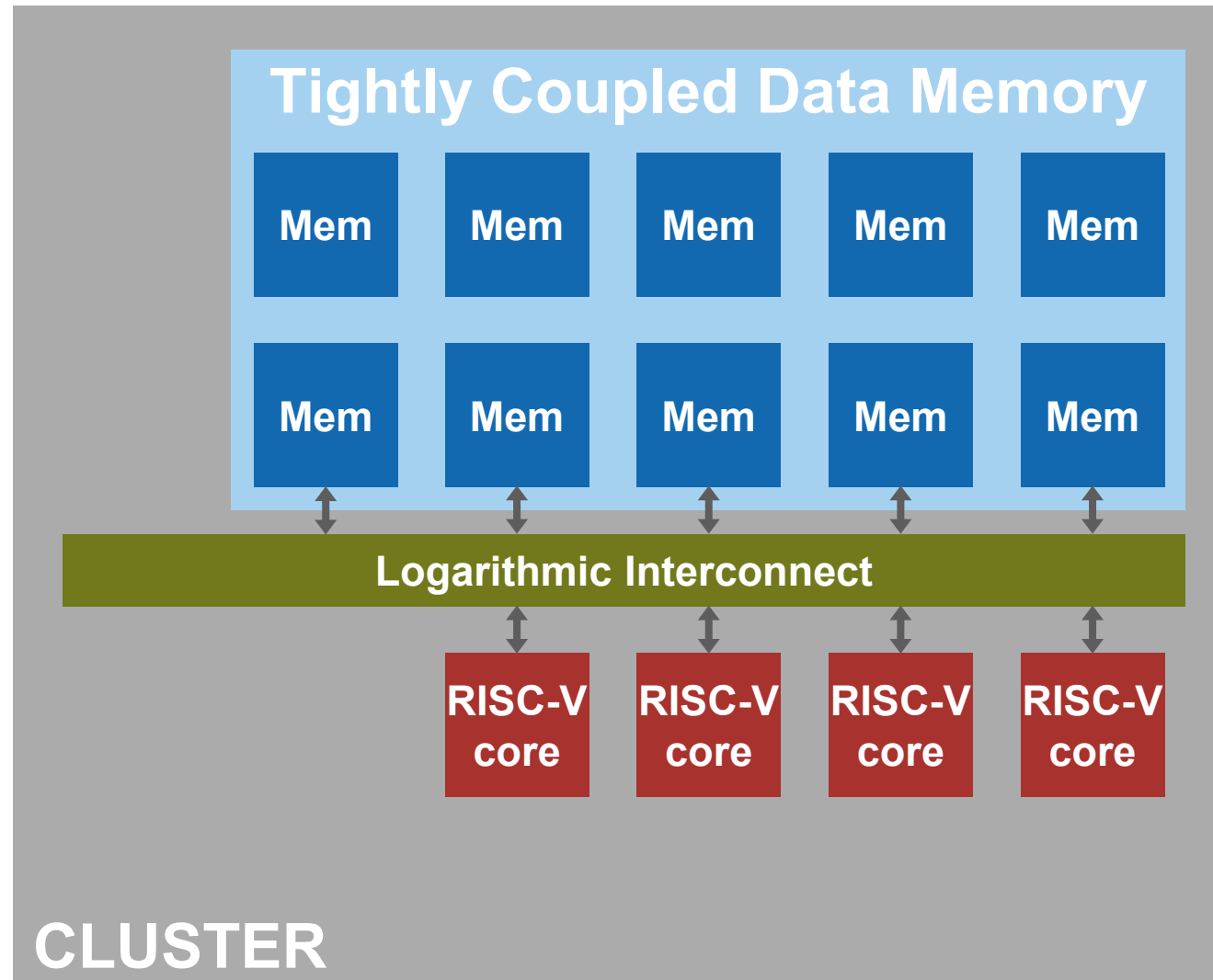
ML is massively parallel and scales well (P/S ↑ with NN size)



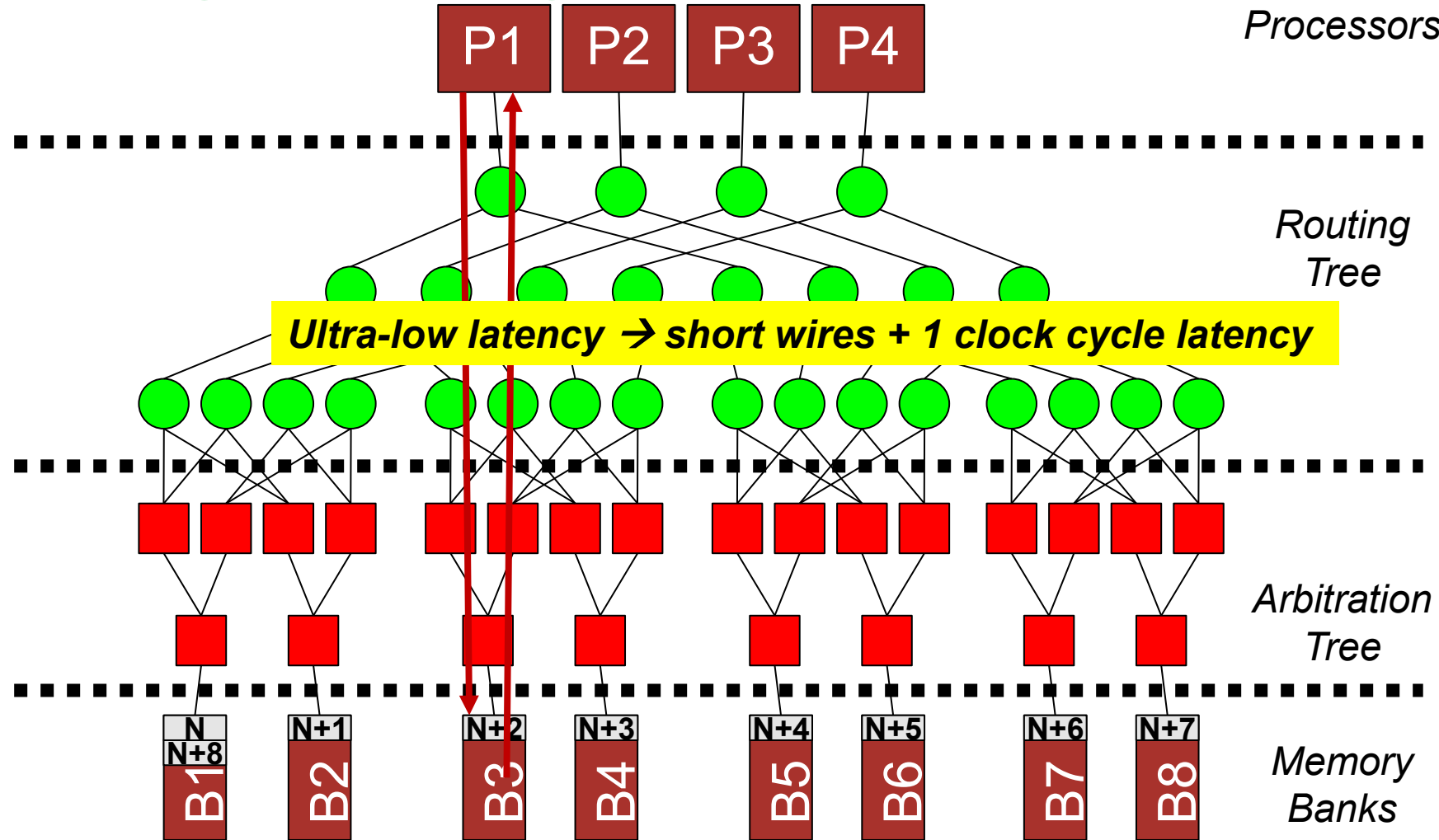
Multiple RI5CY Cores (1-16)



Low-Latency Shared TCDM



High speed single clock logarithmic interconnect

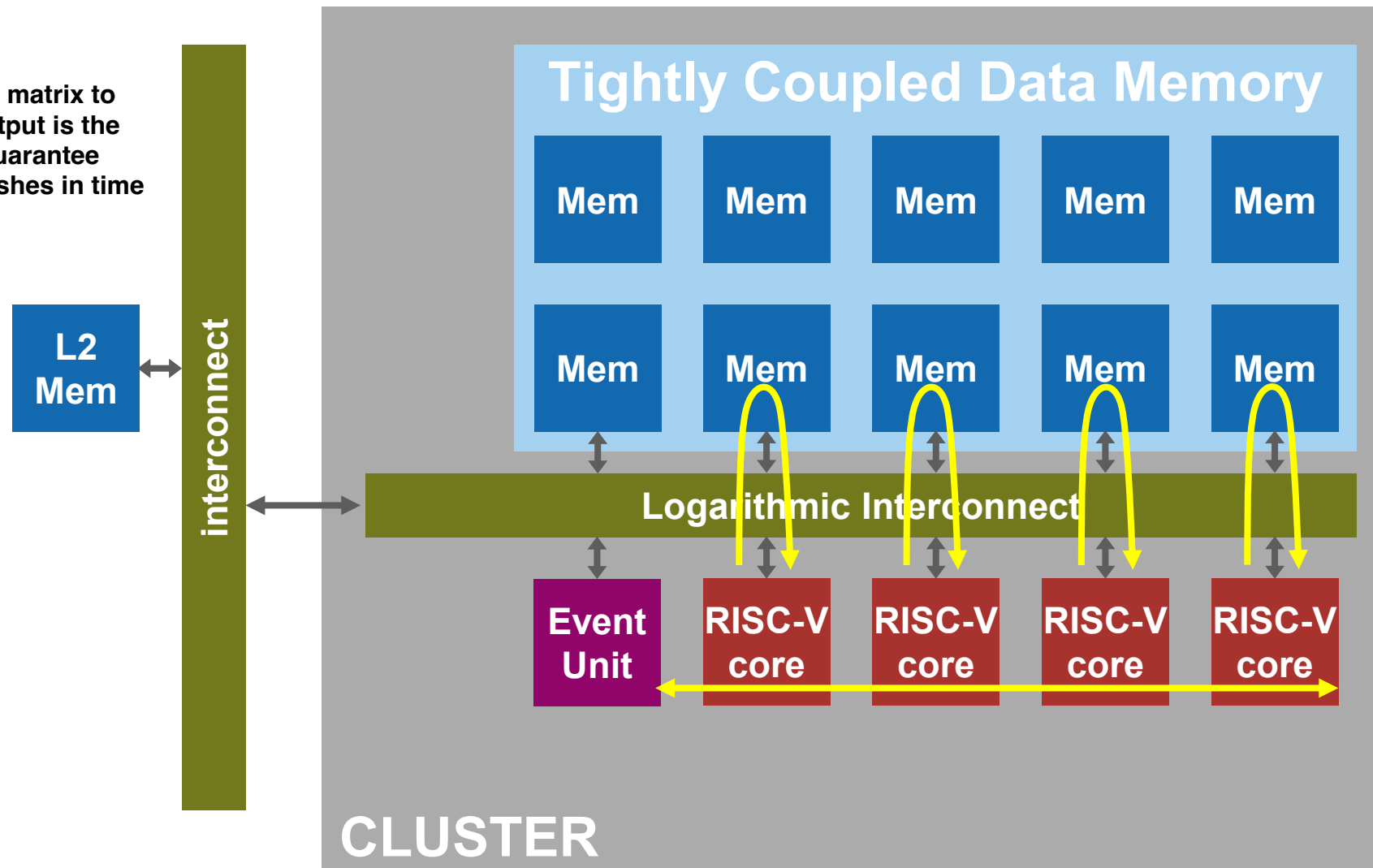


World-level bank interleaving «emulates» multiported mem

A. Rahimi, I. Loi, M. R. Kakoei and L. Benini, "A fully-synthesizable single-cycle interconnection network for Shared-L1 processor clusters," 2011 *Design, Automation & Test in Europe*, 2011, pp. 1-6.

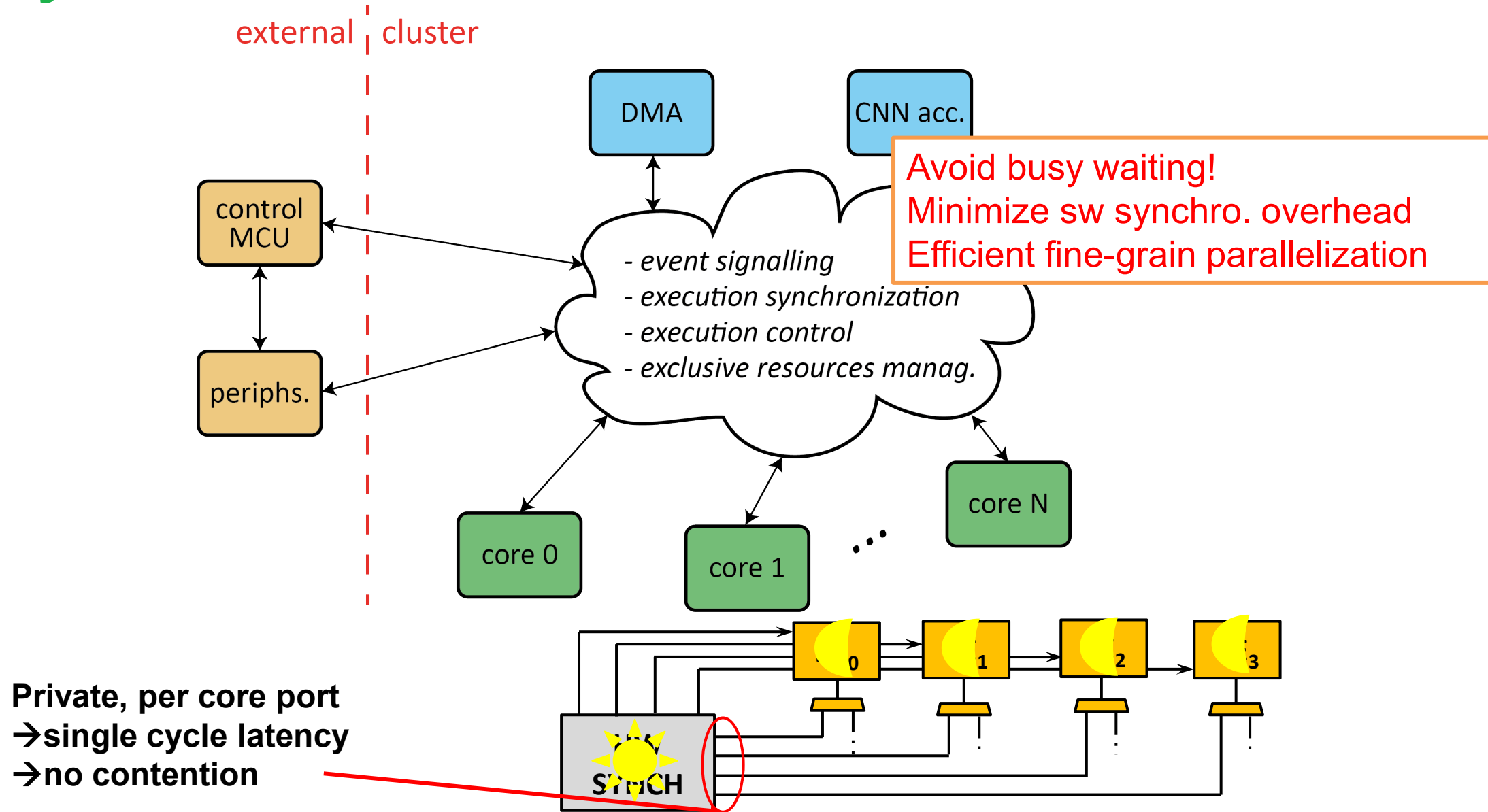
Fast synchronization and Atomics

In matrix multiplication we need some matrix to be synchronized for example if the output is the input of the next matrix. We have to guarantee that at most even the slower of us finishes in time for the next op.

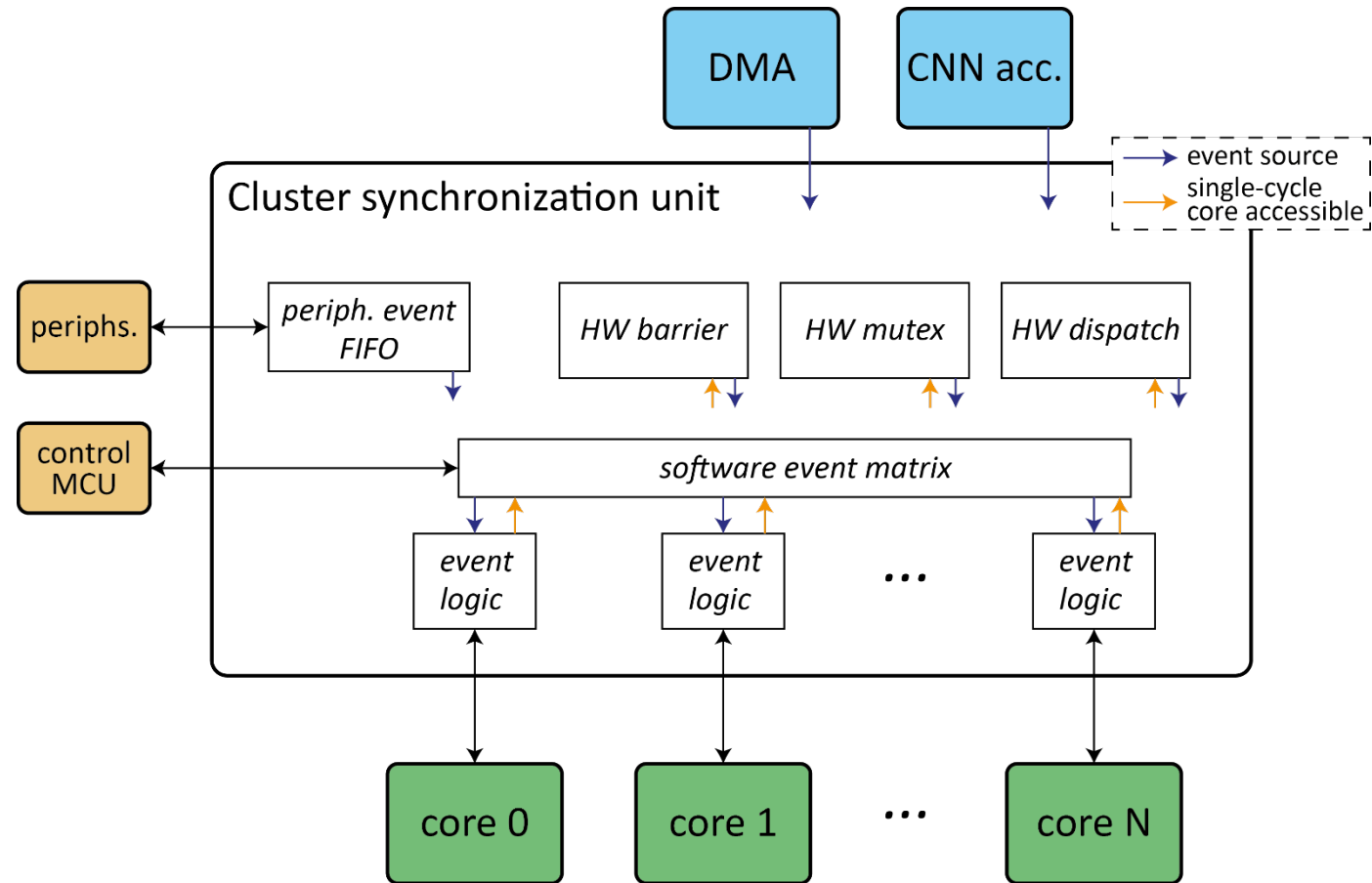


F. Glaser, G. Tagliavini, D. Rossi, G. Haugou, Q. Huang and L. Benini, "Energy-Efficient Hardware-Accelerated Synchronization for Shared-L1-Memory Multiprocessor Clusters," in IEEE TPDS, vol. 32, no. 3, pp. 633-648, 1 March 2021.

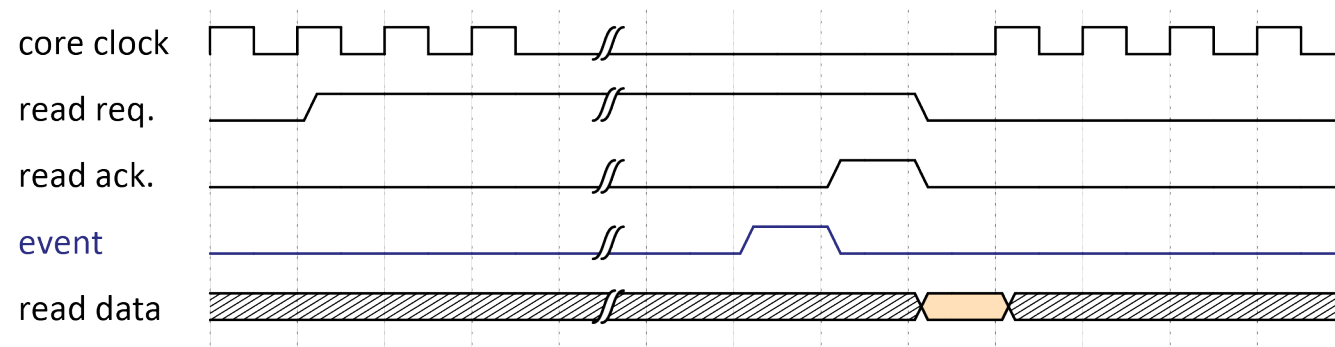
Synchronization & Events



PULP Cluster Event Unit



Energy-efficient Event Handling



instruction turns off
core clock
→ NO pipeline flush!

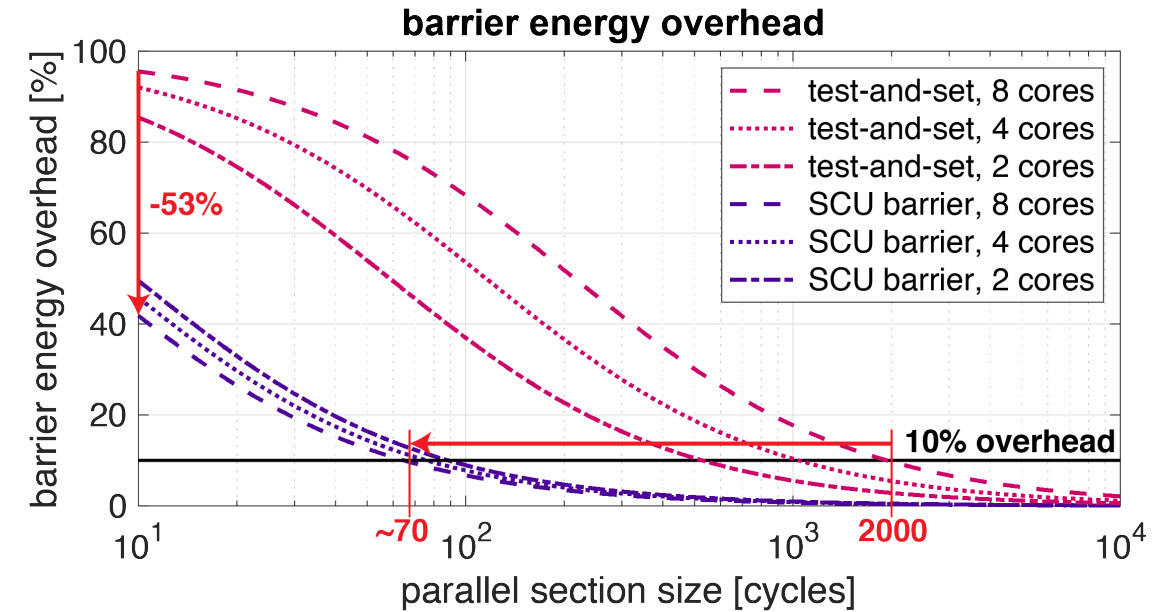
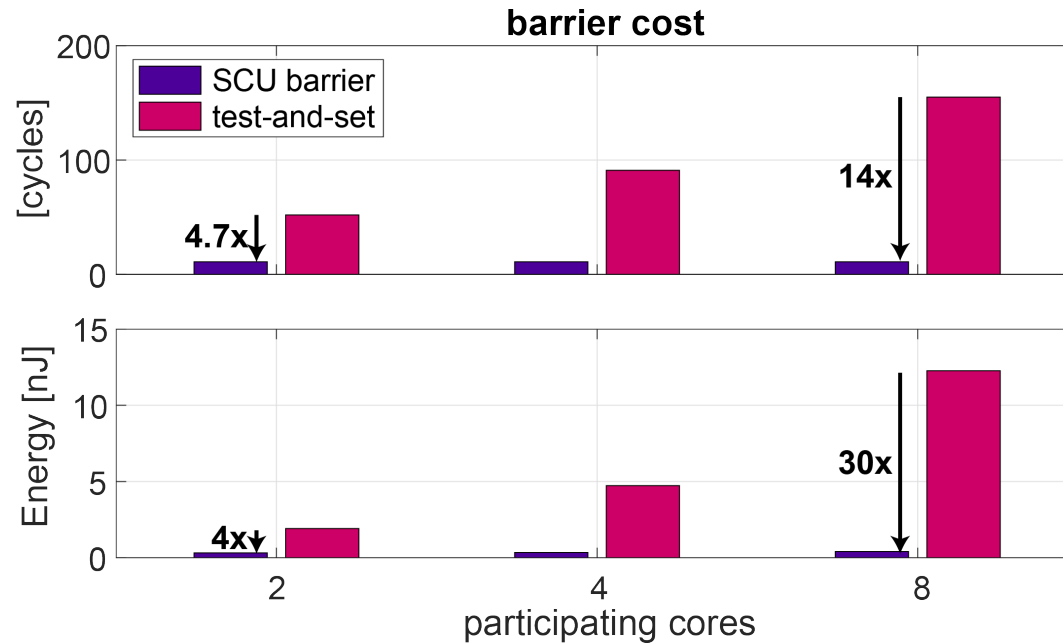
```
evt_data = evt_read(addr);
```

- event buffer content,
- program entry point
- mutex message
- triggering core id
- ...

- trigger sw event,
- trigger barrier,
- try mutex lock,
- read entry point,
- auto buffer clear?
- ...

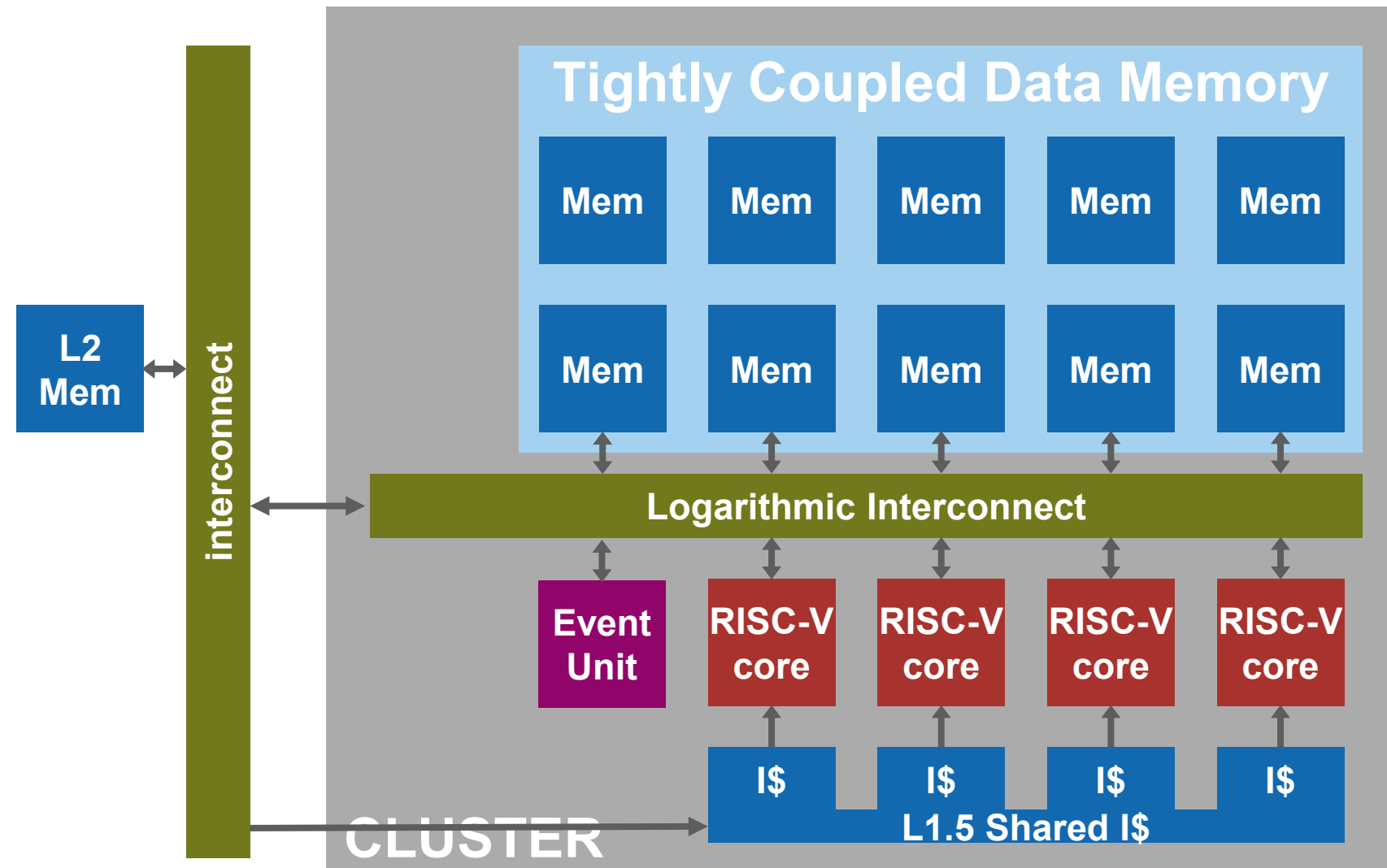
- Single instruction rules it all:
 - address determines action
 - read datum provides corresponding information

Results: Barrier

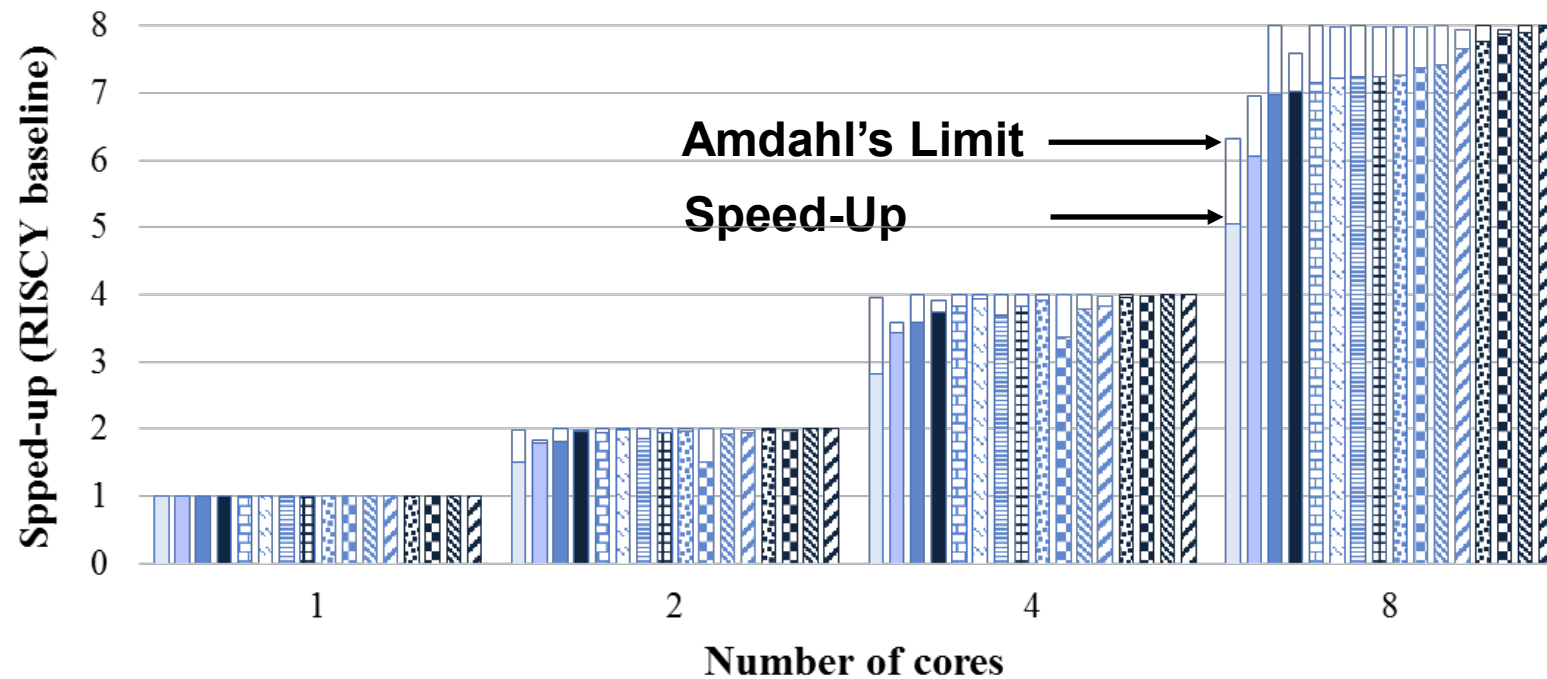


- Fully parallel access to SCU: Barrier cost constant
- Primitive energy cost: Down by up to 30x
- Minimum parallel section for 10% overhead in terms of ...
 - ... cycles: ~100 instead of > 1000 cycles
 - ... energy: ~70 instead of > 2000 cycles

Shared instruction cache with private “loop buffer”



PULP Parallel Speed-Up

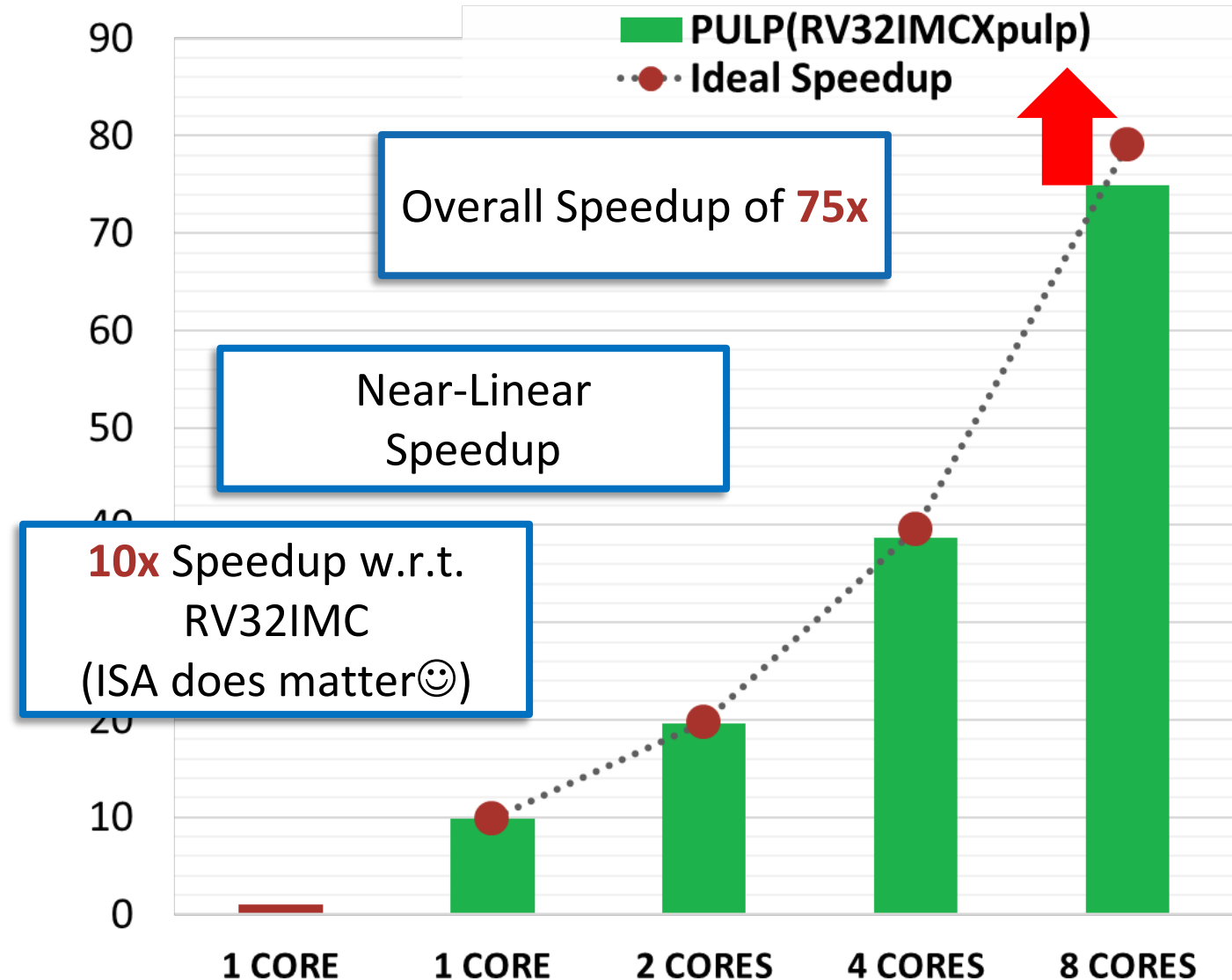


- PCA
- HD
- CNN Layer (16-bit)
- BNN
- CNN Layer (8-bit)
- FIR
- SVM
- MatMul (8-bit)
- 5x5 Conv (32-bit)
- FFT
- MatMul (32-bit)
- CNN Layer (32-bit)
- 5x5 Conv (16-bit)
- 5x5 Conv (8-bit)
- MatMul (16-bit)
- DWT

PULP for ML (DNNs) Speedup

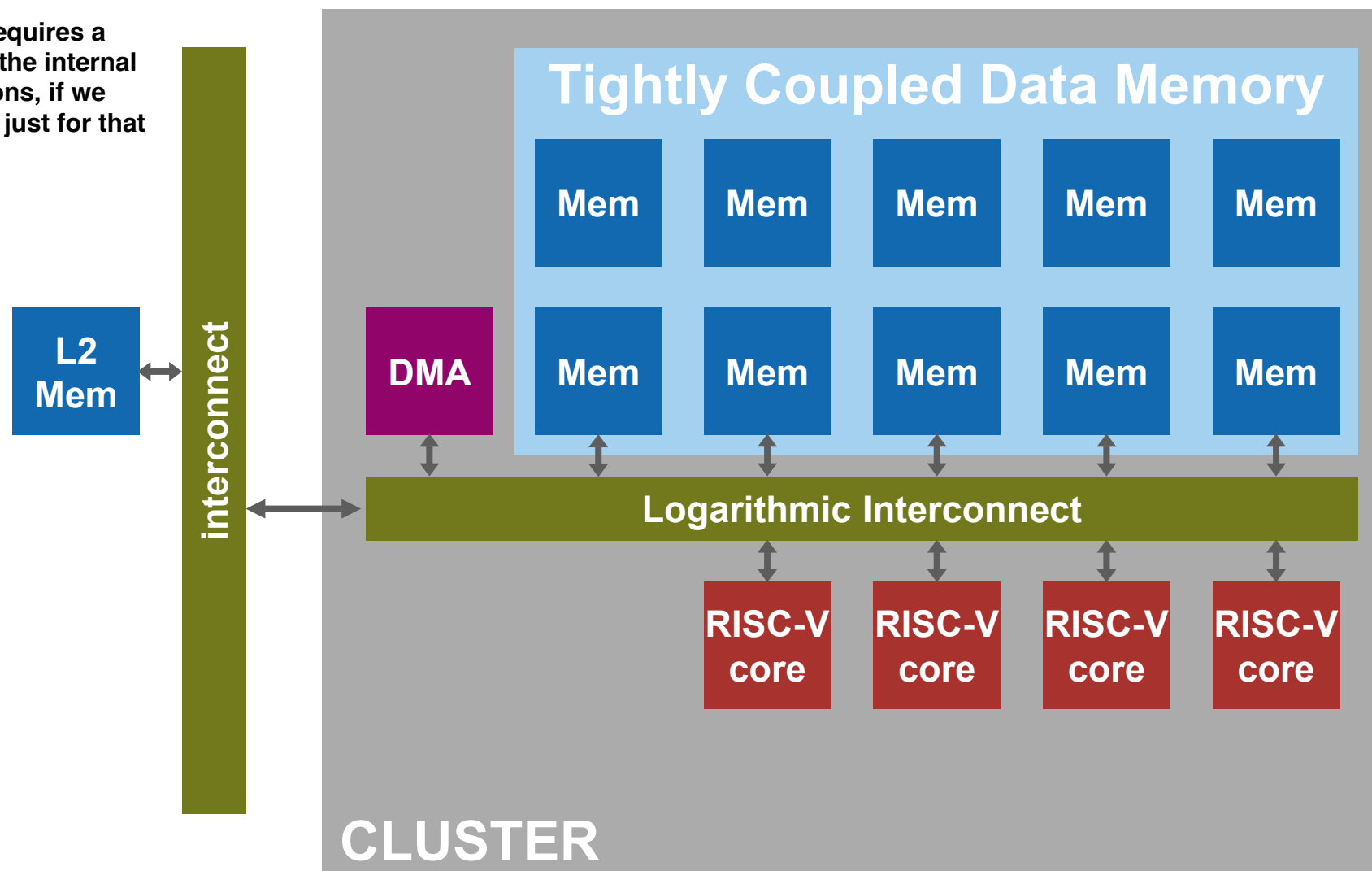
[Garofalo et al. Philos. Trans. R. Soc 20]

- **8-bit convolution**
 - Open source DNN library
- **10x** through xPULP
 - Extensions bring real speedup
- **Near-linear speedup**
 - Scales well for regular workloads
- **75x** overall gain
 - 2 orders of magnitude with DOTP+LW (**122x**)
 - Sub-byte (nibble, crumb) supported (**537x**, **939x**)



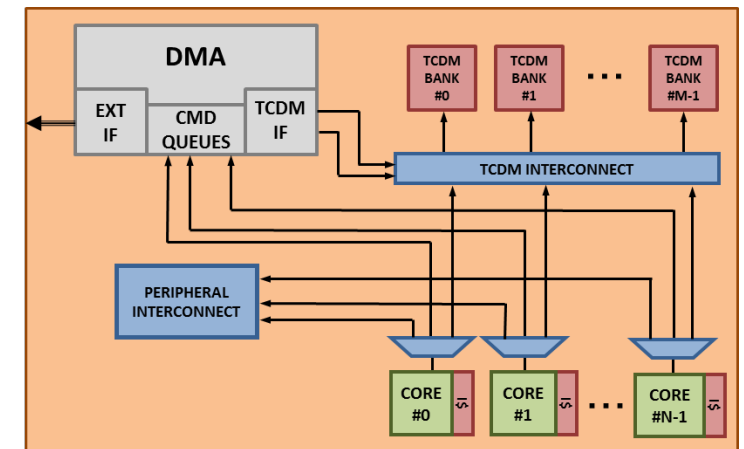
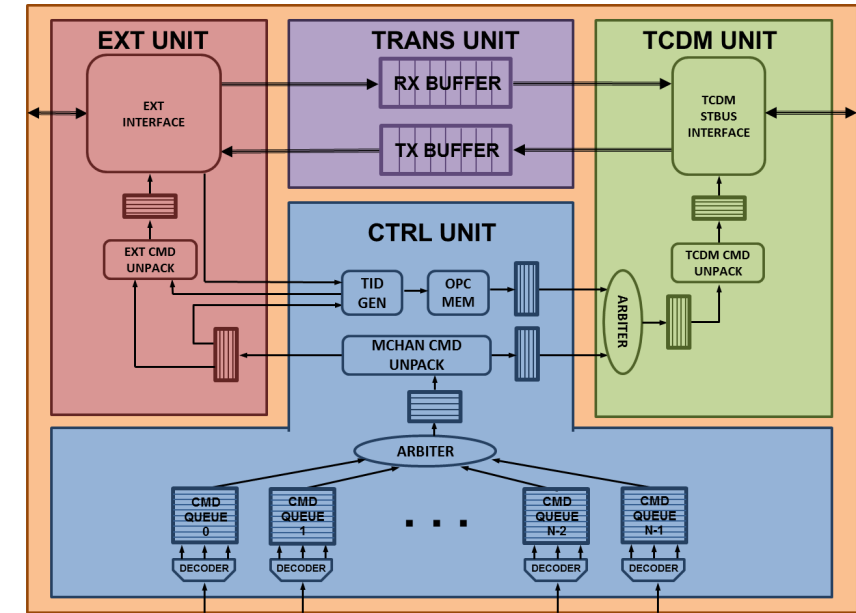
Data memory Hierarchy: DMA-based, SW managed

The external memory (L2) is far and requires a lot of time to be accessed. So even if the internal memories can accelerate the executions, if we need L2 we will become much slower just for that operation.



PULP MCHAN DMA Engine

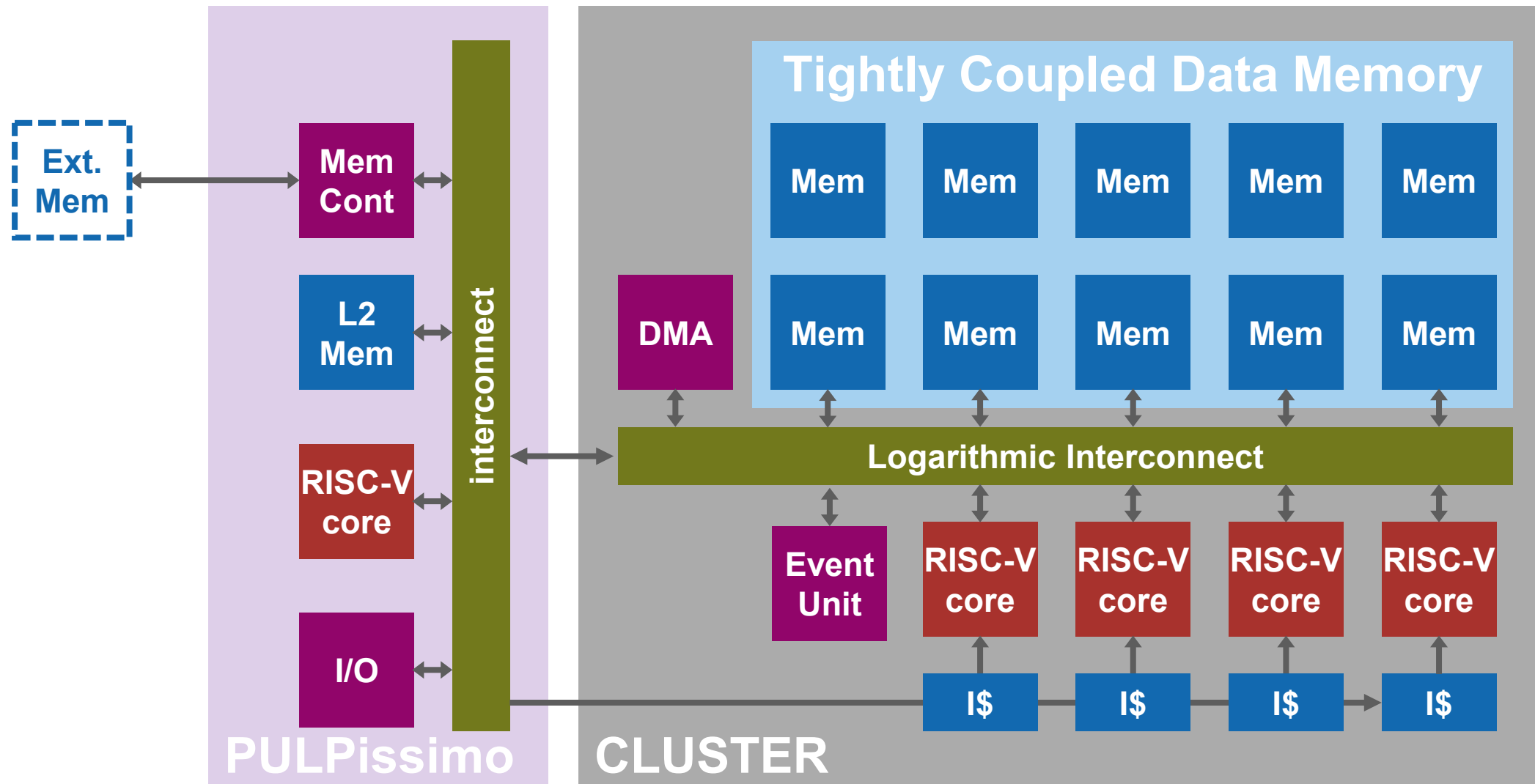
- A DMA engine optimized for integration in tightly coupled processor clusters
 - Dedicated, per-core non blocking programming channels
 - Ultra low latency programming (~10 cycles)
 - Small footprint (30Kgates) : avoid usage of large local FIFOs by forwarding data directly to TCDM (no store and forward)
 - Support for multiple outstanding transactions
 - Parallel RX/TX channels allow achieving full bandwidth for concurrent load and store operations
- Configurable parameters:
 - # of core channels
 - Size of command queues
 - Size of RX/TX buffer
 - # of outstanding transactions



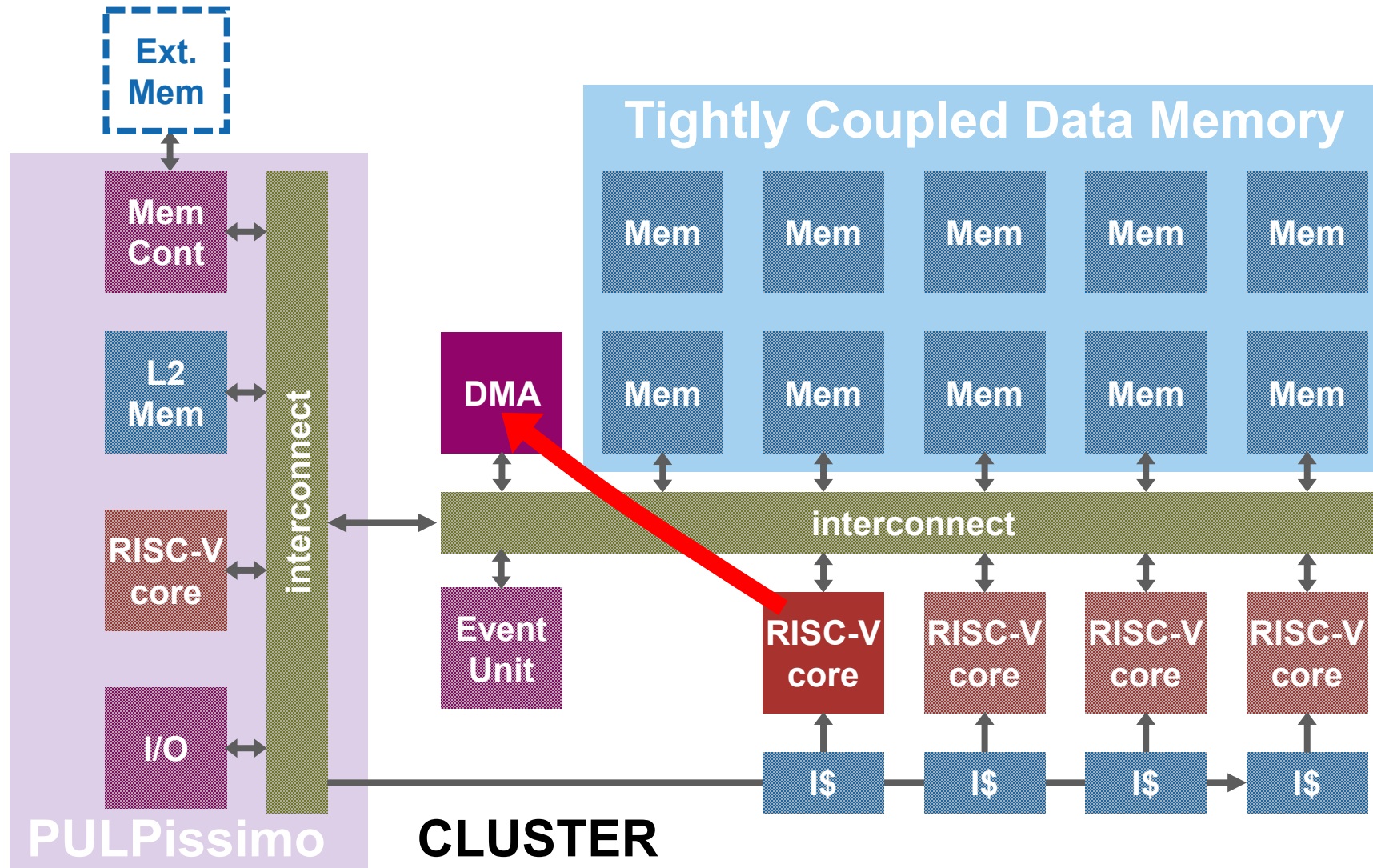
Integration in a PULP cluster

D. Rossi, I. Loi, G. Haugou, and L. Benini. 2014. Ultra-low-latency lightweight DMA for tightly coupled multi-core clusters. In Proceedings of the 11th ACM Conference on Computing Frontiers (CF '14).

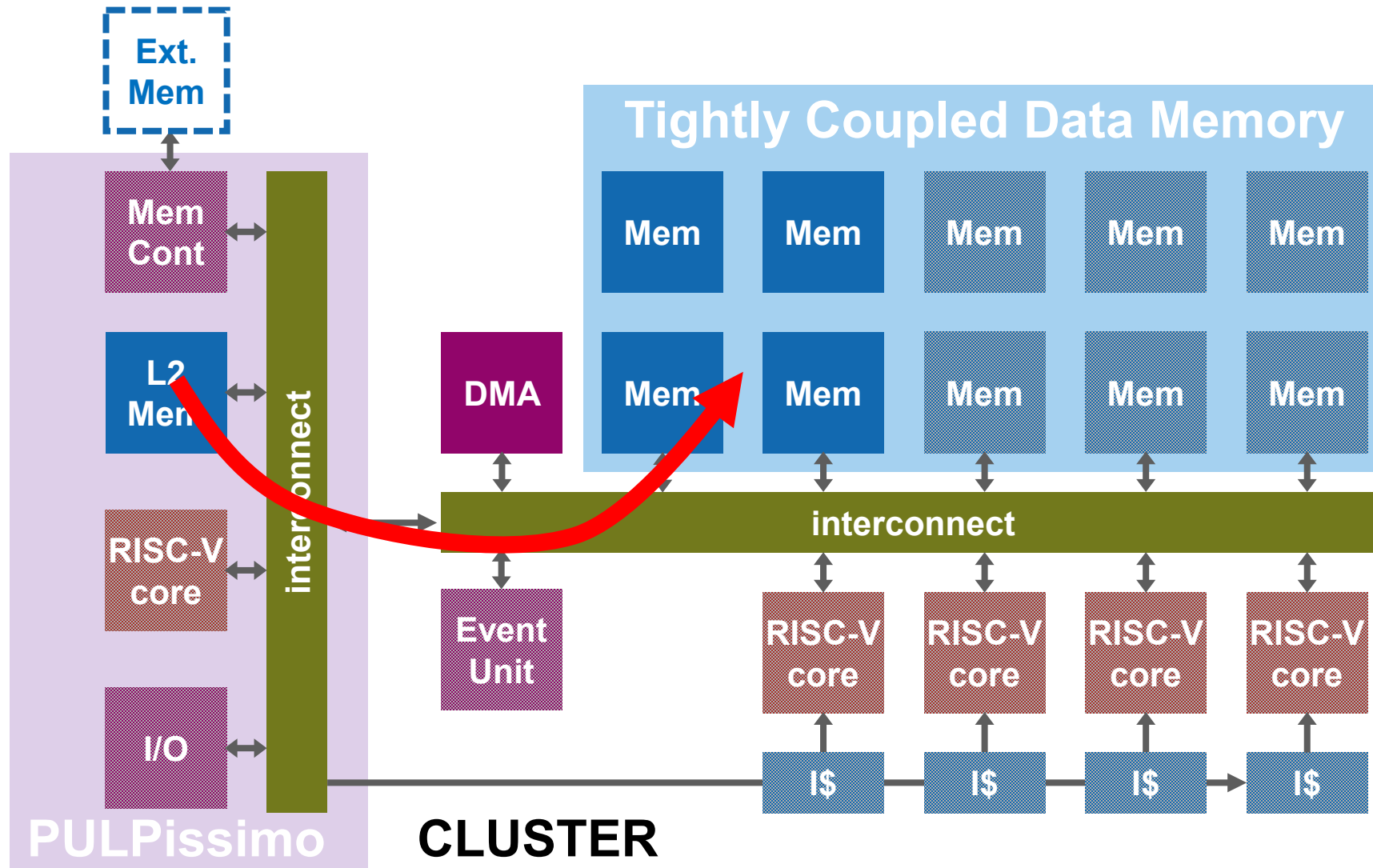
An additional I/O controller for IO, off-chip Memory



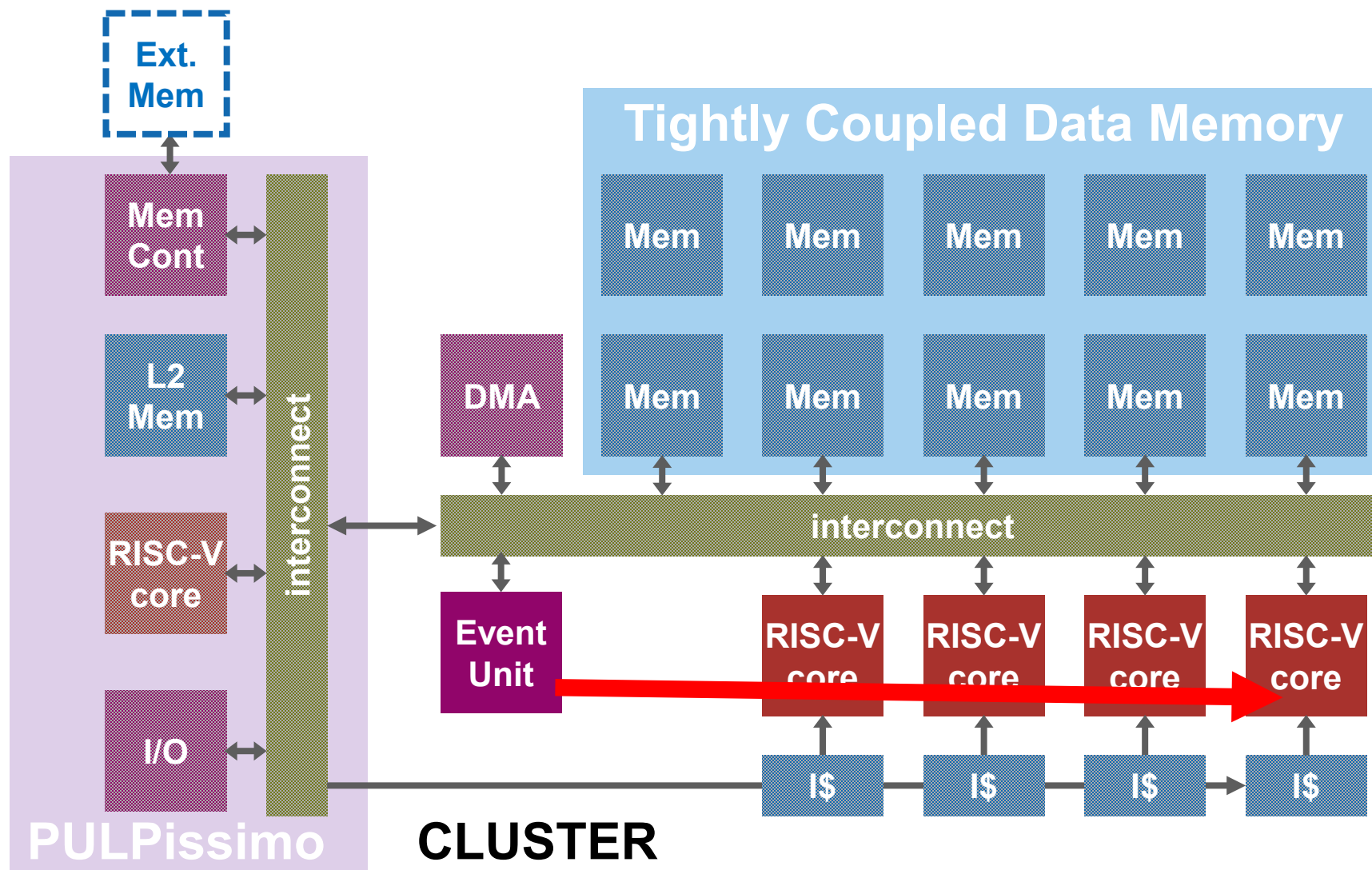
How do we work: Initiate a DMA transfer



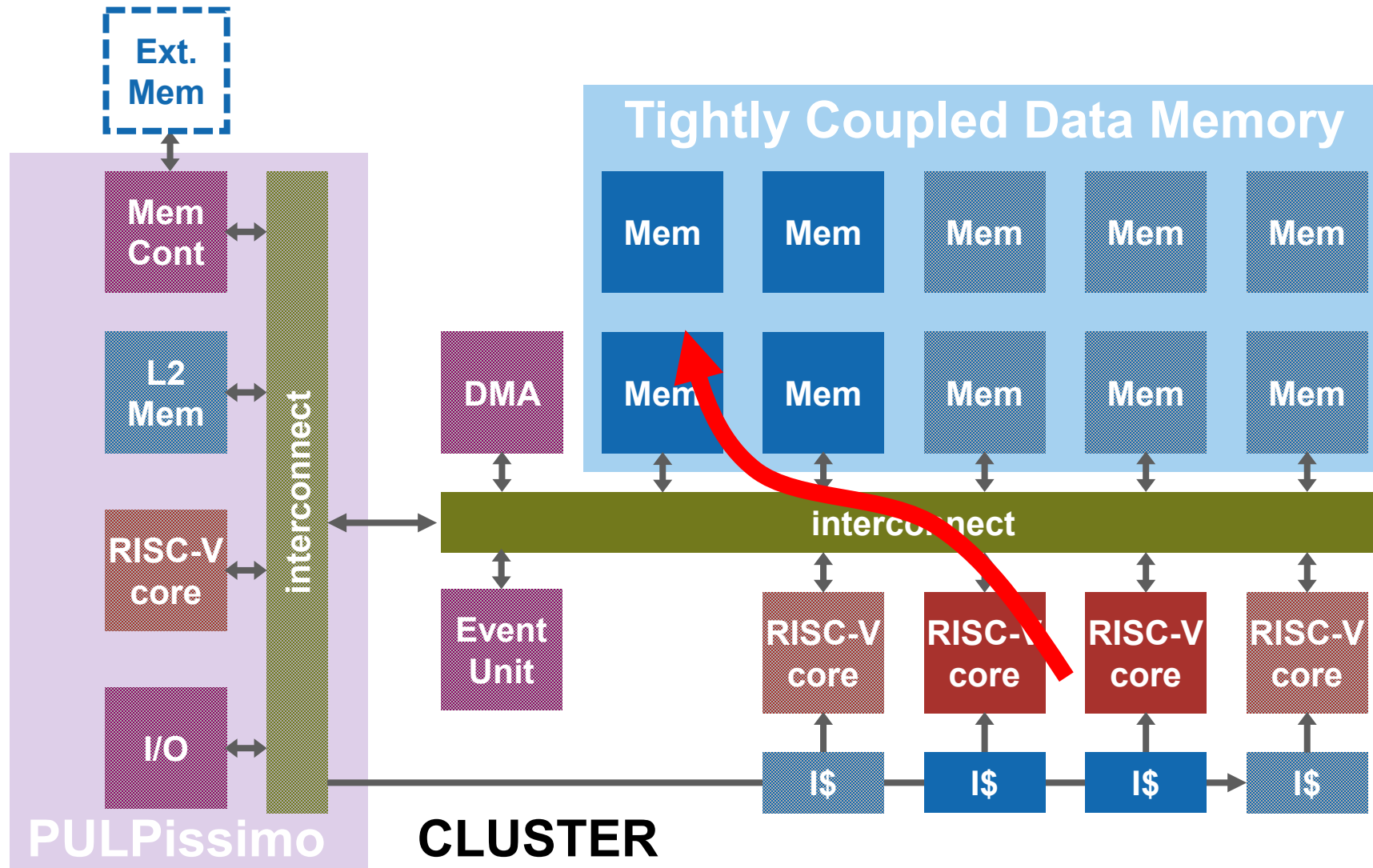
Data copied from L2 into TCDM



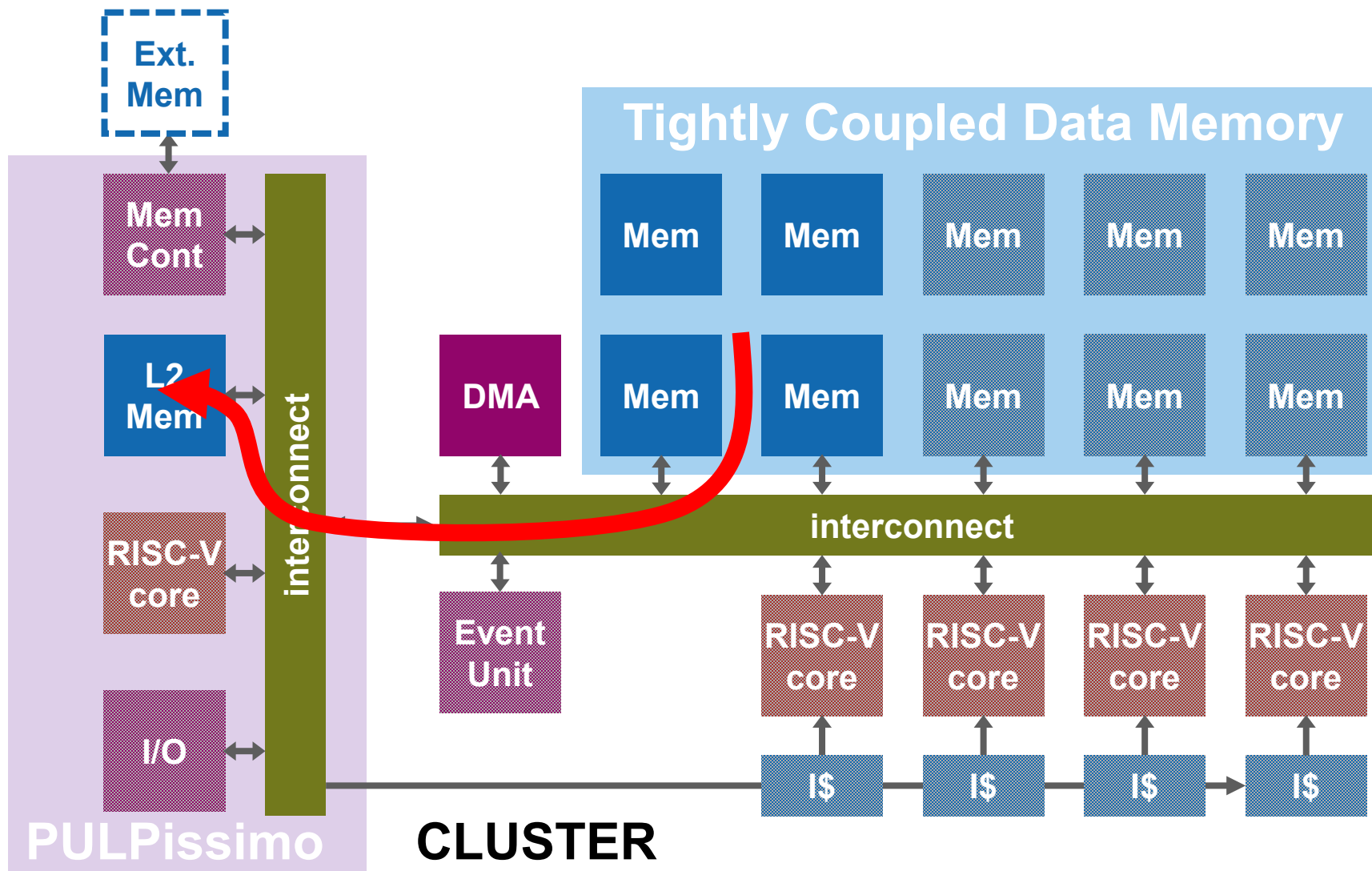
Once data is transferred, event unit notifies cores



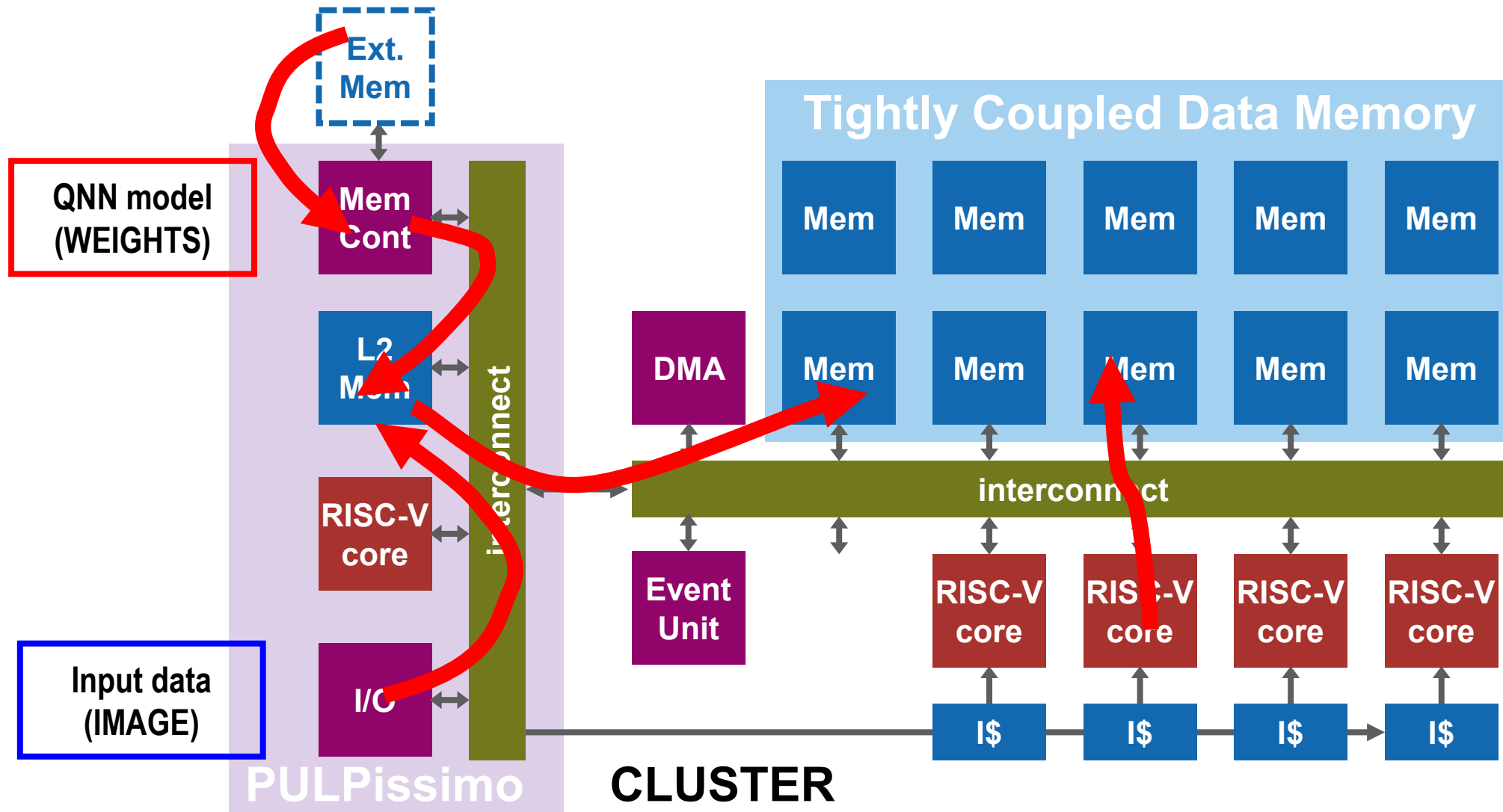
Cores can work on the data transferred



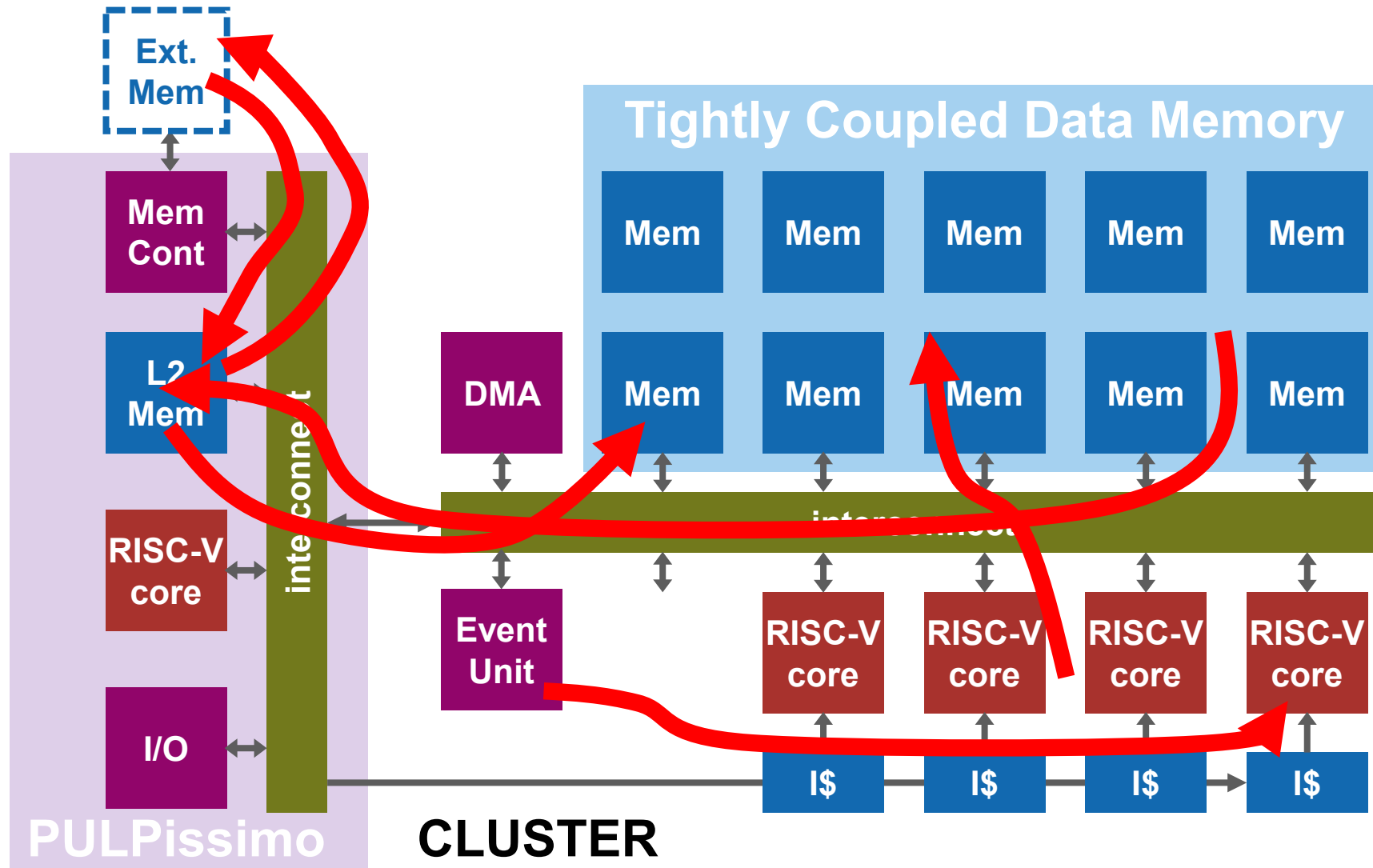
Once our work is done, DMA copies data back



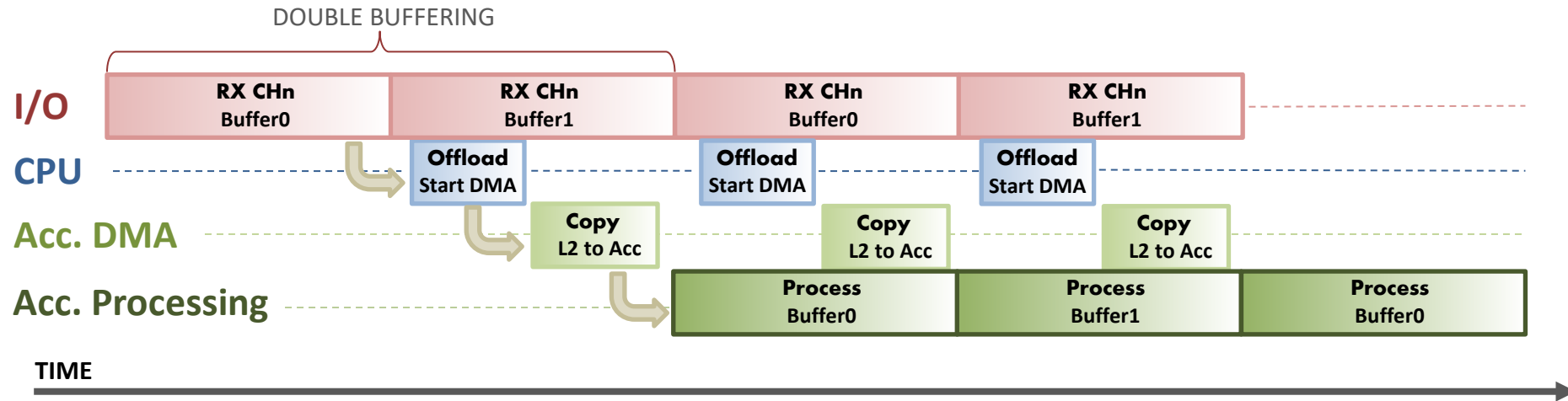
The IO DMA Subsystem Supports Triple Buffering



During normal operation all occur concurrently



A typical processing pipeline for end-to-end inference of QNNs on edge AI processors



- Efficient use of system resources
- HW support for double buffering allows continuous data transfers
- Multiple data streams can be time multiplexed

How to execute a CONV layer on a parallel (multi-core) computing architecture

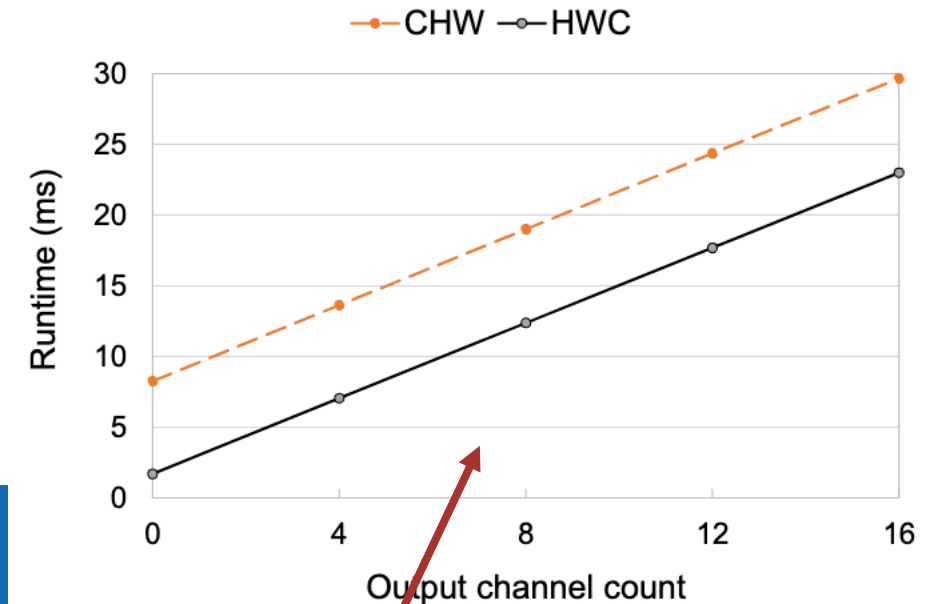
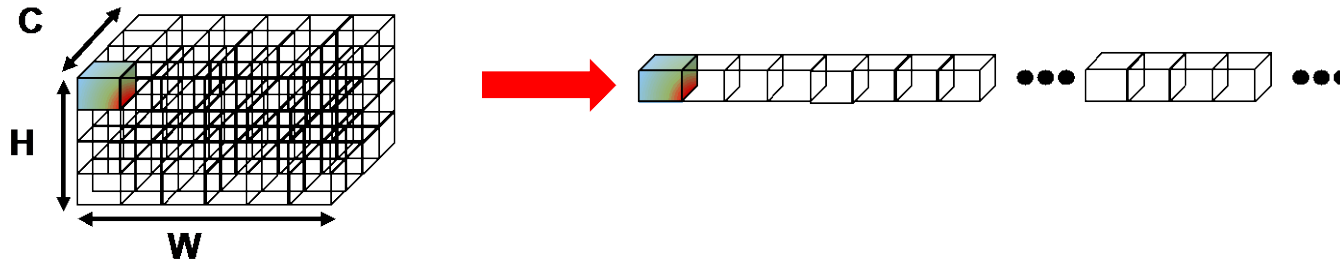
PULP-NN: Optimized Back-End

Target **int8** execution of CONV, FC, ... primitives

1) maximize **data reuse in register file** 2) improve **kernel regularity** 3) exploit **parallelism**

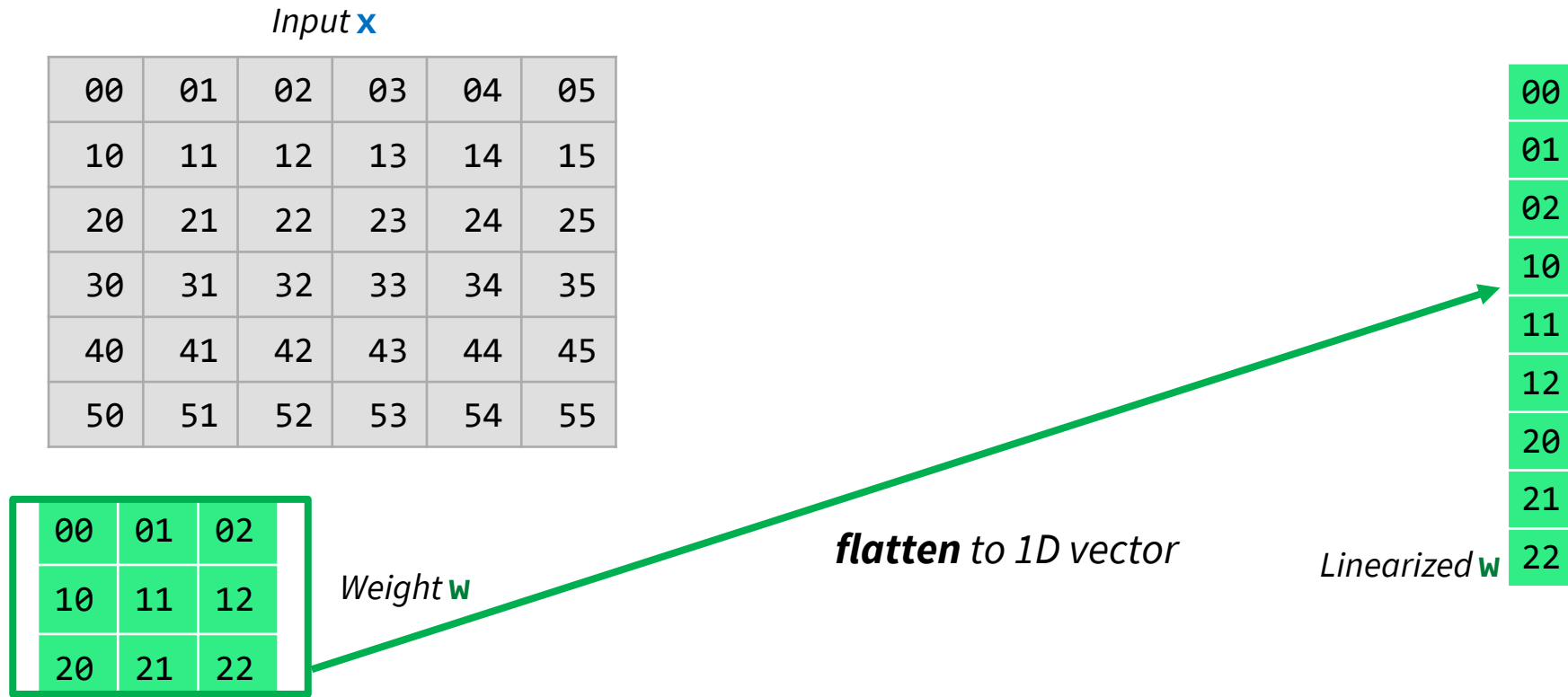


HWC format



- Usually, two types of memory layouts: CHW, HWC
- Data layout does not directly affect the matrix-multiplication performance, but..
- HWC layout enable efficient data movement with SIMD instructions – im2col is faster

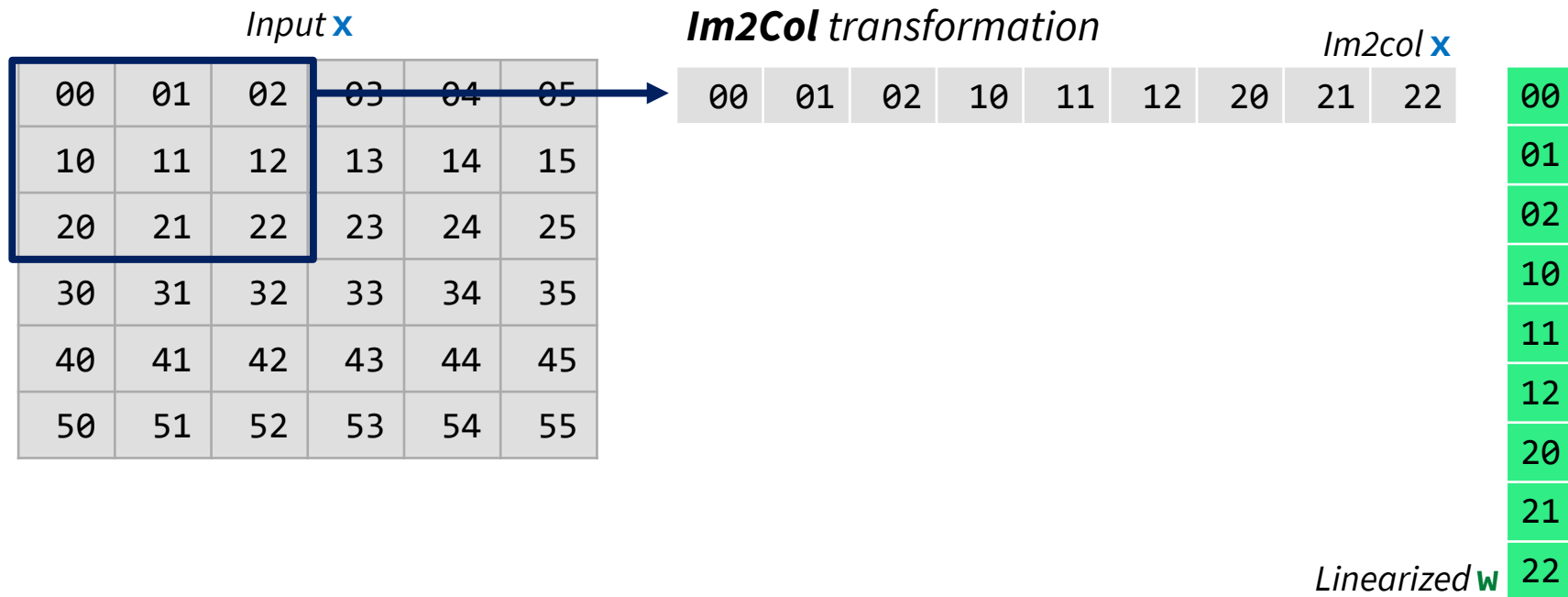
2D conv as a GEMV



$$y[i,j] = \Sigma(w[0:S,0:R] * x[i:i+S,j:j+R])$$

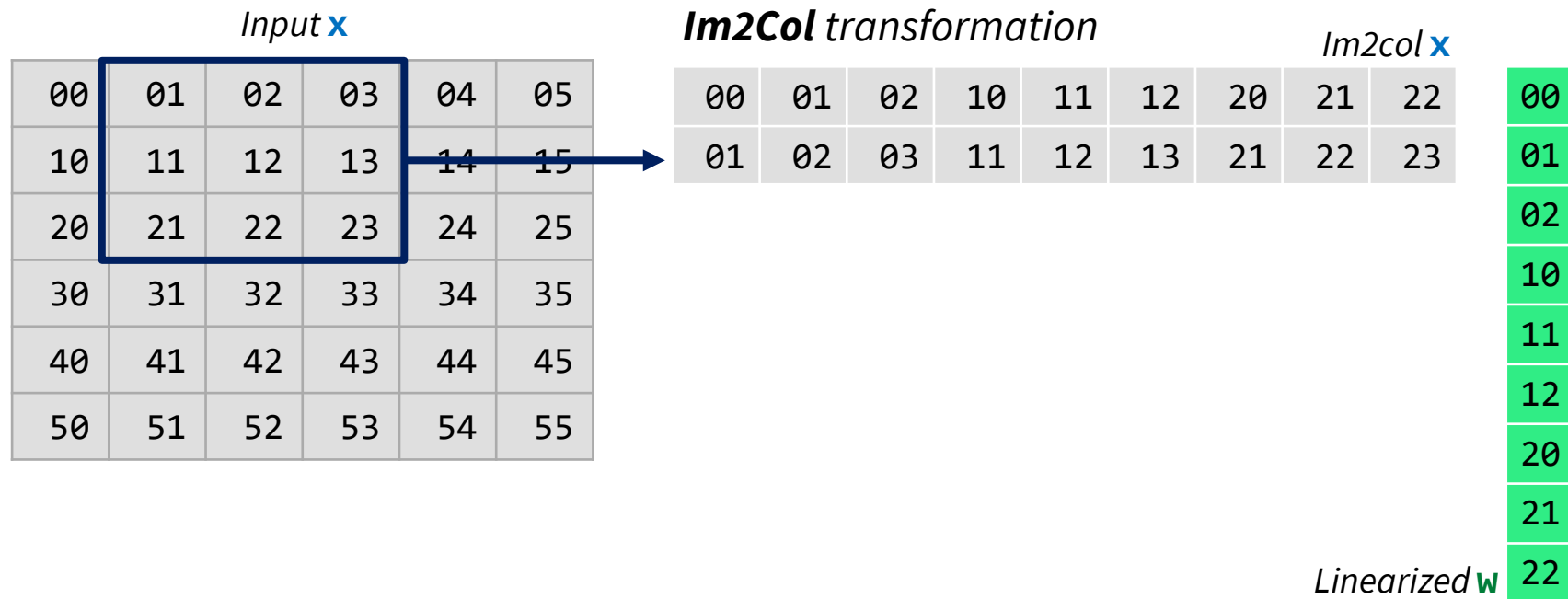
2D conv as a GEMV

Let's now consider $C=1$ (only 1 IFM channel) and $K=1$ (only one conv kernel)



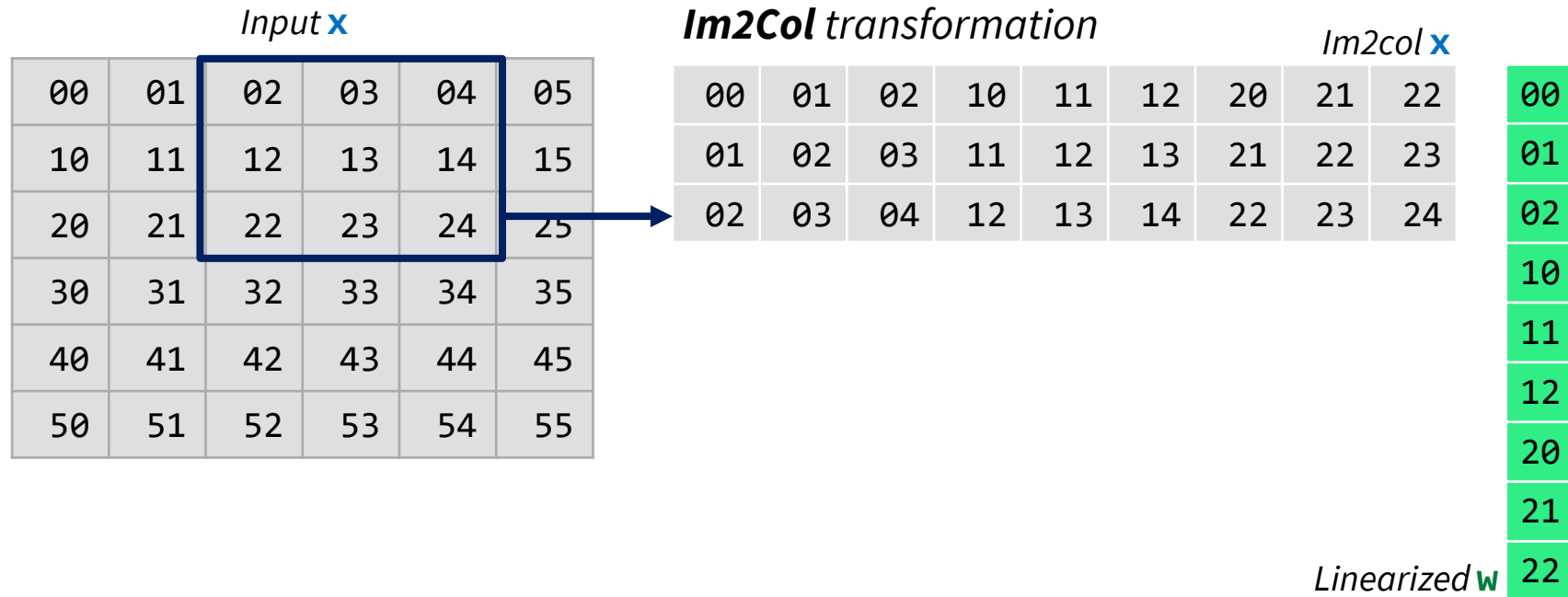
$$y[i,j] = \sum(w[0:S,0:R] * x[i:i+S,j:j+R])$$

2D conv as a GEMV



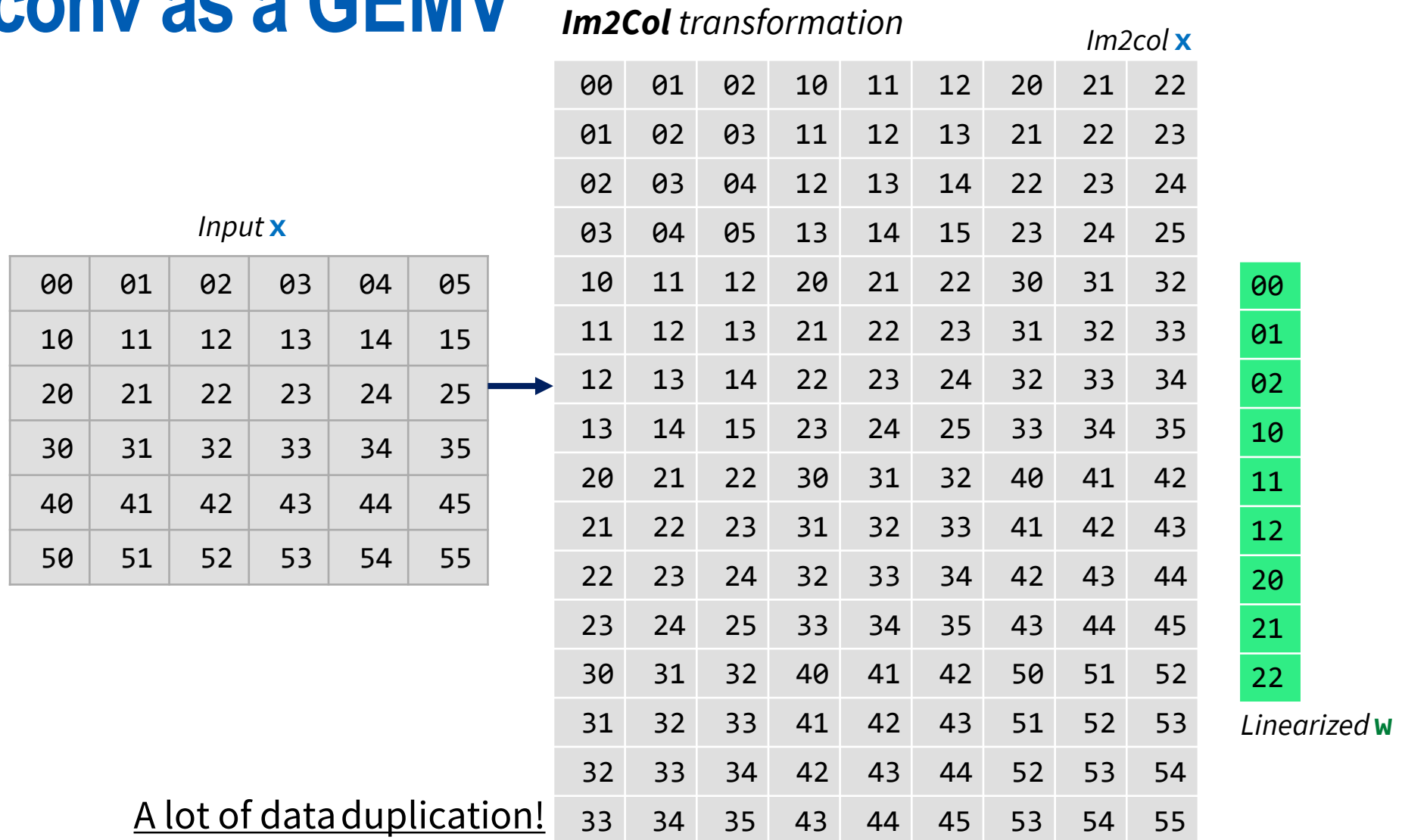
$$y[i,j] = \sum(w[0:S,0:R] * x[i:i+S,j:j+R])$$

2D conv as a GEMV



$$y[i,j] = \sum(w[0:S,0:R] * x[i:i+S,j:j+R])$$

2D conv as a GEMV



$$y[i,j] = \sum(w[0:S,0:R] * x[i:i+S,j:j+R])$$

2D conv as a GEMV

00	01	02	10	11	12	20	21	22
01	02	03	11	12	13	21	22	23
02	03	04	12	13	14	22	23	24
03	04	05	13	14	15	23	24	25
10	11	12	20	21	22	30	31	32
11	12	13	21	22	23	31	32	33
12	13	14	22	23	24	32	33	34
13	14	15	23	24	25	33	34	35
20	21	22	30	31	32	40	41	42
21	22	23	31	32	33	41	42	43
22	23	24	32	33	34	42	43	44
23	24	25	33	34	35	43	44	45
30	31	32	40	41	42	50	51	52
31	32	33	41	42	43	51	52	53
32	33	34	42	43	44	52	53	54
33	34	35	43	44	45	53	54	55

Im2col $\mathbf{x}$

• $\begin{bmatrix} 00 \\ 01 \\ 02 \\ 10 \\ 11 \\ 12 \\ 20 \\ 21 \\ 22 \end{bmatrix}$ = $\begin{bmatrix} 00 \\ 01 \\ 02 \\ 10 \\ 11 \\ 12 \\ 20 \\ 21 \\ 22 \end{bmatrix}$
Linearized $\mathbf{w}$

00
01
02
03
10
11
12
13
20
21
22
23
30
31
32
33

Linearized $\mathbf{y}$

00	01	02	03
10	11	12	13
20	21	22	23
30	31	32	33

Output $\mathbf{y}$

$$\mathbf{y}[\mathbf{i}, \mathbf{j}] = \Sigma(\mathbf{w}[\mathbf{0}:\mathbf{S}, \mathbf{0}:\mathbf{R}] * \mathbf{x}[\mathbf{i}:\mathbf{i}+\mathbf{S}, \mathbf{j}:\mathbf{j}+\mathbf{R}])$$

CONV Layer as a GEMM

Let's bring in $C > 1$, still considering $M=1$ for simplicity

Input $\mathbf{x}$

00	01	02	03	04	05
10	11	12	13	14	15
20	21	22	23	24	25
30	31	32	33	34	35
40	41	42	43	44	45
50	51	52	53	54	55

00	01	02
10	11	12
20	21	22

Weight $\mathbf{w}$

flatten to 1D vector

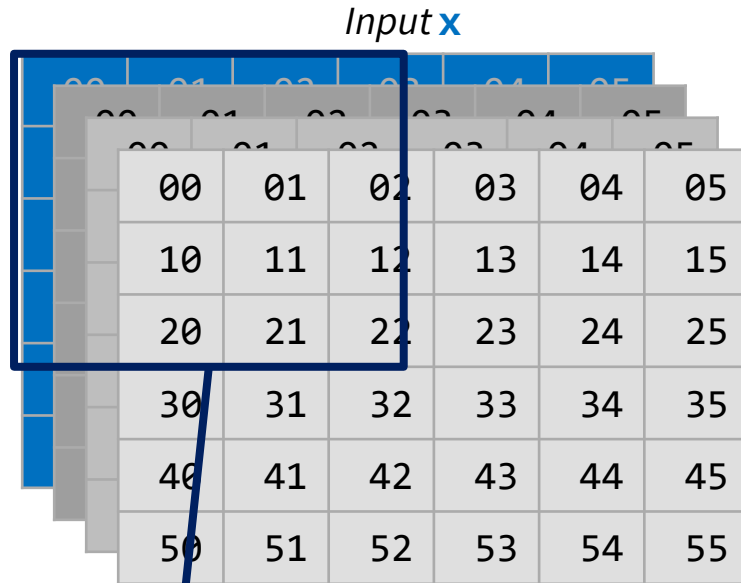
Linearized $\mathbf{w}$

00
01
02
10
11
12
20
21
22
...
21
22
00
01
02
10
11
12
20
21
22

$$y[m, i, j] = \Sigma(w[m, 0:C, 0:S, 0:R] * x[0:C, i:i+S, j:j+R])$$

CONV Layer as a GEMM

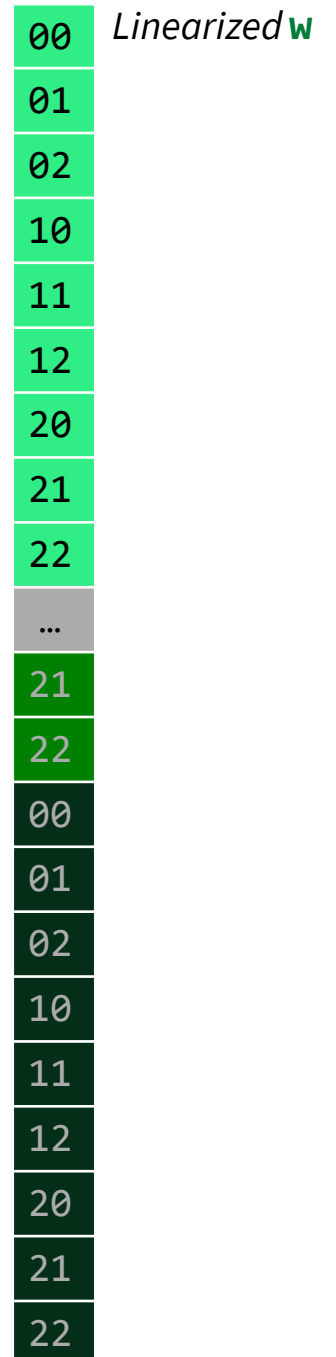
Considering M=1 for simplicity



$\downarrow$ **Im2Col** transformation



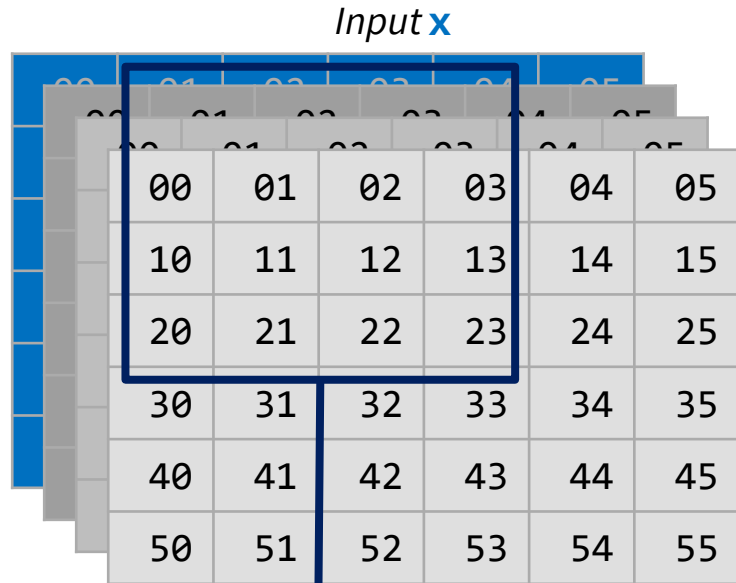
Im2col x



$$y[m, i, j] = \Sigma(w[m, 0:C, 0:S, 0:R] * x[0:C, i:i+S, j:j+R])$$

CONV Layer as a GEMM

Considering M=1 for simplicity



Im2Col transformation

00	01	02	10	11	12	20	21	22	...	21	22	00	01	02	10	11	12	20	21	22
01	02	03	11	12	13	21	22	23		22	23	01	02	03	11	12	13	21	22	23

Im2col x

00	Linearized w
01	
02	
10	
11	
12	
20	
21	
22	
...	
21	
22	
00	
01	
02	
10	
11	
12	
20	
21	
22	

$$y[m, i, j] = \Sigma(w[m, 0:C, 0:S, 0:R] * x[0:C, i:i+S, j:j+R])$$

CONV Layer as a GEMM

00	01	02	10	11	12	20	21	22		21	22	00	01	02	10	11	12	20	21	22
01	02	03	11	12	13	21	22	23		22	23	01	02	03	11	12	13	21	22	23
02	03	04	12	13	14	22	23	24		23	24	02	03	04	12	13	14	22	23	24
03	04	05	13	14	15	23	24	25		24	25	03	04	05	13	14	15	23	24	25
10	11	12	20	21	22	30	31	32		31	32	10	11	12	20	21	22	30	31	32
11	12	13	21	22	23	31	32	33		32	33	11	12	13	21	22	23	31	32	33
12	13	14	22	23	24	32	33	34		33	34	12	13	14	22	23	24	32	33	34
13	14	15	23	24	25	33	34	35		34	35	13	14	15	23	24	25	33	34	35
20	21	22	30	31	32	40	41	42	...	41	42	20	21	22	30	31	32	40	41	42
21	22	23	31	32	33	41	42	43		42	43	21	22	23	31	32	33	41	42	43
22	23	24	32	33	34	42	43	44		43	44	22	23	24	32	33	34	42	43	44
23	24	25	33	34	35	43	44	45		44	45	23	24	25	33	34	35	43	44	45
30	31	32	40	41	42	50	51	52		51	52	30	31	32	40	41	42	50	51	52
31	32	33	41	42	43	51	52	53		52	53	31	32	33	41	42	43	51	52	53
32	33	34	42	43	44	52	53	54		53	54	32	33	34	42	43	44	52	53	54
33	34	35	43	44	45	53	54	55		54	55	33	34	35	43	44	45	53	54	55

Im2col x

Considering M=1 for simplicity

$$y[m, i, j] = \sum (w[m, 0:C, 0:S, 0:R] * x[0:C, i:i+S, j:j+R])$$

00	Linearized w
01	
02	
10	
11	
12	
20	
21	
22	
...	
21	
22	
00	
01	
02	
10	
11	
12	
20	
21	
22	

CONV Layer as a GEMM

With $M > 1$

00	01	02	10	11	12	20	21	22		21	22	00	01	02	10	11	12	20	21	22
01	02	03	11	12	13	21	22	23		22	23	01	02	03	11	12	13	21	22	23
02	03	04	12	13	14	22	23	24		23	24	02	03	04	12	13	14	22	23	24
03	04	05	13	14	15	23	24	25		24	25	03	04	05	13	14	15	23	24	25
10	11	12	20	21	22	30	31	32		31	32	10	11	12	20	21	22	30	31	32
11	12	13	21	22	23	31	32	33		32	33	11	12	13	21	22	23	31	32	33
12	13	14	22	23	24	32	33	34		33	34	12	13	14	22	23	24	32	33	34
13	14	15	23	24	25	33	34	35		34	35	13	14	15	23	24	25	33	34	35
20	21	22	30	31	32	40	41	42	...	41	42	20	21	22	30	31	32	40	41	42
21	22	23	31	32	33	41	42	43		42	43	21	22	23	31	32	33	41	42	43
22	23	24	32	33	34	42	43	44		43	44	22	23	24	32	33	34	42	43	44
23	24	25	33	34	35	43	44	45		44	45	23	24	25	33	34	35	43	44	45
30	31	32	40	41	42	50	51	52		51	52	30	31	32	40	41	42	50	51	52
31	32	33	41	42	43	51	52	53		52	53	31	32	33	41	42	43	51	52	53
32	33	34	42	43	44	52	53	54		53	54	32	33	34	42	43	44	52	53	54
33	34	35	43	44	45	53	54	55		54	55	33	34	35	43	44	45	53	54	55

$Im2col \times$

$$y[m, i, j] = \sum (w[m, 0:C, 0:S, 0:R] * x[0:C, i:i+S, j:j+R])$$

Matrix w

00	00	00		00
01	01	01		01
02	02	02		02
10	10	10		10
11	11	11		11
12	12	12		12
20	20	20		20
21	21	21		21
22	22	22		22
			...	
21	21	21		21
22	22	22		22
00	00	00		00
01	01	01		01
02	02	02		02
10	10	10		10
11	11	11		11
12	12	12		12
20	20	20		20
21	21	21		21
22	22	22		22

PULP-NN: optimized computational back-end

- Standard output stationary loop nest, HWC layout for activations, MHWC for weights

```
for i in range(0, E):  
    for j in range(0, F):  
        for m in range(0, M):  
            psum = 0                                # ignore bias  
            for ui in range(0, S):  
                for uj in range(0, R):  
                    for n in range(0, C):  
                        psum += w[m,ui,uj,n] * x[i+ui,j+uj,n]  
            y[i,j,m] = act(psum)
```

PULP-NN: optimized computational back-end

- Standard output stationary loop nest, HWC layout for activations, CoHWCi for weights

```
for i in range(0, E):
    for j in range(0, F):
        for m in range(0, M):
            psum = 0                                # ignore bias
            for ui in range(0, S):
                for uj in range(0, R):
                    for n in range(0, C):
                        psum += w[m,ui,uj,n] * x[i+ui,j+uj,n]
            y[i,j,m] = act(psum)
```

1. Make x access more regular: reorder in **im2col** buffer

PULP-NN: optimized computational back-end

- Standard output stationary loop nest, HWC layout for activations, CoHWCi for weights

```
for i in range(0, E):
    for j in range(0, F):
        for m in range(0, M):
            for ui in range(0, S):
                for uj in range(0, R):
                    for n in range(0, C):
                        im2col[ui,uj,n] = x[i+ui,j+uj,n]
                        psum = 0 # ignore bias
                    for ui in range(0, S):
                        for uj in range(0, R):
                            for n in range(0, C):
                                psum += w[m,ui,uj,n] * im2col[ui,uj,n]
                        y[i,j,m] = act(psum)
```

1. Make **x** access more regular: reorder in **im2col** buffer

PULP-NN: optimized computational back-end

- Standard output stationary loop nest, HWC layout for activations, CoHWCi for weights

```
for i in range(0, E):
    for j in range(0, F):
        for m in range(0, M):
            for ui in range(0, S):
                for uj in range(0, R):
                    for n in range(0, C):
                        im2col[ui,uj,n] = x[i+ui,j+uj,n]
                    psum = 0 # ignore bias
                    for idx in range(0, S*R*C):
                        psum += w[m,idx] * im2col[idx]
                    y[i,j,m] = act(psum)
```

2. *im2col* , *w* access are linear (and identical)

PULP-NN: optimized computational back-end

- Standard output stationary loop nest, HWC layout for activations, CoHWCi for weights

```
for i in range(0, E):
    for j in range(0, F):
        for m in range(0, M):
            for ui in range(0, S):
                for uj in range(0, R):
                    for n in range(0, C):
                        im2col[ui,uj,n] = x[i+ui,j+uj,n]
                    psum = 0 # ignore bias
                    for idx in range(0, S*R*C):
                        psum += w[m,idx] * im2col[idx]
                    y[i,j,m] = act(psum)
```

3. Highlighted kernel is a **Matrix Mul!**

PULP-NN: Optimized Back-End

Target **int8** execution of CONV, FC, ... primitives

1) maximize **data reuse in register file** 2) improve **kernel regularity** 3) exploit **parallelism**

HWC format

lp.setup

p.lw w0, 4(W0!)

p.lw w1, 4(W1!)

p.lw w2, 4(W2!)

p.lw w3, 4(W3!)

p.lw x1, 4(X0!)

p.lw x2, 4(X1!)

pv.sdotsp.b acc1, w0, x0

pv.sdotsp.b acc2, w0, x1

pv.sdotsp.b acc3, w1, x0

pv.sdotsp.b acc4, w1, x1

pv.sdotsp.b acc5, w2, x0

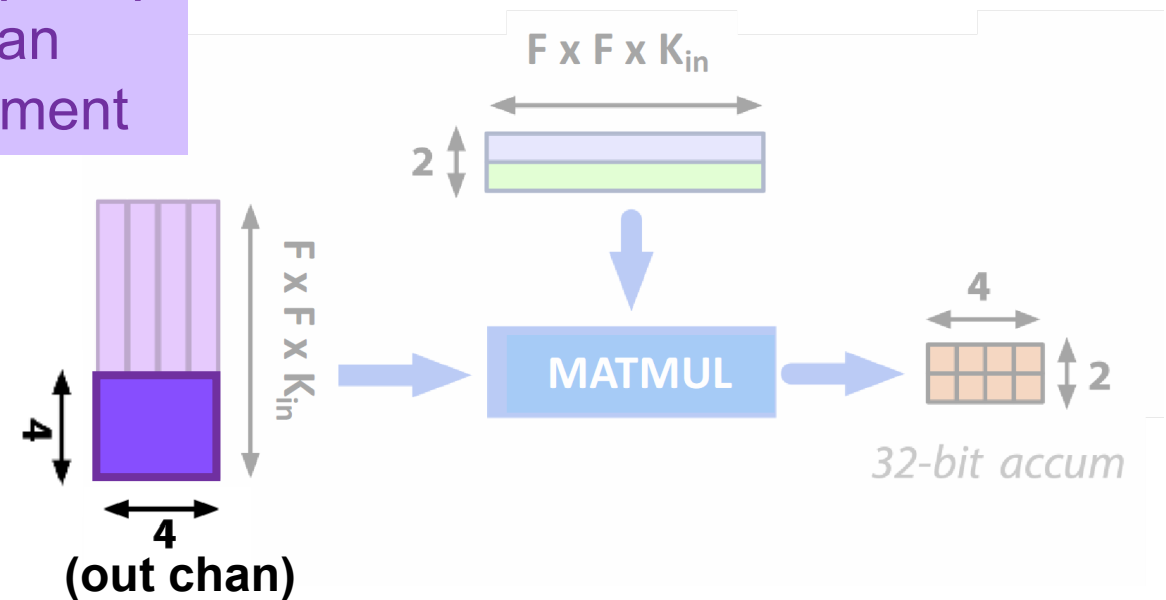
pv.sdotsp.b acc6, w2, x1

pv.sdotsp.b acc7, w3, x0

pv.sdotsp.b acc8, w3, x1

end

Load 16 weights (8-bit)
4 out chan, 4 in chan
address post-increment



PULP-NN: Optimized Back-End

Target **int8** execution of CONV, FC, ... primitives

1) maximize **data reuse in register file** 2) improve **kernel regularity** 3) exploit **parallelism**

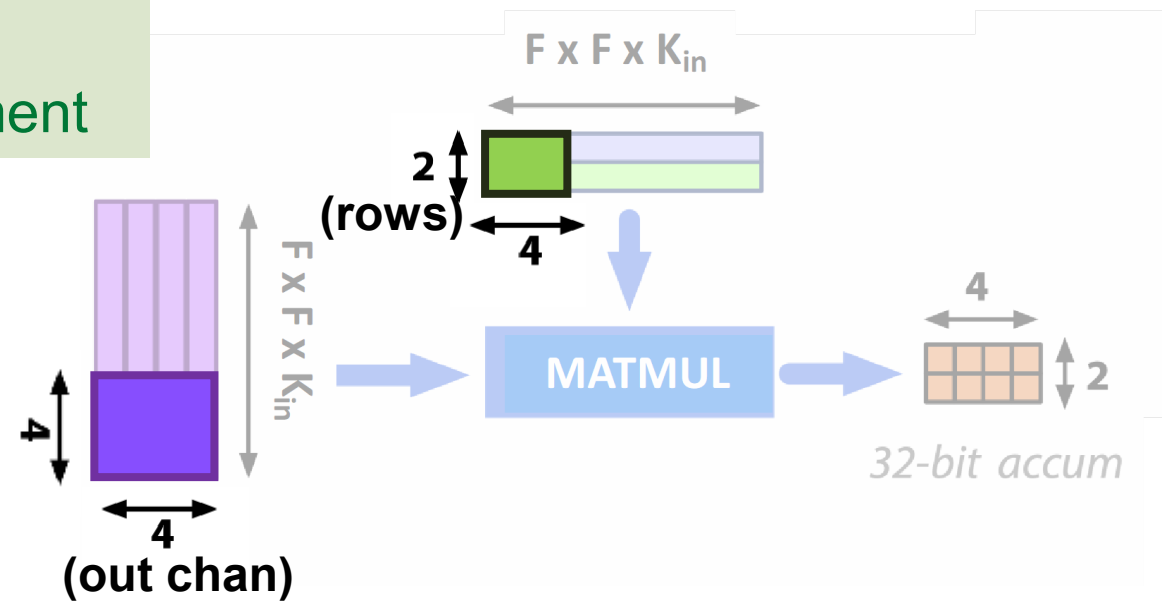
lp.setup

```
p.lw w0, 4(W0!)  
p.lw w1, 4(W1!)  
p.lw w2, 4(W2!)  
p.lw w3, 4(W3!)  
p.lw x1, 4(X0!)  
p.lw x2, 4(X1!)
```

```
pv.sdotsp.b acc1, w0, x0  
pv.sdotsp.b acc2, w0, x1  
pv.sdotsp.b acc3, w1, x0  
pv.sdotsp.b acc4, w1, x1  
pv.sdotsp.b acc5, w2, x0  
pv.sdotsp.b acc6, w2, x1  
pv.sdotsp.b acc7, w3, x0  
pv.sdotsp.b acc8, w3, x1
```

end

Load 8 pixels
2 rows, 4 in chan
address post-increment



PULP-NN: Optimized Back-End

Target **int8** execution of CONV, FC, ... primitives

1) maximize **data reuse in register file** 2) improve **kernel regularity** 3) exploit **parallelism**

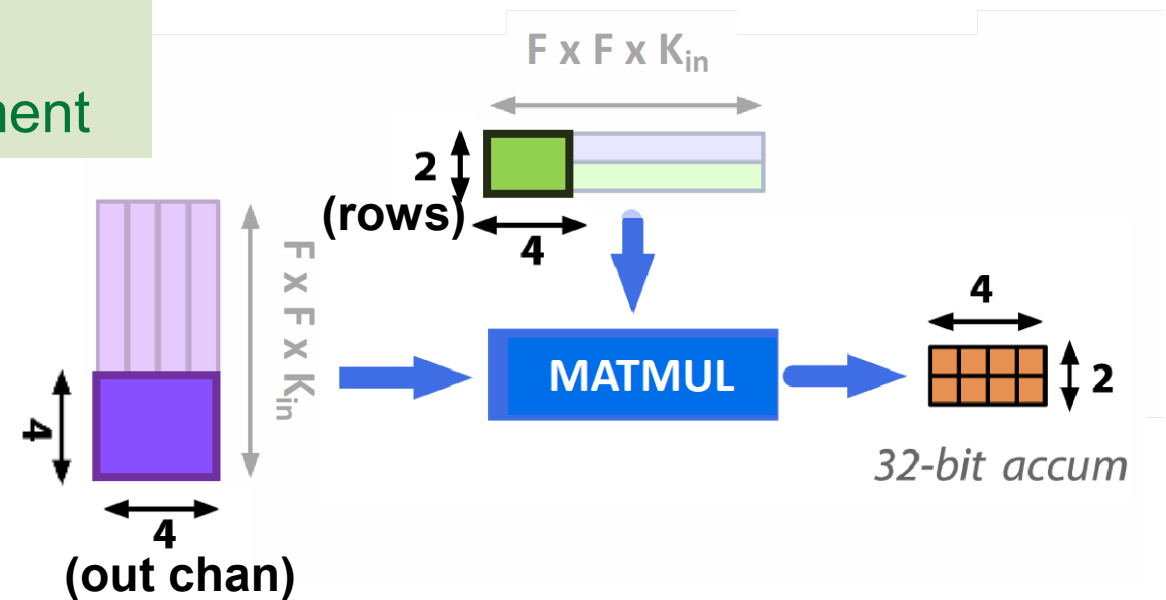
lp.setup

```
p.lw w0, 4(W0!)  
p.lw w1, 4(W1!)  
p.lw w2, 4(W2!)  
p.lw w3, 4(W3!)  
p.lw x1, 4(X0!)  
p.lw x2, 4(X1!)
```

```
pv.sdotsp.b acc1, w0, x0  
pv.sdotsp.b acc2, w0, x1  
pv.sdotsp.b acc3, w1, x0  
pv.sdotsp.b acc4, w1, x1  
pv.sdotsp.b acc5, w2, x0  
pv.sdotsp.b acc6, w2, x1  
pv.sdotsp.b acc7, w3, x0  
pv.sdotsp.b acc8, w3, x1
```

end

Load 8 pixels
2 rows, 4 in chan
address post-increment



Compute 32 MAC over 8 accumulators
dot-product instructions

PULP-NN: Optimized Back-End

Target **int8** execution of CONV, FC, ... primitives

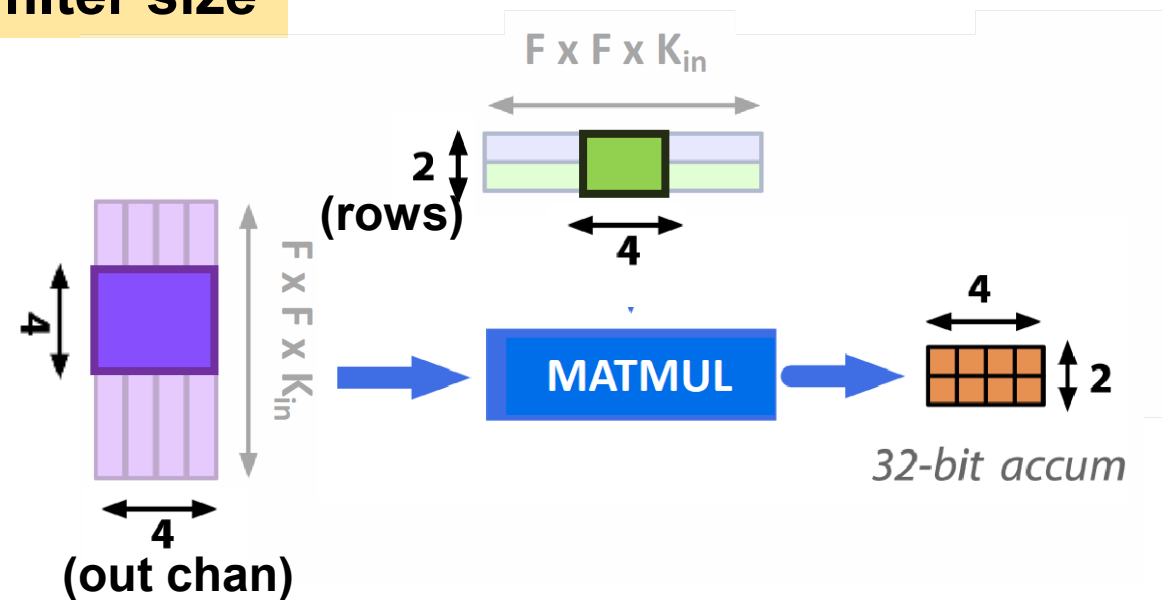
1) maximize **data reuse in register file** 2) improve **kernel regularity** 3) exploit **parallelism**

lp.setup

```
p.lw w0, 4(W0!)  
p.lw w1, 4(W1!)  
p.lw w2, 4(W2!)  
p.lw w3, 4(W3!)  
p.lw x1, 4(X0!)  
p.lw x2, 4(X1!)  
pv.sdotsp.b acc1, w0, x0  
pv.sdotsp.b acc2, w0, x1  
pv.sdotsp.b acc3, w1, x0  
pv.sdotsp.b acc4, w1, x1  
pv.sdotsp.b acc5, w2, x0  
pv.sdotsp.b acc6, w2, x1  
pv.sdotsp.b acc7, w3, x0  
pv.sdotsp.b acc8, w3, x1
```

end

Loop over in chan, filter size



PULP-NN: Optimized Back-End

Target **int8** execution of CONV, FC, ... primitives

1) maximize **data reuse in register file** 2) improve **kernel regularity** 3) exploit **parallelism**

```
lp.setup
```

```
  p.lw  w0, 4(W0!)
```

```
  p.lw  w1, 4(W1!)
```

```
  p.lw  w2, 4(W2!)
```

```
  p.lw  w3, 4(W3!)
```

```
  p.lw  x1, 4(X0!)
```

```
  p.lw  x2, 4(X1!)
```

```
  pv.sdotsp.b  acc1, w0, x0
```

```
  pv.sdotsp.b  acc2, w0, x1
```

```
  pv.sdotsp.b  acc3, w1, x0
```

```
  pv.sdotsp.b  acc4, w1, x1
```

```
  pv.sdotsp.b  acc5, w2, x0
```

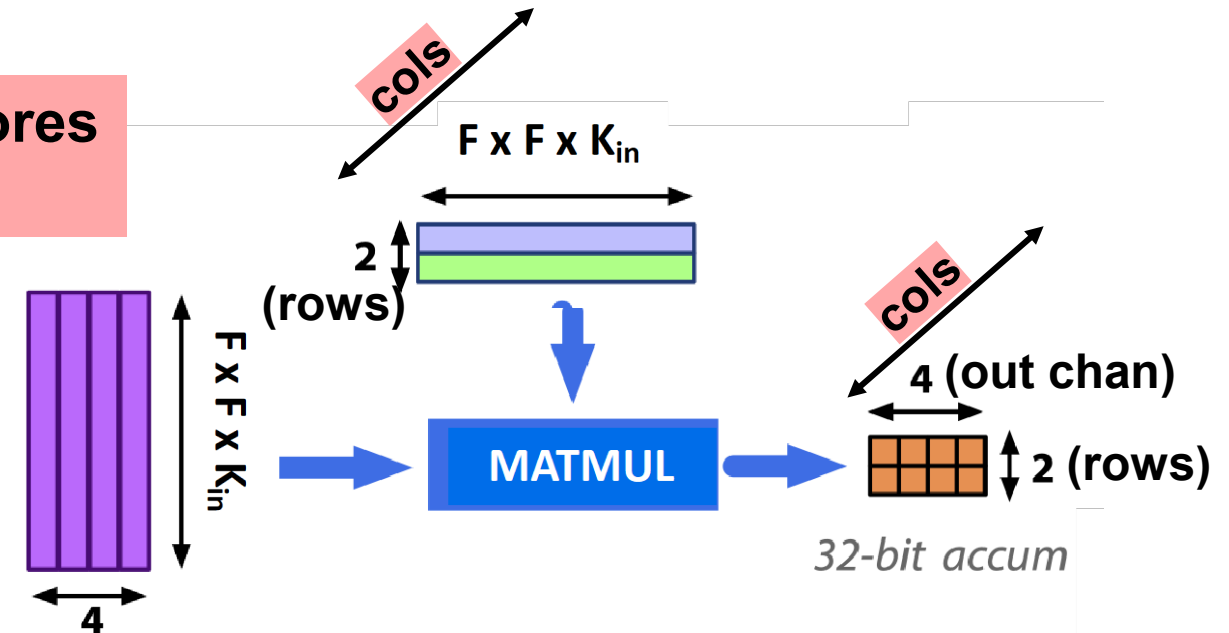
```
  pv.sdotsp.b  acc6, w2, x1
```

```
  pv.sdotsp.b  acc7, w3, x0
```

```
  pv.sdotsp.b  acc8, w3, x1
```

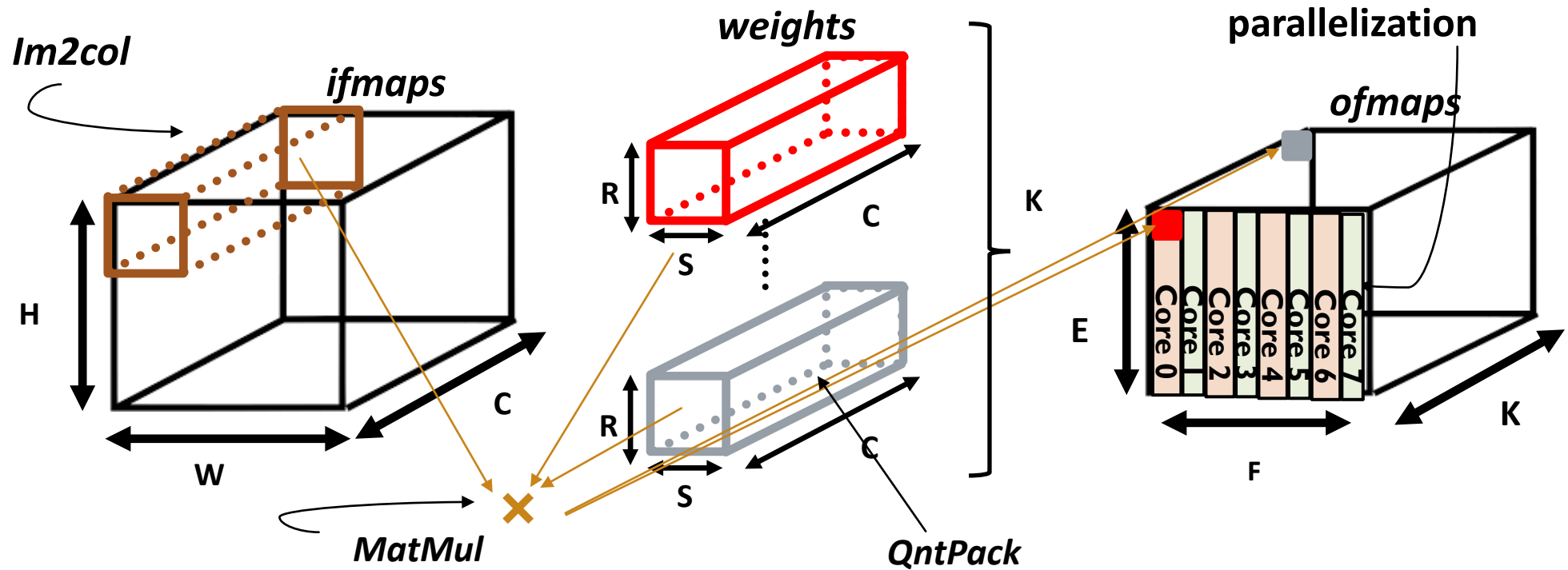
```
end
```

Parallelize over 8 cores
column dimension

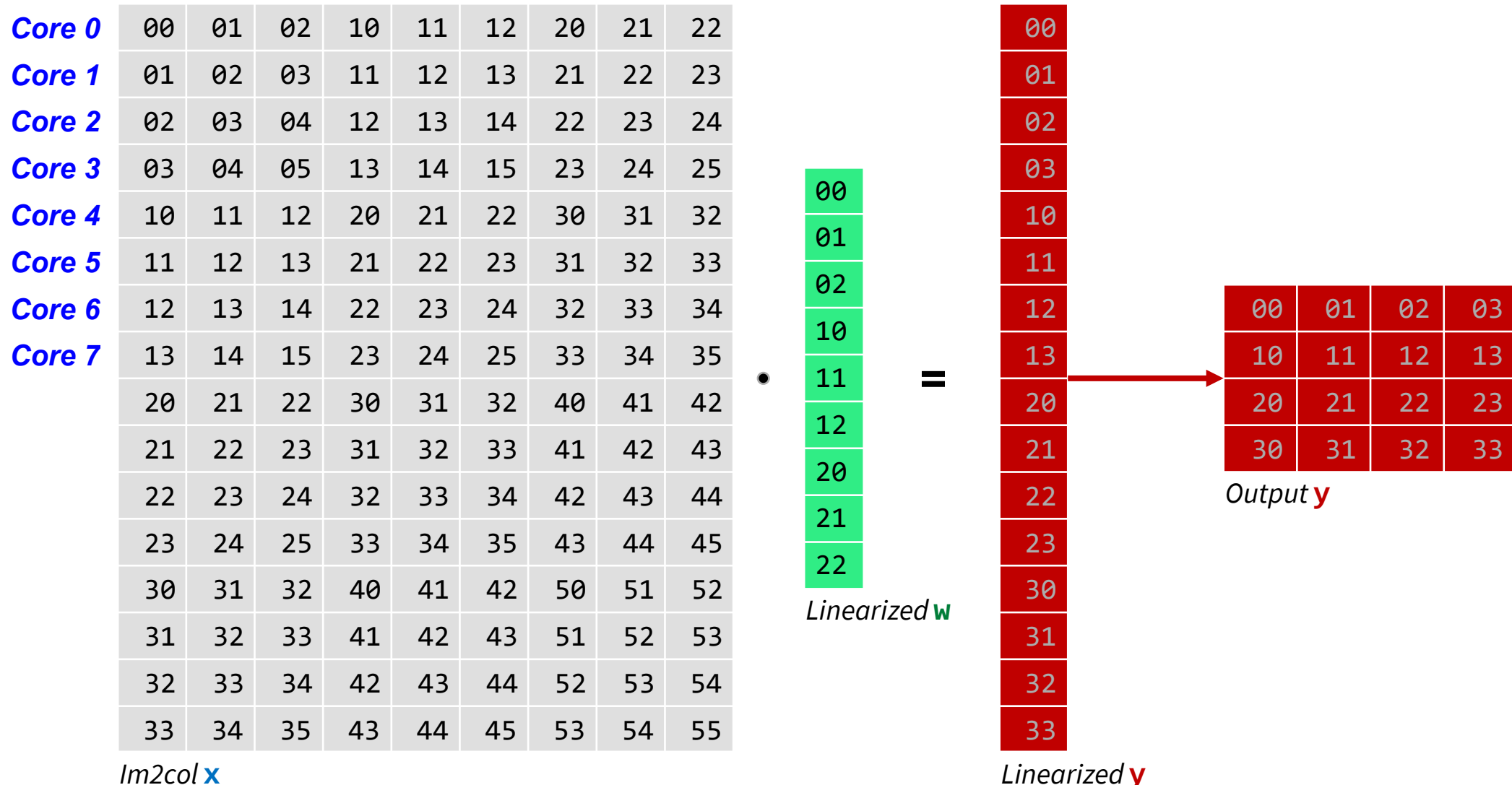


Parallel Execution of a Conv Kernel

Convolution kernel



Key idea: distribute cols/rows across processors



$$y[i,j] = \Sigma(w[0:F,0:F] * x[i:i+F,j:j+F])$$

More on Parallel Execution and The Amdhal's Law

Task parallelism

```
for(i=0; i<N; i++) {  
    res1[i] = a[i] + b[i];  
}
```

```
for(i=0; i<N; i++) {  
    res2[i] = 2 * a[i] * b[i];  
}
```

Synchronization barrier

```
for(i=0; i<N; i++) {  
    res3[i] = sin(res2[i]+3.0);  
}
```

Task 1

Task 2

Task 3

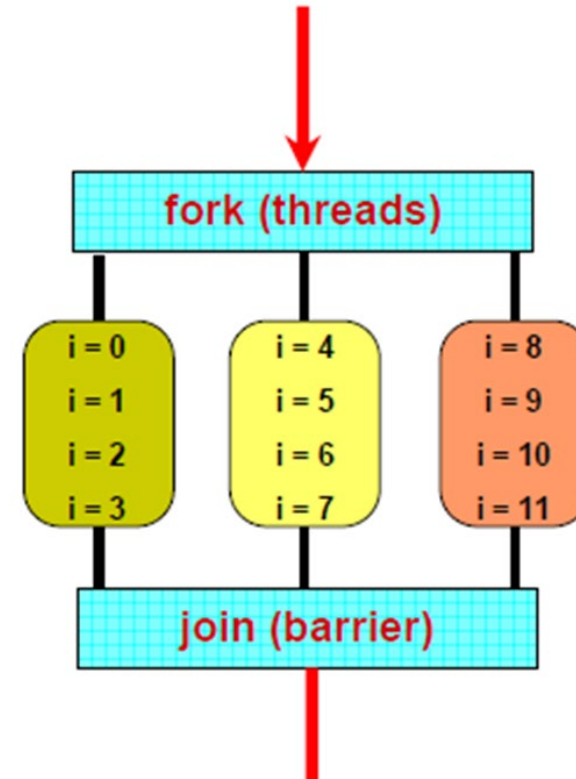
Independent
tasks

Dependent
tasks

Fork/Join and data parallelism in a shared memory computing system

```
for (i = 0; i < 12; i++)  
    C[i] = A[i] + B[i];
```

- Data parallelism
 - Perform the same computation but operate on different data
- A single process can fork multiple concurrent threads
 - Each thread encapsulates its own execution path
 - Each thread has local state and shared resources
 - Threads communicate through shared resources such as global memory



Static loop partitioning

```
#pragma omp for schedule(static)
{
    for (i=0; i<12; i++)
        a[i] = i;
}
```

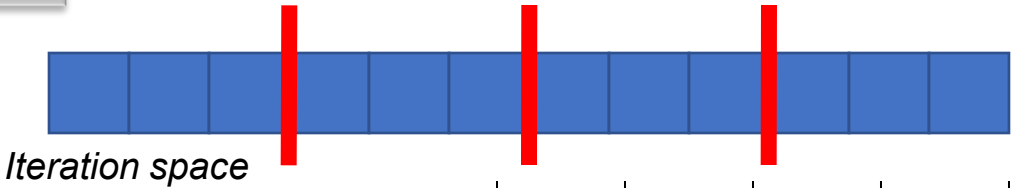
- Useful for:
- Simple, regular loops
 - Iterations with equal duration

Es. 12 iterations (N), 4 threads (N_{threads})

DATA CHUNK

$$Chunk = \left\lfloor \frac{N}{N_{threads}} \right\rfloor$$

3 iterations thread



LOWER BOUND

LB = C * TID

UPPER BOUND

UB = min {C * (TID + 1), N}

Thread ID (TID)	0	1	2	3
LB = C * TID	0	3	6	9
UB = min {C * (TID + 1), N}	3	6	9	12

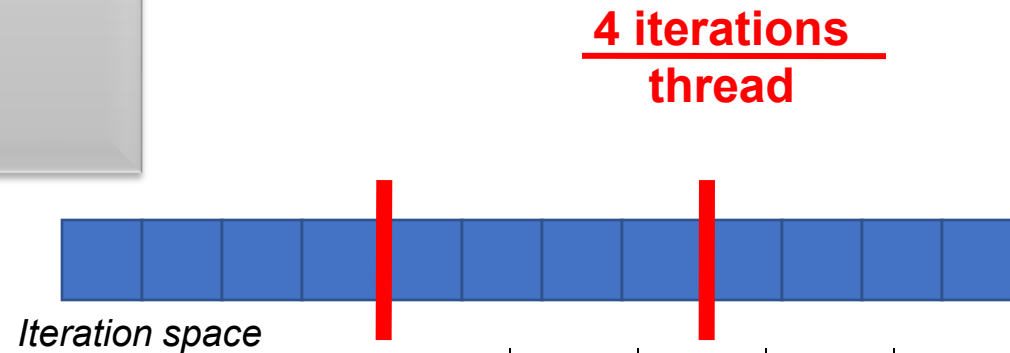
Static loop partitioning with user-defined chunk size (1)

Es. 12 iterations (N), 4 threads (N_{threads})

```
#pragma omp for schedule(static, 4)
{
    for (i=0; i<12; i++)
        a[i] = i;
}
```

Useful for:

- *Simple, regular loops*
- *Iterations with equal duration*
- *Additional constraints on iterations*



Thread ID (TID)	0	1	2	3
$LB = C * TID < N ? C * TID : -1$	0	4	8	-1
$UB = LB == -1 ? -1 : \min \{C * (TID + 1), N\}$	4	8	12	-1

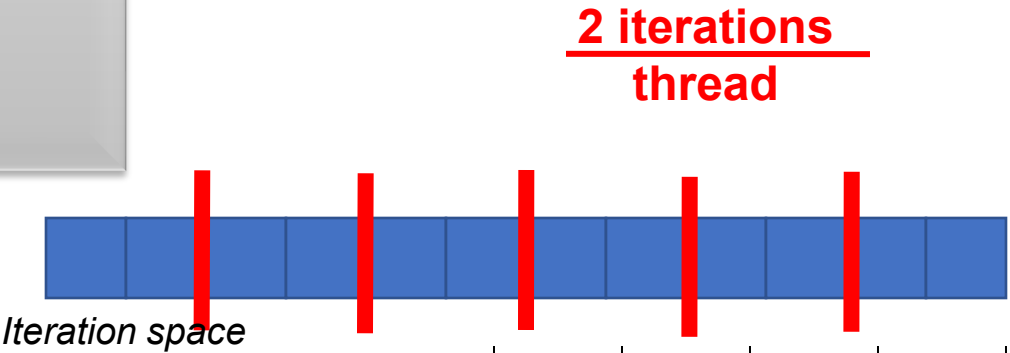
Static loop partitioning with user-defined chunk size (2)

Es. 12 iterations (N), 4 threads (N_{threads})

```
#pragma omp for schedule(static, 2)
{
    for (i=0; i<12; i++)
        a[i] = i;
}
```

Useful for:

- Simple, regular loops
- Iterations with equal duration
- Additional constraints on iterations



2 chunking iterations

Thread ID (TID)	0	1	2	3
$LB = C * TID < N ? C * TID : -1$	0	2	4	6
$UB = LB == -1 ? -1 : \min \{ C * (TID + 1), N \}$	2	4	6	8
$LB = C * (TID + N_{\text{threads}}) < N ? C * (TID + N_{\text{threads}}) : -1$	8	10	-1	-1
$UB = LB == -1 ? -1 : \min \{ C * (TID + N_{\text{threads}} + 1), N \}$	10	12	-1	-1

Overhead of static loop scheduling

- Chunk size: $Chunk = \left\lceil \frac{N}{N_{threads}} \right\rceil$

- Chunking iterations: $Chunk_{iters} = \left\lceil \frac{\left\lceil \frac{N}{N_{threads}} \right\rceil}{N_{threads}} \right\rceil$

- Overhead of static scheduling:

$$Ovh_{static-loop} = Chunk_{iters} * Ovh_{static-chunk}$$

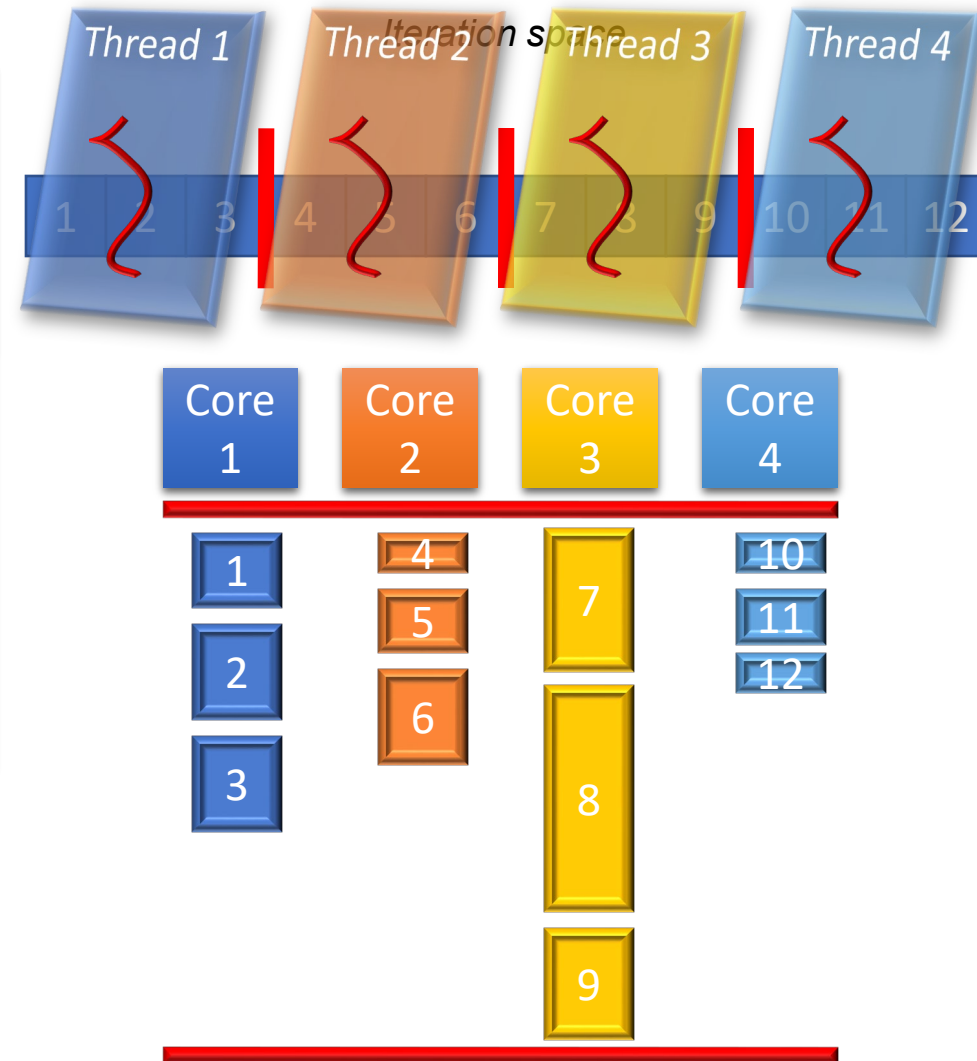
Static loop partitioning with unbalanced workload

```
#pragma omp for schedule(static)
{
    for (i=0; i<12; i++)
    {
        int start = rand();
        int count = 0;

        while (start++ < 256)
            count++;

        a[count] = foo();
    }
}
```

A variable amount of work
in each iteration → the
workload is unbalanced



Amdahl's Law: definition

- **Amdahl's Law:** *The performance improvement to be gained from using some faster mode of execution is limited by the fraction of the time the faster mode can be used*

$$Speedup = \frac{Time_{1core}}{Time_{ncores}} = \frac{1}{(1 - p) + \frac{p}{n}}$$

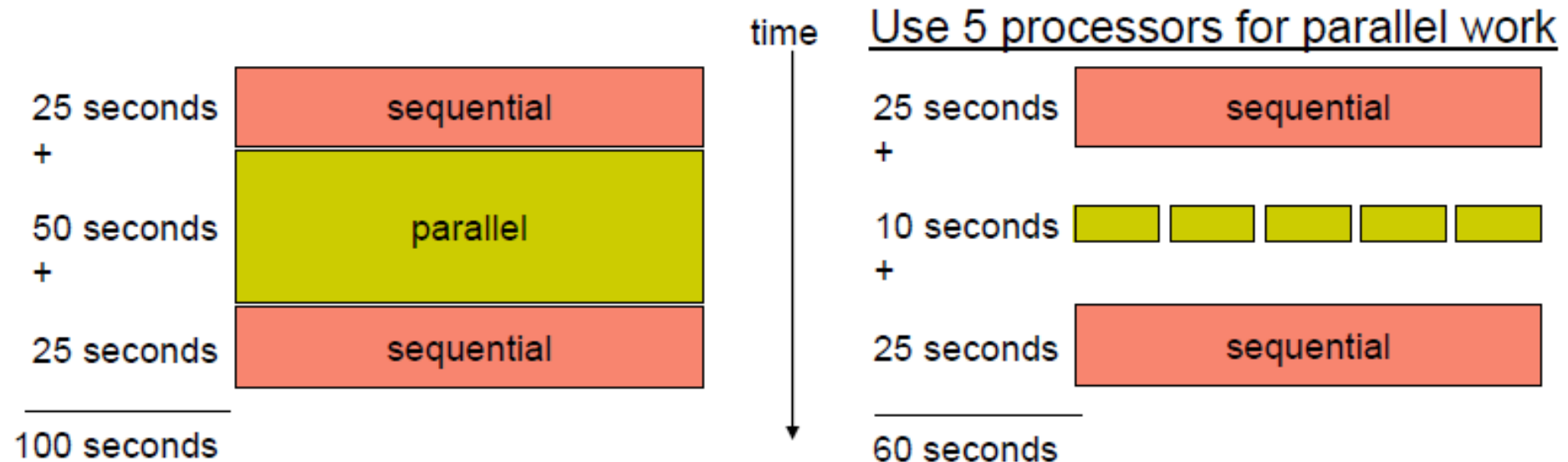
p → fraction that can run in parallel

1-p → fraction that must run serially

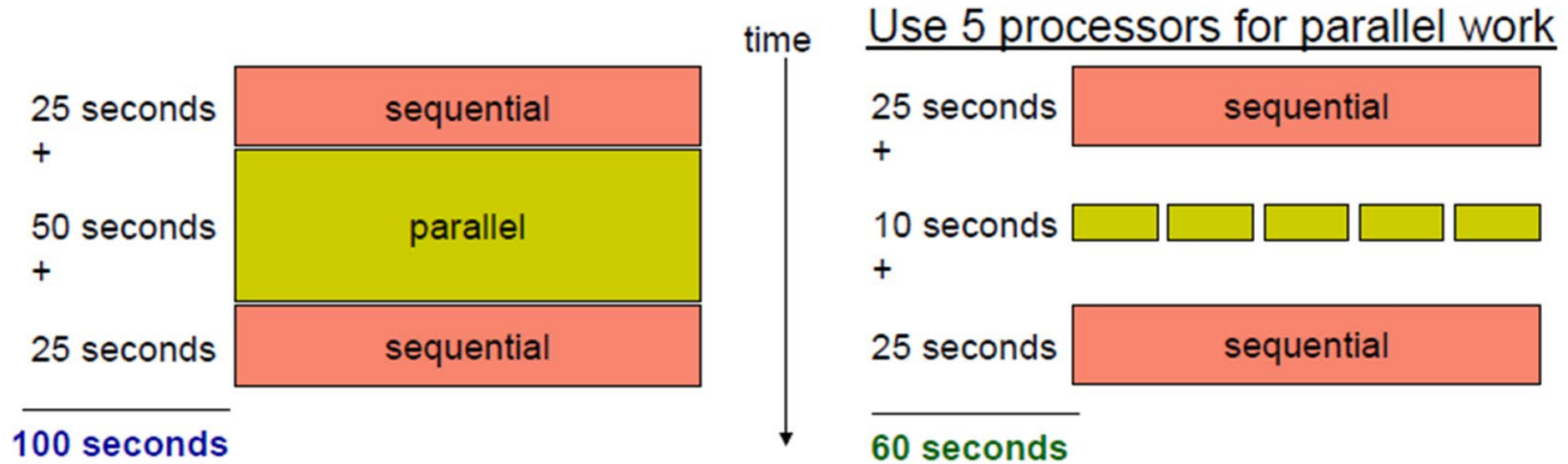
n → number of cores

Amdahl's Law: example (1/2)

- The expected performance speedup is defined by the fraction of code that can be parallelized (**parallel part**)



Amdahl's Law: example (2/2)



- Speedup = single-core time / multi-core time
= 100 seconds / 60 seconds
= 1.67

This means that the parallel version is 1.67 times faster

Applications of Amdahl's Law:

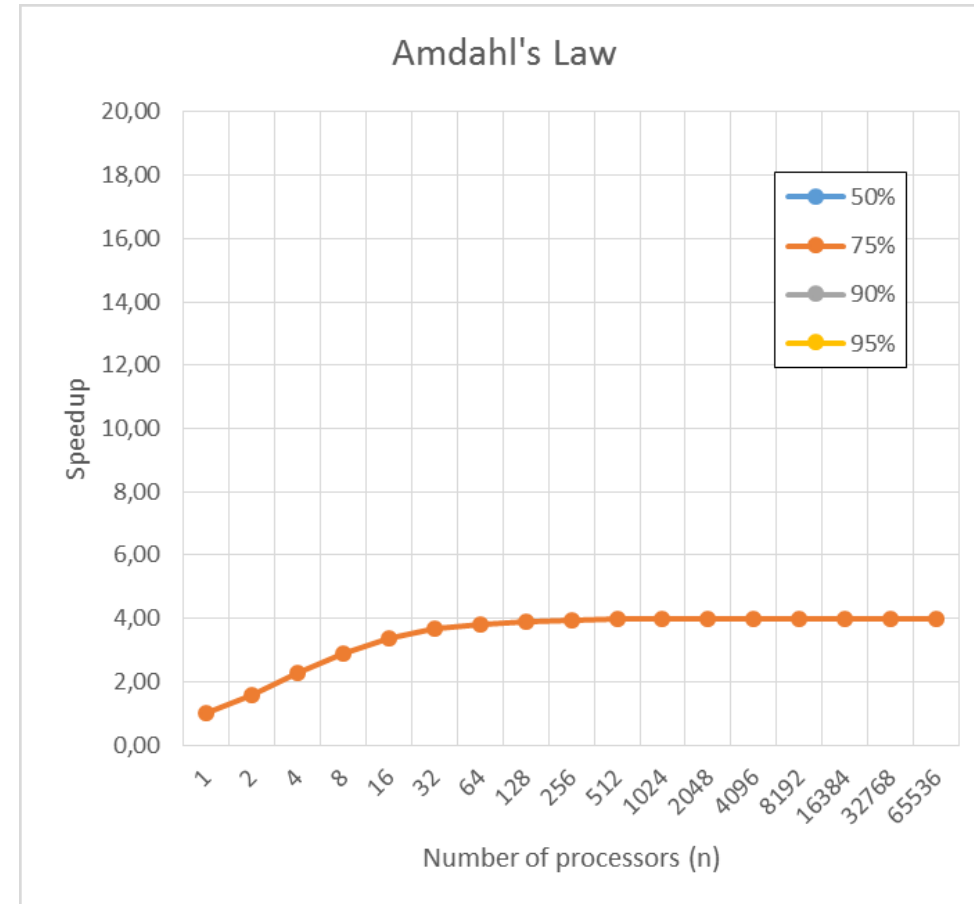
Computing the speedup for N cores

- Suppose that 75% of a program can be parallelized
- What is the speedup that could be achieved with 256 processors?

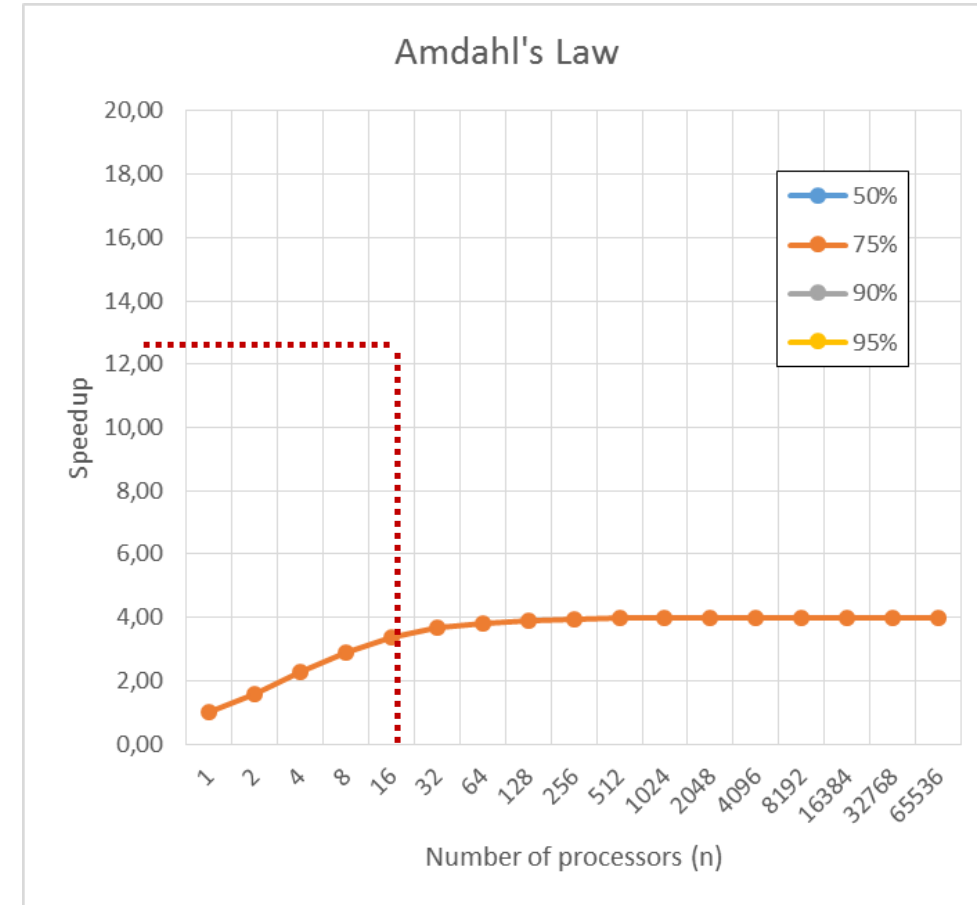
- $$S = \frac{1}{(1-0,75)+(\frac{0,75}{256})} = 3,95$$

- And for 65536 processors?

- $$S = \frac{1}{(1-0,75)+(\frac{0,75}{65536})} = 4$$

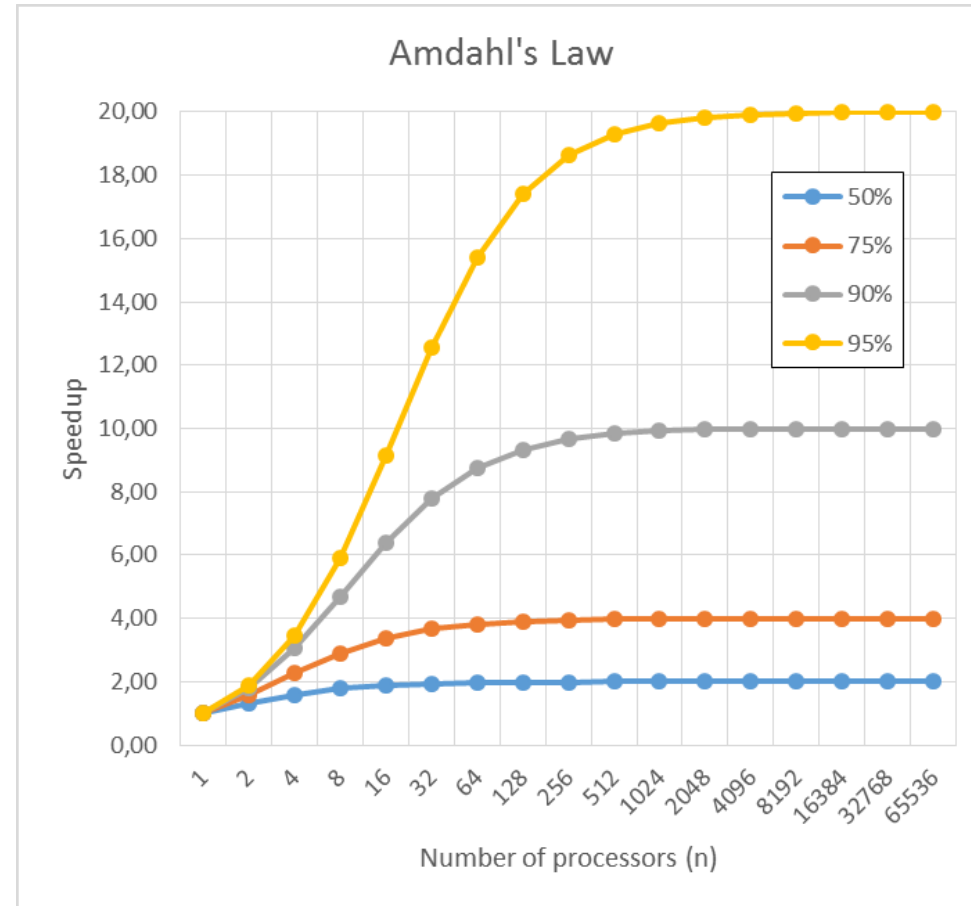


- Suppose that $n=32$
- Which p is required for $S=12.55$?
 - $\frac{1}{S} = 1 - p + \frac{p}{n}$
 - $\left(\frac{p}{n} - p\right) = \left(\frac{1}{S} - 1\right)$
 - $p \cdot \left(\frac{1}{n} - 1\right) = \left(\frac{1}{S} - 1\right)$
 - $p = \frac{1 - \frac{1}{S}}{1 - \frac{1}{n}}$
 - $p = \frac{0.92032}{0.96875} = 0.95001$ (**95%**)



Applications of Amdahl's Law: Maximum speedup

- If the portion of the program that can be parallelized is small, then the speedup is limited
- The non-parallel portion limits the performance
- The speedup tends to $1/(1-p)$ as the number of processors tends to infinity
- This is a theoretical limit → the number of active cores can be limited by the workload decomposition (*granularity*)



Analysis of parallel speedup

- Split in chunks of independent workload (easy for NN)
 - Per row in matrix-vector multiplication
 - Tiles in matrix-matrix multiplication
 - Per output channel in NN layer
 -
- Assign processors to execute disjoint sub-set of chunks
- Fork parallel threads and join at the end with a barrier
- Overheads
 - Banking conflicts in accessing memory
 - Fork cost, barrier cost
 - Workload unbalance (all waiting at the barrier for the slowest one)
- Speedup for a parallel loop:
- If there is a non parallelizable part

$$\frac{N_{cycles-sequential}}{N_{cycles-parallel}} = \frac{N_{cycles-sequential}}{\max(N_{chunks/processor}) N_{cycles/chunk} + C_{fork} + C_{join}}$$

$$\frac{N_{cycles-sequential}}{N_{cycles-parallel}} = \frac{N_{cycles-parallelizable} + N_{cycles-nonpar}}{N_{cycles-nonpar} + \max(N_{chunks/processor}) N_{cycles/chunk} + C_{fork} + C_{join}}$$

Exercise (1/2)

```
int a[100], b[100], c[100];  
work1(a, b, c); // 50 us (sequential part)  
  
for(i=0; i<100; i++)  
    work2(a, b, c, i); // 25 us, c[i] = f(a[i], b[i])  
  
work3(a, b, c); // 50 us (sequential part)
```

Compute the maximum theoretical speedup applying Amdahl's Law



Exercise (2/2)

T_{seq_part} → time to execute the sequential parts

T_{par_part} → time to execute the parallel part (on a single core)

$$p = \frac{T_{par_part}}{T_{seq_part} + T_{par_part}} = \frac{25 * 100}{(50 + 50) + 25 * 100} = 0.962$$

$$S_{ideal} = \frac{1}{1 - p} = 26.32$$

How many cores are required to achieve this speedup?

- 25 cores → $S = 12.76$
- 100 cores → $S = 20.16$
- 1000 cores → $S = 24.41$ *Does this really make sense in the program???*
→ *NO, because we have 100 loop iterations*



Overhead of parallelism

- Given enough parallel work, this is the biggest barrier to getting desired speedup
- Parallelism overheads include:
 - starting a thread
 - data dependencies
 - synchronizing
 - communicating shared data
 - extra (redundant) computation
 - Each of these can take a very large amount of time if a parallel program is not carefully designed
 - **Tradeoff:** Algorithm needs sufficiently large units of work to run fast in parallel (i.e., large **granularity**), but not so large that there is not enough parallel work



Exercise (1/2)

```
int a[100], b[100], c[100];  
work1(a, b, c); // 50 us (sequential part)
```

```
for(i=0; i<100; i++)  
    work2(a, b, c, i); // 25 us (ok for data parallelism)
```

```
work3(a, b, c); // 50 us (sequential part)
```

Parallel overheads:

- 1.2 μ s to create each thread
- 0.5 μ s to synchronize all threads after execution

Find the number of cores to achieve the maximum speedup



Exercise (2/2)

$n \rightarrow$ number of cores available to execute the parallel version of the code

$T_{seq_part} \rightarrow$ time to execute the sequential parts

$T_{par_part} \rightarrow$ time to execute the parallel part on a single core

$T_{par} \rightarrow$ time to execute the full program on n cores

$T_{n_ovh} \rightarrow$ time to execute the overheads which are proportional to n

$T_{const_vh} \rightarrow$ time to execute the constant overheads (not depending on n)

$$T_{par} = T_{seq_part} + \frac{T_{par_part}}{n} + T_{n_ovh} \times n + T_{const_ovh}$$

Maximizing the speedup is equivalent to minimizing T_{par}

$$-\frac{T_{par_part}}{n^2} + T_{ovh1} = 0 \rightarrow n = \left\lceil \sqrt{\frac{T_{par_part}}{T_{n_ovh}}} \right\rceil = \left\lceil \frac{2500}{1.2} \right\rceil = 46$$

Now the speedup is 12.42

$\lceil x \rceil \rightarrow$ ceiling (rounding to the upper integer)

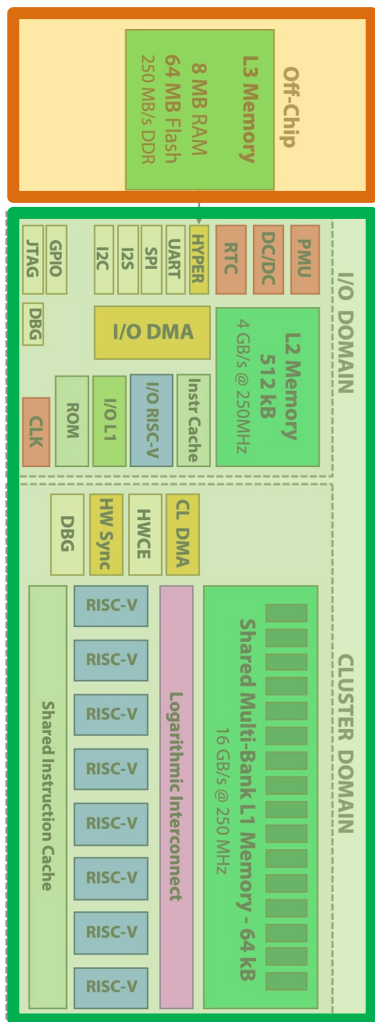


The Quest for Efficient Data Movements: Introduction to Tensor Tiling

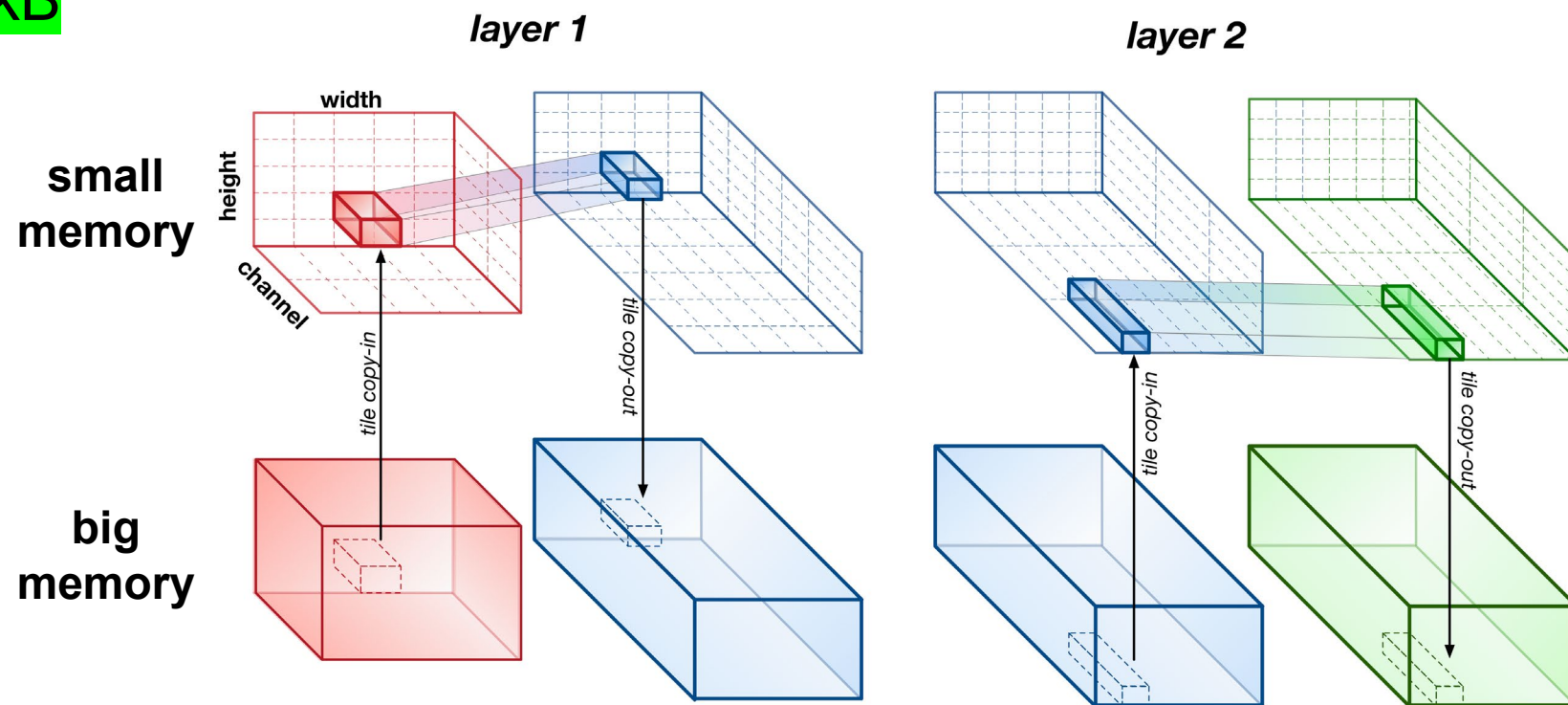
Explicit Memory Management: MobileNet Example



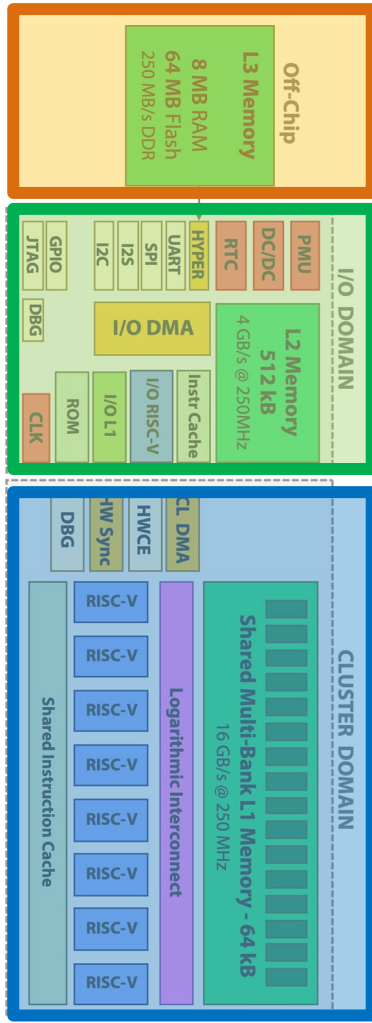
Tensor tiling



L3 / L2 tiling
64 MB / 512 kB



Tensor tiling

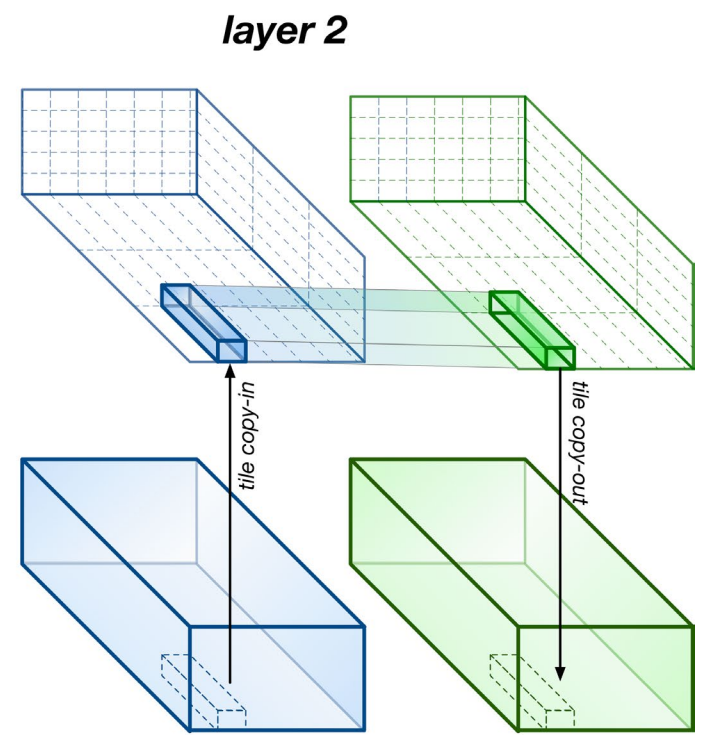
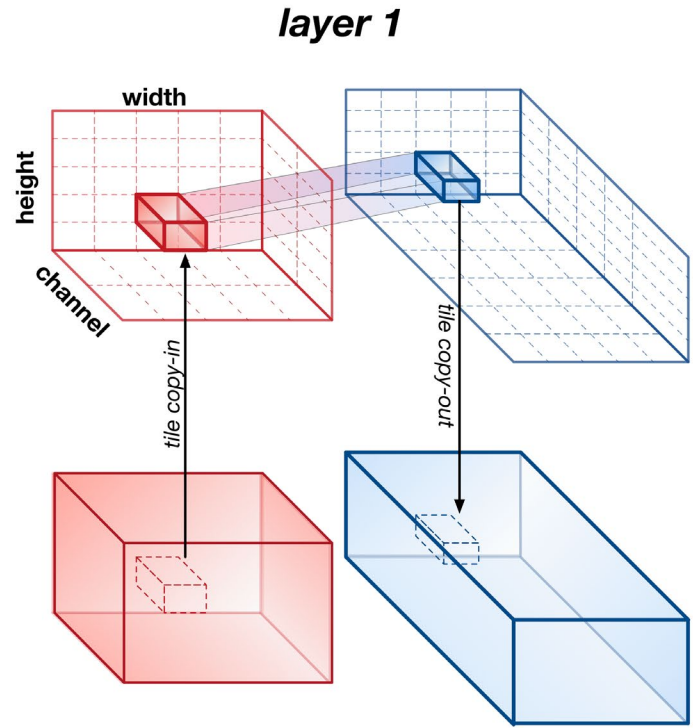


L3 / L2 tiling
64 MB / 512 kB

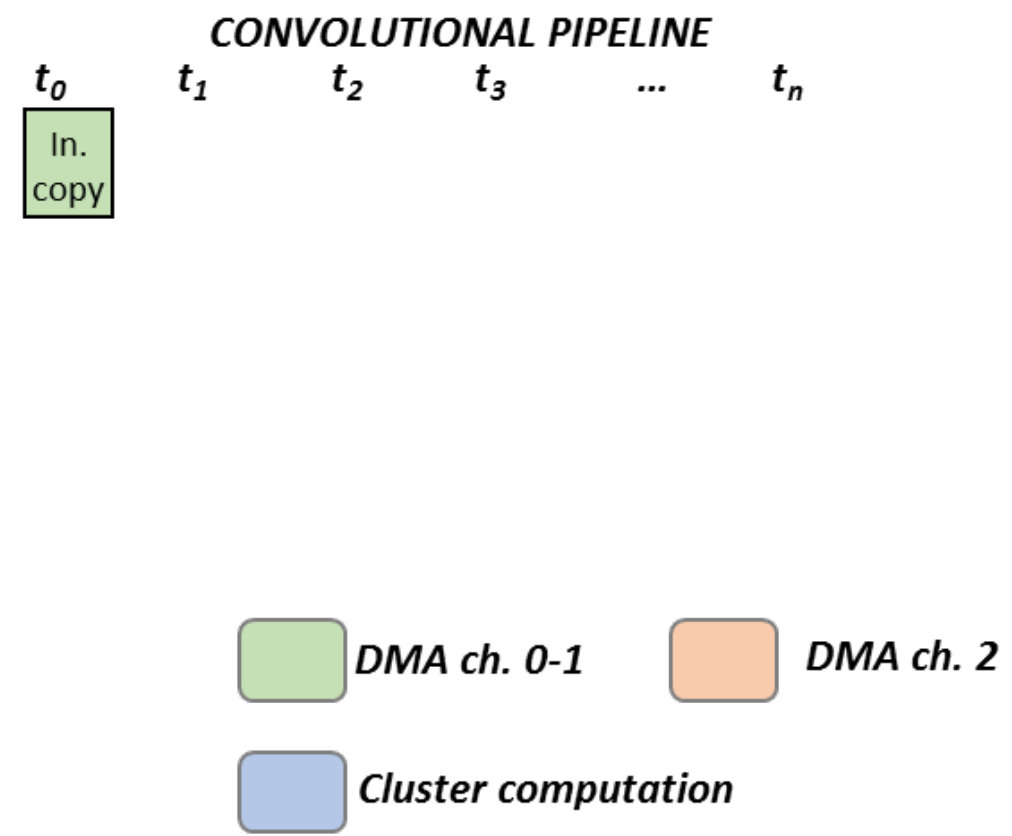
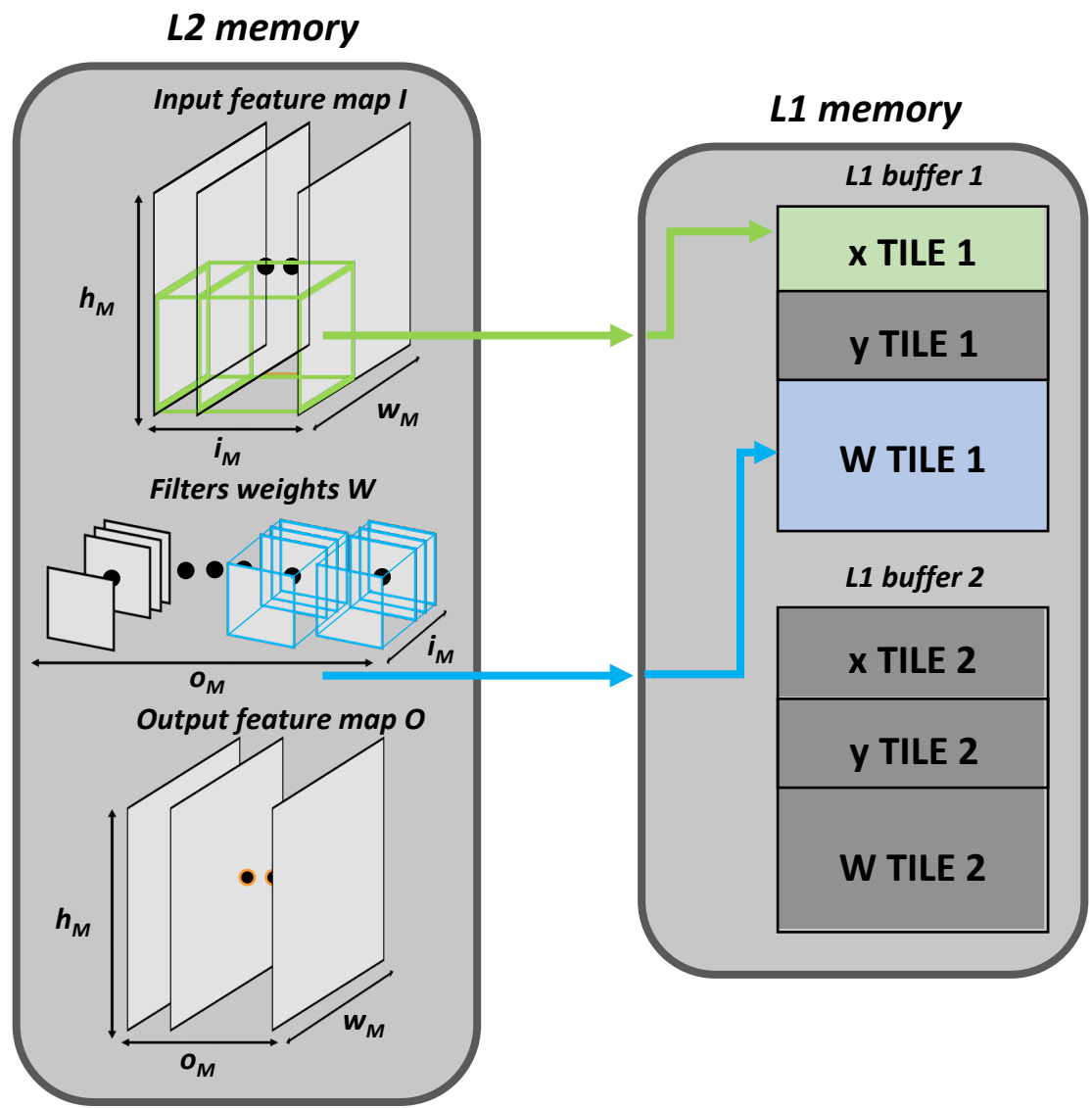
L2 / L1 tiling
512 kB / 64 kB

small
memory

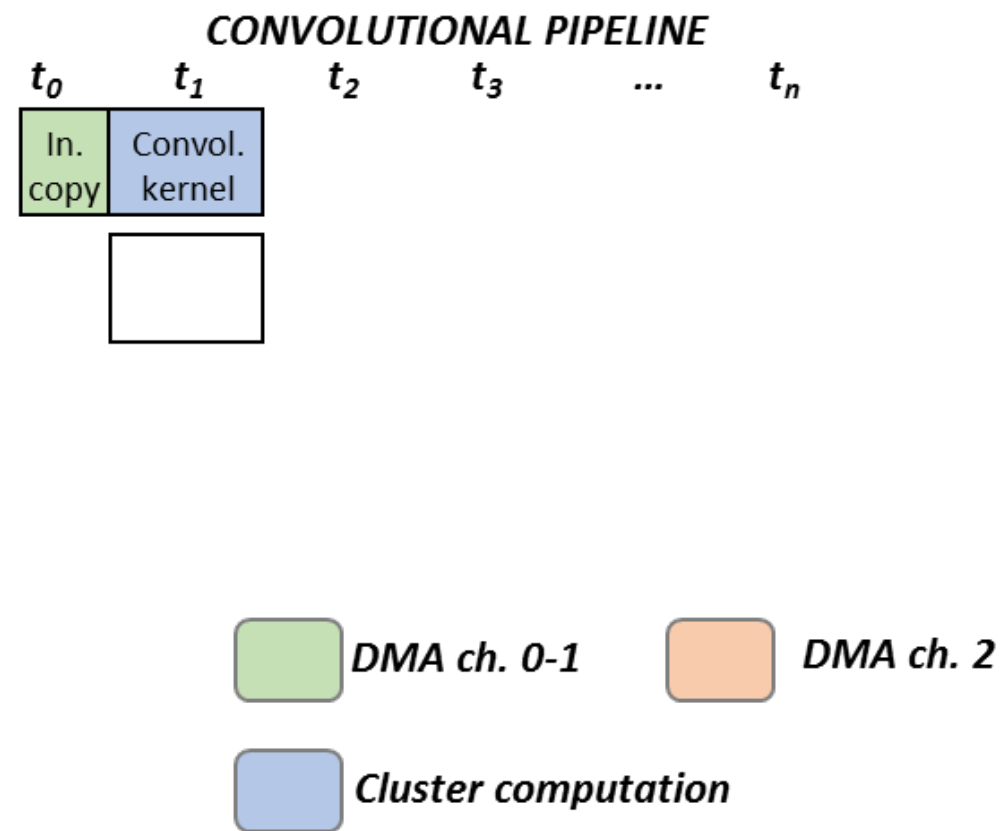
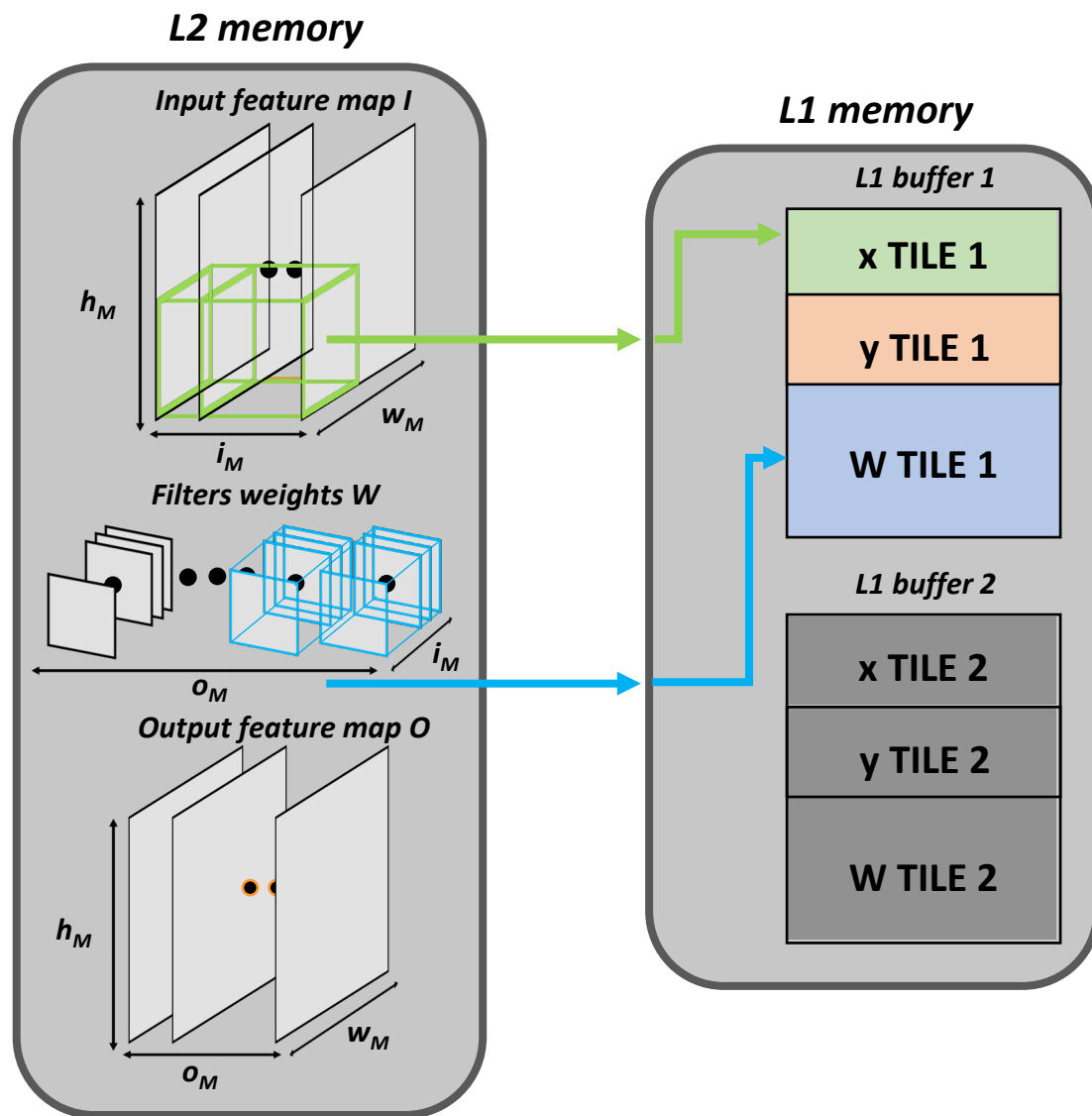
big
memory



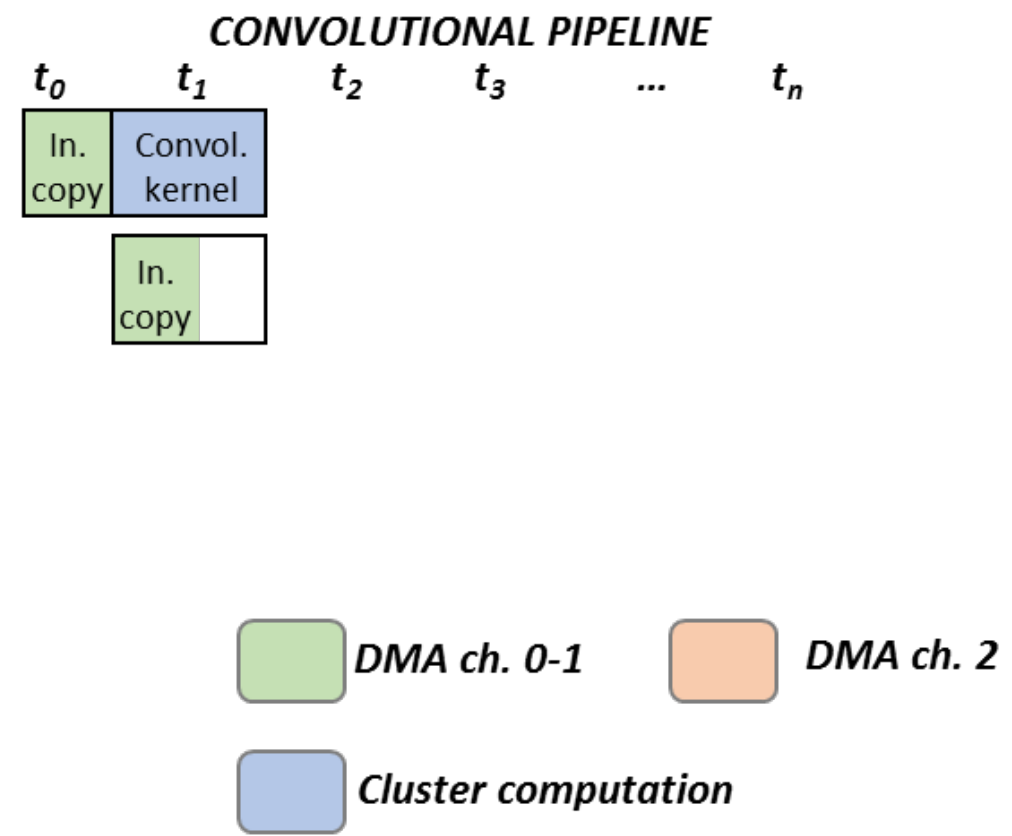
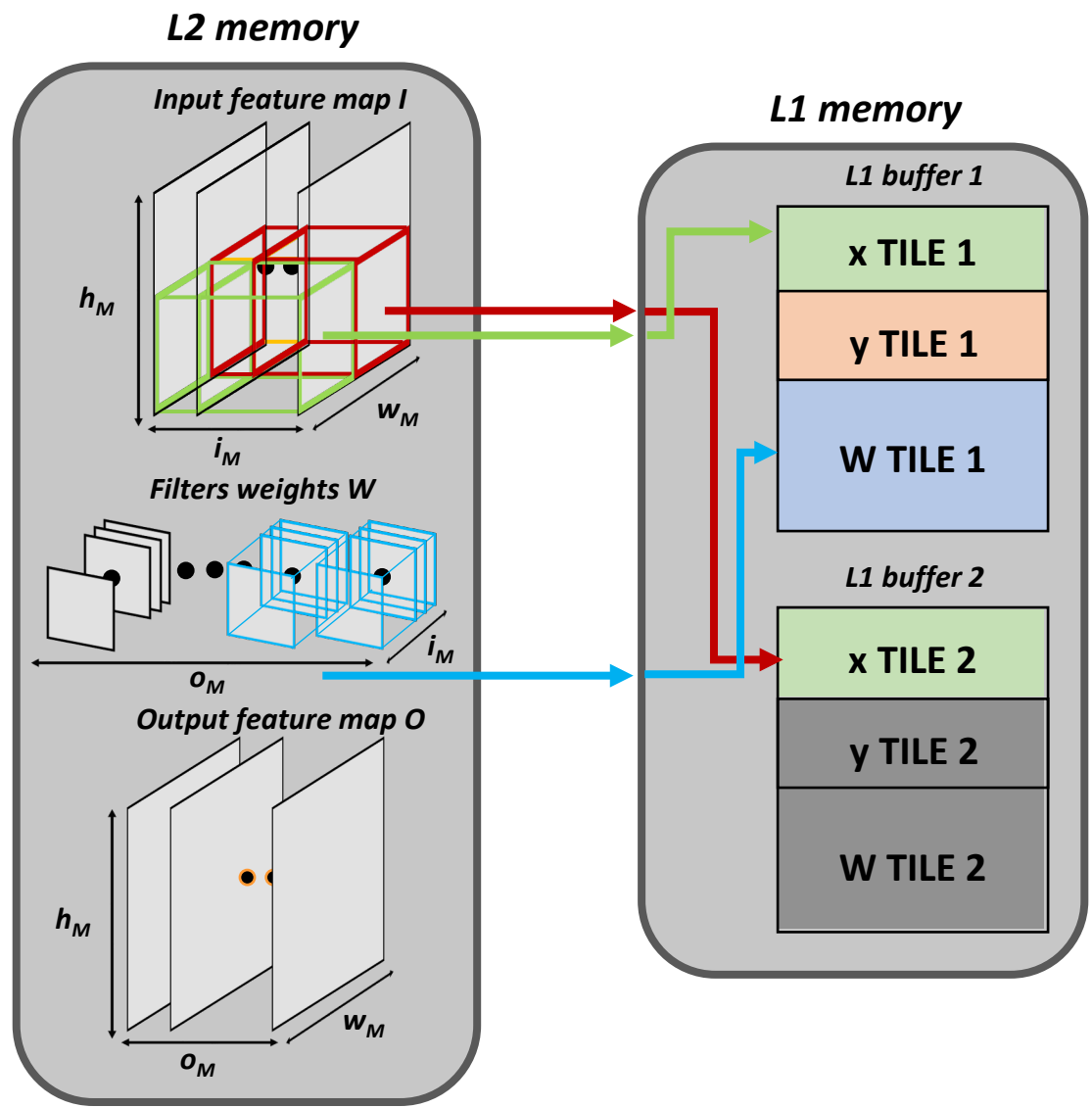
Tile Data Movement



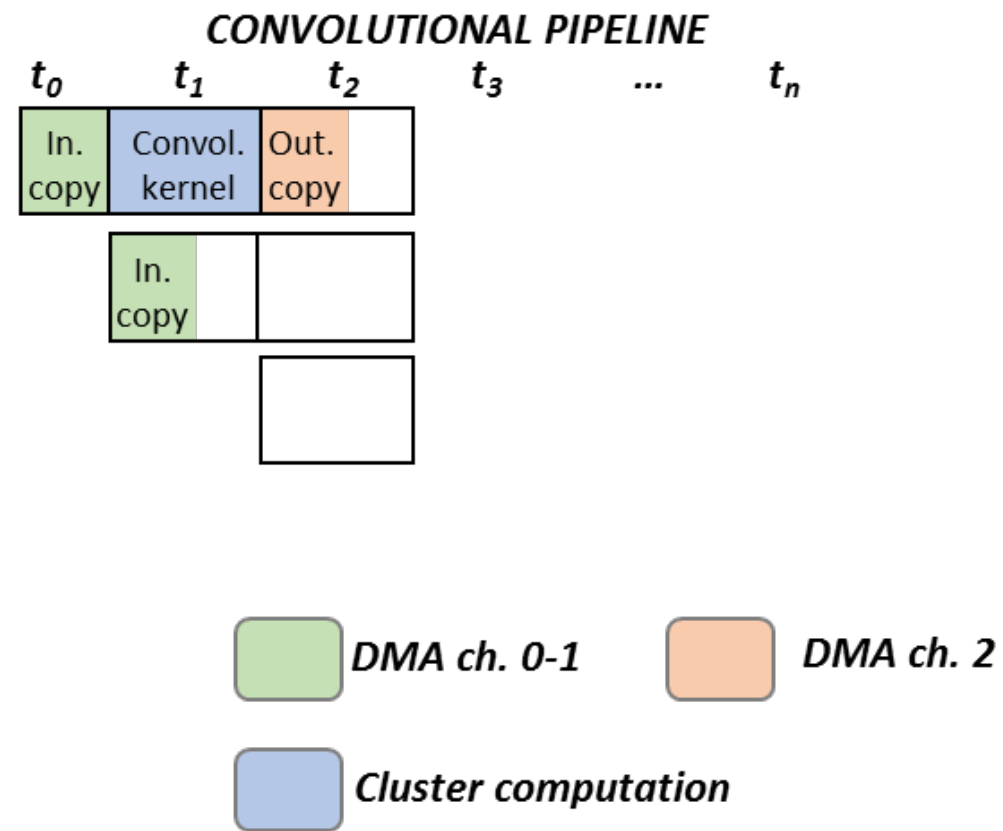
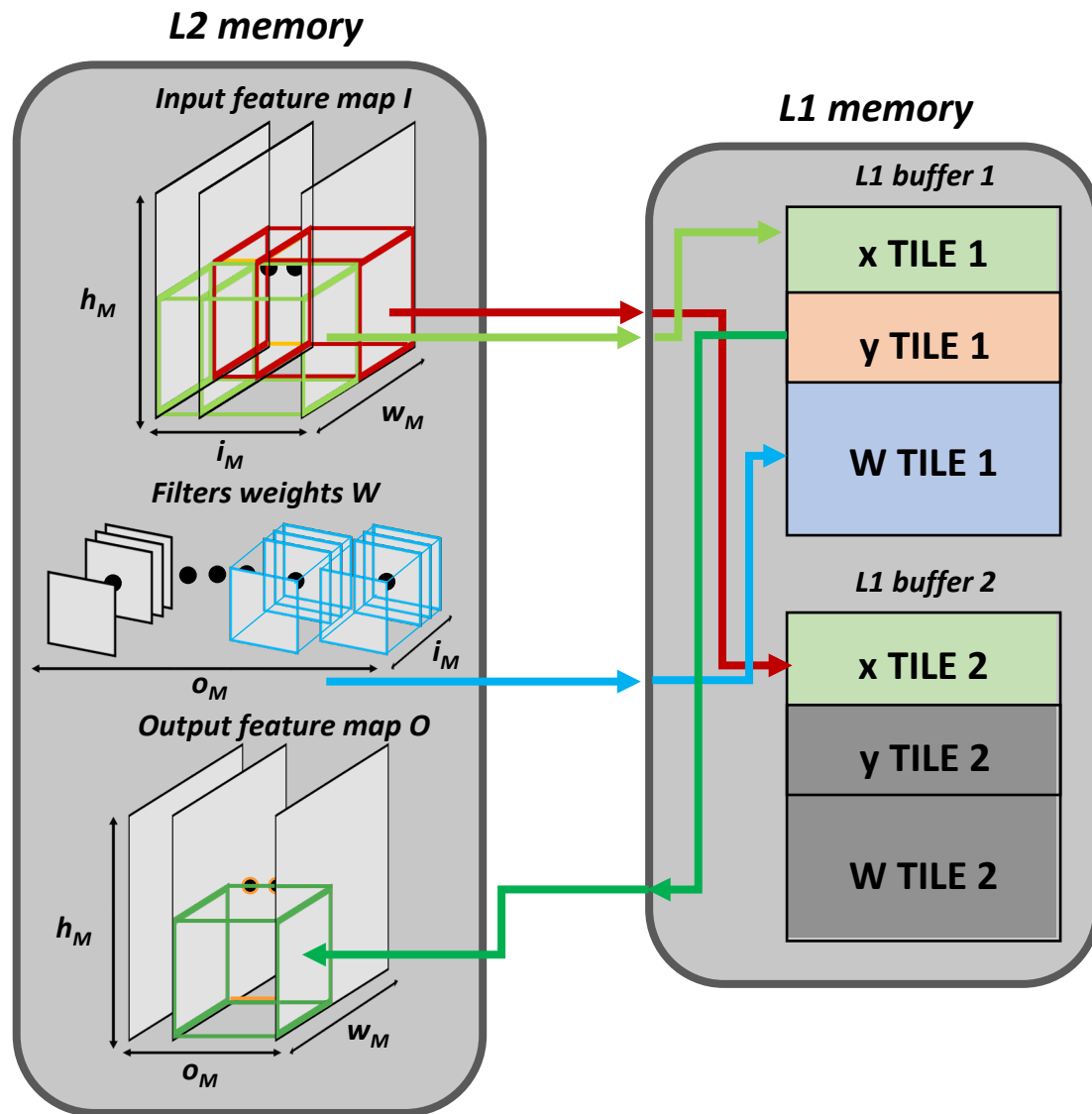
Tile Data Movement



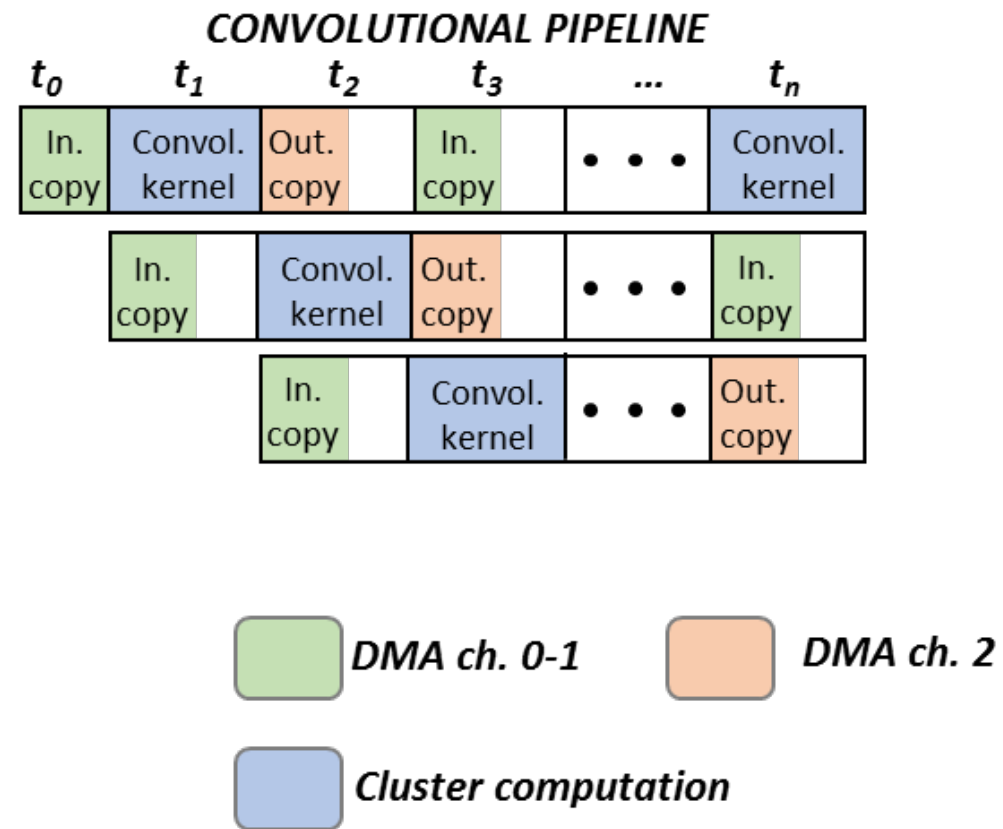
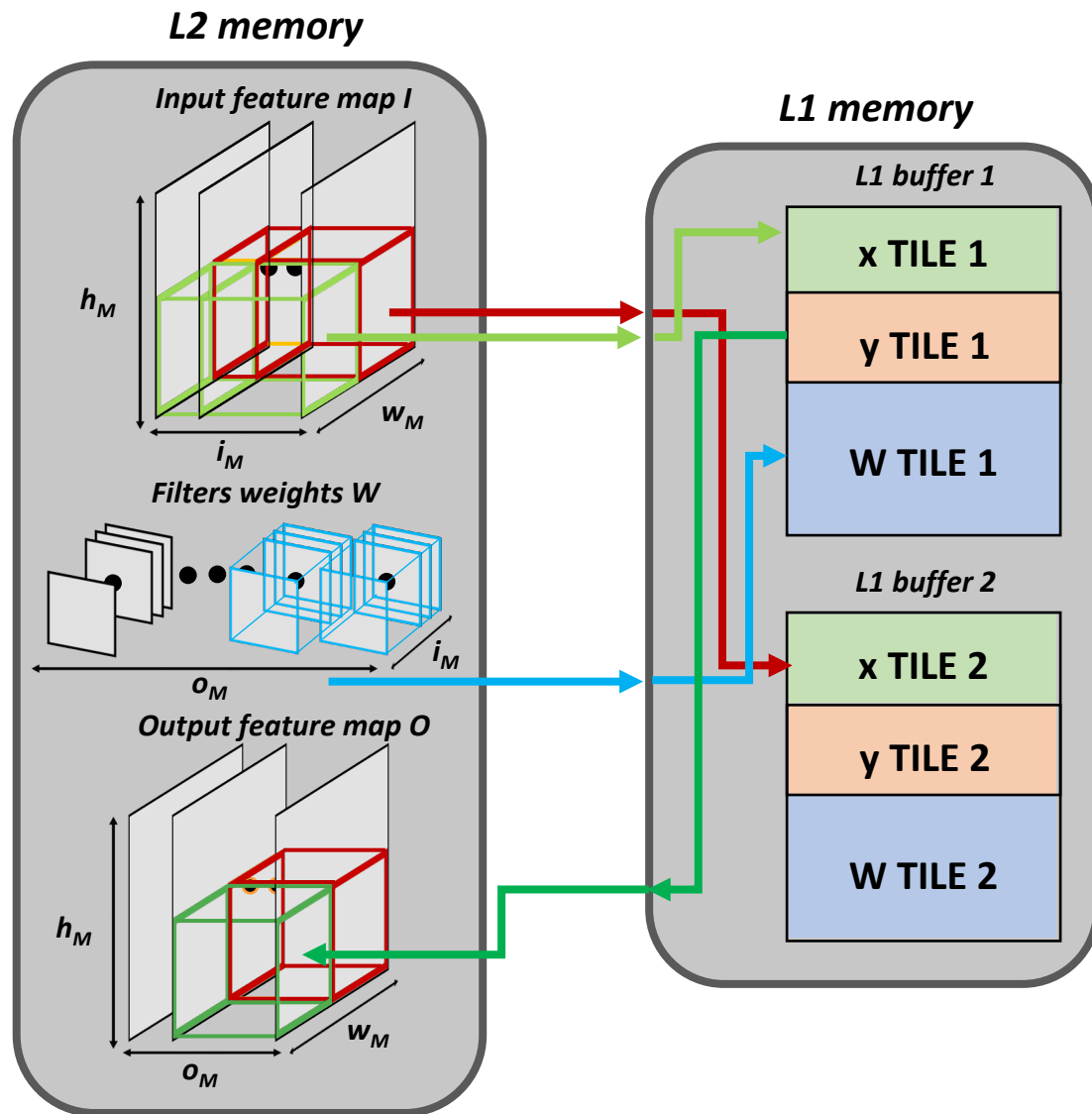
Tile Data Movement



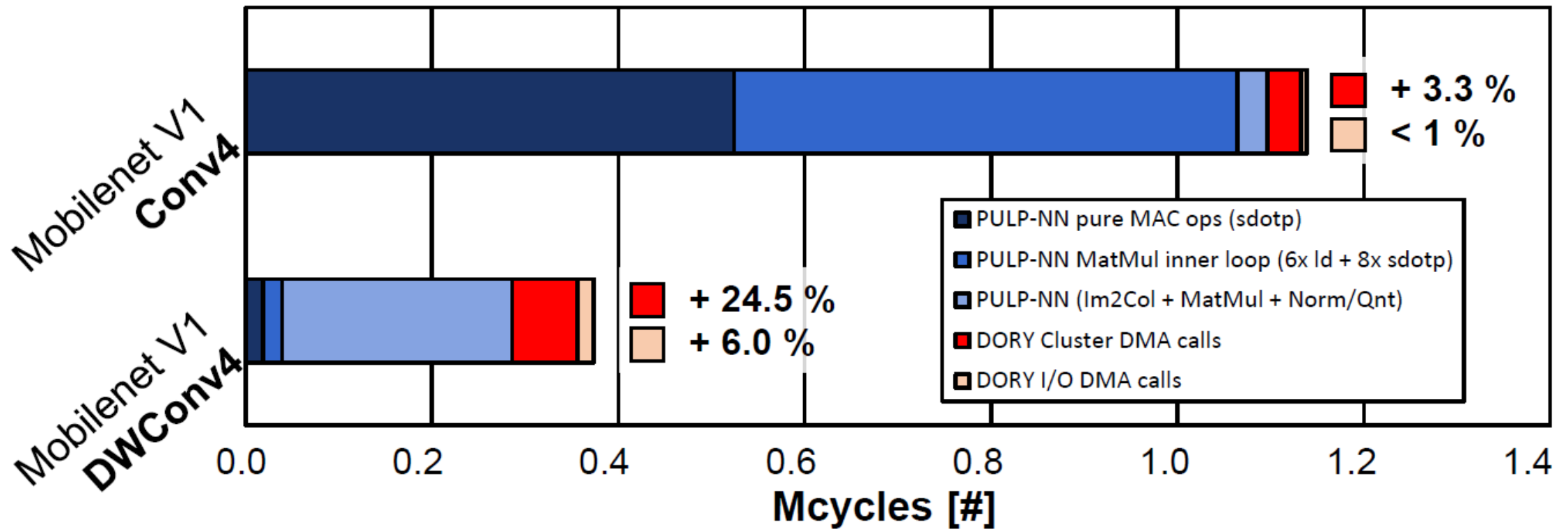
Tile Data Movement



Tile Data Movement



Tiling Overhead on MobilenetV1



A. Burrello, A. Garofalo, N. Bruschi, G. Tagliavini, D. Rossi and F. Conti, "DORY: Automatic End-to-End Deployment of Real-World DNNs on Low-Cost IoT MCUs," in *IEEE Transactions on Computers*, vol. 70, no. 8, pp. 1253-1268, 1 Aug. 2021.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Credits

Prof. Francesco Conti's Teaching Material on ADCs for APSEI23 (Unibo)

ARM Selected Courses

Wikipedia